

TƯ DUY VÀ NỀN TẢNG



TÓM TẮT

Phần A không dạy một thuật toán nào. Nó dựng bốn thứ mà mọi chương sau đều dùng: một quy trình để đối mặt với bài lạ (chương 1), kinh nghiệm luyện tập đã được kiểm chứng của những người giỏi nhất (chương 2), thước đo chi phí để biết lời giải có đủ nhanh không (chương 3), công cụ chứng minh để biết lời giải có đúng không (chương 4), và bộ toán tối thiểu quay lại nhiều lần nhất (chương 5). Đọc xong phần này, bạn có một quy trình để chạy khi không biết làm gì, và hai thước đo để tự chấm lời giải của mình trước khi gõ code. Nếu chỉ nhớ một điều: người giải giỏi không khác người giải kém ở lượng thuật toán thuộc lòng, mà ở khả năng tự giám sát — biết mình đang đi hướng nào, đi bao lâu rồi, và khi nào phải bỏ hướng đó.

Có một lý do rất cụ thể để phần này đứng trước tất cả. Khi bạn bí một bài, cái thiếu hầu như không bao giờ là một thuật toán bạn chưa từng nghe tên. Cái thiếu là một trong ba thứ: bạn chưa hiểu đúng đề, bạn chưa thử đủ ví dụ nhỏ để thấy cấu trúc, hoặc bạn đang đi một hướng sai mà không chịu bỏ. Cả ba đều thuộc về quy trình chứ không thuộc về kiến thức, và cả ba đều luyện được — đó chính là nội dung của chương 1 và chương 2.

Chương 3 và chương 4 là hai cái thước. Không có thước đo chi phí, bạn không biết ý tưởng vừa nghĩ ra có đáng gõ hay không, và bạn sẽ mất hàng giờ cài một lời giải $O(n^2)$ cho $n = 10^6$. Không có thước đo tính đúng, bạn viết code rồi tin vào vài ví dụ tự bịa, và cái sai chỉ lộ ra ở test 47. Hai chương này dạy cách trả lời hai câu hỏi *trước khi gõ dòng code đầu tiên*: bài này cho phép độ phức tạp nào, và lời giải của tôi giữ bất biến gì.

Chương 5 khác ba chương kia: nó là kho tra cứu chứ không phải mạch đọc. Ở đó có tổng, hệ thức truy hồi, tổ hợp, xác suất và một ít đại số — đúng những mảnh toán quay lại nhiều nhất trong phần B, C và D. Đừng đọc một mạch; đọc lướt để biết trong đó có gì, rồi quay lại đúng mục khi một chương sau cần đến.

Bản đồ phần A — Tư duy và nền tảng

TÓM TẮT

Phần A gồm năm chương chia theo *câu hỏi chúng trả lời*, không theo môn học. Chương 1 trả lời “bí thì làm gì”; chương 2 trả lời “người giỏi luyện thế nào”; chương 3 trả lời “lời giải này có đủ nhanh không”; chương 4 trả lời “lời giải này có đúng không”; chương 5 là kho toán tra khi cần. Chuỗi bắt buộc chỉ là $1 \rightarrow 3 \rightarrow 4$. Chương 2 đọc được bất cứ lúc nào và nên đọc sớm, chương 5 không đọc tuần tự.

A. Phần này có gì

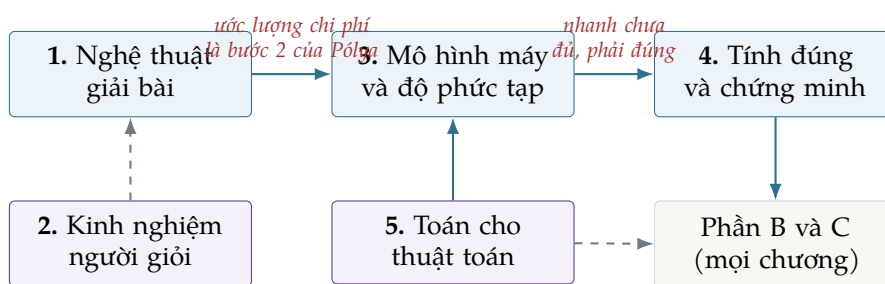
- **01-nghe-thuat-giai-bai** — bốn bước Pólya áp vào lập trình, bộ heuristic sinh ý tưởng, mô hình Schoenfeld về tự giám sát, cách hỏi đúng câu hỏi khi bí.
- **02-kinh-nghiem-nguoi-gioi** — khảo sát cách luyện của Knuth, Dijkstra, Tarjan, Bentley, Skiena, Erik Demaine, các kỳ thủ lập trình hàng đầu; bản đồ sách kinh điển; luyện tập có chủ đích và nhật ký lỗi.
- **03-mo-hinh-may-va-do-phuc-tap** — mô hình RAM, ký hiệu tiệm cận, hệ thức truy hồi, phân tích khâu hao, hằng số thật, phân cấp bộ nhớ, cách đọc giới hạn đề bài ra độ phức tạp cho phép.
- **04-tinh-dung-va-chung-minh** — bất biến vòng lặp, quy nạp, tính dừng, lập luận đổi chỗ, phản chứng, phản ví dụ, và kỷ luật viết chương trình cùng lúc với chứng minh của Dijkstra và Gries.

- **05-toan-cho-thuat-toan** — tổng và chặn, tổ hợp đếm, xác suất và kỳ vọng, số học modulo, đại số tuyến tính rời rạc, lý thuyết trò chơi nhỏ.

B. Vì sao chia như vậy

Một giáo trình thường gộp “toán rời rạc” và “phân tích thuật toán” thành một khối lớn đặt ở đầu sách, và người đọc bỏ dở ở khối đó. Cây này tách thành ba vai trò khác nhau, vì chúng được dùng vào ba thời điểm khác nhau trong quá trình giải. Mô hình chi phí (chương 3) dùng *trước* khi gõ code, để loại các hướng chắc chắn quá chậm. Công cụ chứng minh (chương 4) dùng *trong lúc* nghĩ, vì bất biến vừa là công cụ chứng minh vừa là công cụ sinh ý tưởng. Toán nền (chương 5) dùng *khi cần*, rải rác. Gộp cả ba vào một chương sẽ che mất sự khác nhau đó.

C. Phụ thuộc và thứ tự đọc



Hình 1. Phụ thuộc giữa năm chương của phần A. Chuỗi ngang $1 \rightarrow 3 \rightarrow 4$ là bắt buộc. Hai chương tô tím vào được bất cứ lúc nào: chương 2 là kinh nghiệm luyện tập, chương 5 là kho toán tra cứu.

D. Cốt lõi hay tham khảo

Cốt lõi: toàn bộ chương 1; mục “đọc giới hạn đề bài ra độ phức tạp” và mục phân tích khấu hao của chương 3; mục bất biến vòng lặp và lập luận đổi chỗ của chương 4; mục tổng–chặn và kỳ vọng tuyến tính của chương 5. **Tham khảo:** phần chứng minh hình thức kiểu Hoare ở chương 4, phần giải hệ thức truy hồi bằng hàm sinh ở chương 5.

Nghệ thuật giải bài (The Art of Problem Solving)

PHỤ THUỘC không — đây là chương mở đầu. Mọi chương sau đều giả định bạn đã có quy trình bốn bước và bộ heuristic của chương này trong đầu.

TÓM TẮT

Chương này dựng một quy trình để chạy khi bạn đứng trước một bài chưa biết làm: hiểu đề cho thật kỹ, tìm mối nối với thứ đã biết, thực hiện, rồi nhìn lại. Bốn bước đó là của Pólya, viết cho toán học năm 1945, và chúng áp thẳng vào lập trình mà gần như không phải sửa. Phần thứ hai của chương là bộ heuristic sinh ý tưởng — mười hai câu hỏi cụ thể để hỏi khi bí, mỗi câu kèm ví dụ thuật toán mà nó từng đẻ ra. Phần thứ ba là điều Schoenfeld phát hiện khi quay video người giải bài: cái phân biệt người giỏi với người kém không phải lượng kiến thức mà là *khả năng tự giám sát*. Nếu chỉ nhớ một điều: khi bí, đừng cố nghĩ mạnh hơn — hãy đổi câu hỏi bạn đang tự hỏi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán (problem)** — một tình huống trong đó bạn biết mình muốn gì nhưng chưa biết cách đi tới đó. Nếu bạn đã biết cách đi thì đó không còn là bài toán, chỉ là bài tập.
- **heuristic** — một mẹo tìm đường, không đảm bảo thành công nhưng làm tăng xác suất tìm ra ý tưởng. Khác với thuật toán ở chỗ nó không hứa hẹn gì cả.
- **vét cạn (brute force)** — cách giải duyệt qua mọi khả năng. Luôn đúng, thường quá chậm, nhưng là điểm xuất phát của gần như mọi lời giải nhanh.
- **bất biến (invariant)** — một mệnh đề luôn đúng tại một điểm nhất định trong thuật toán, chẳng hạn “sau mỗi vòng lặp, đoạn đầu của mảng đã sắp xếp”. Chương [A/04](#) nói kỹ.
- **độ phức tạp thời gian** — cách nói “khi dữ liệu lớn gấp đôi thì thời gian chạy tăng bao nhiêu lần”. Chương [A/03](#) nói kỹ.
- **ví dụ nhỏ nhất** — trường hợp nhỏ nhất mà vẫn còn giữ được cái khó của bài. Công cụ mạnh nhất và bị bỏ qua nhiều nhất trong chương này.

CÂU HỎI MỞ ĐẦU

- Q1. Hai người cùng biết đúng một lượng thuật toán như nhau, vì sao một người giải được bài mới còn người kia thì không?
- Q2. Vì sao lời khuyên “hãy nghĩ kỹ hơn” gần như vô dụng khi bạn đang bí, còn lời khuyên “hãy vẽ ví dụ $n = 4$ ra giấy” thì thường hiệu quả?
- Q3. Vét cạn là cách giải ai cũng nghĩ ra và luôn quá chậm. Vậy vì sao chương này lại bảo bạn phải viết nó ra trước?
- Q4. Bước “nhìn lại” của Pólya là bước bị bỏ qua nhiều nhất. Bỏ nó đi thì bạn mất chính xác cái gì?
- Q5. Khi nào thì việc bỏ hướng đang đi là quyết định đúng, và làm sao biết được điều đó *trước khi* hết giờ?

Hãy bắt đầu bằng một cảnh quen thuộc. Bạn đọc một đề bài, hiểu từng câu, và rồi không biết làm gì tiếp. Không phải bạn thiếu kiến thức — nếu ai đó nói “bài này dùng quy hoạch động” thì bạn làm được ngay. Cái thiếu nằm ở chỗ khác: bạn không có *quy trình* để đi từ đề bài tới cái gọi ý

đó. Trong lúc bí, đầu bạn chạy một vòng lặp vô ích — đọc lại đề, thấy vẫn thế, thử nhớ xem bài này giống bài nào từng gặp, không nhớ ra, đọc lại đề. Vòng lặp đó không hội tụ vì nó không thay đổi thông tin bạn có.

Điều đáng nói là tình trạng này đã được nghiên cứu nghiêm túc, và không phải bởi dân lập trình. Năm 1945, nhà toán học George Pólya viết *How to Solve It*[1], một quyển sách mỏng cố gắng trả lời câu hỏi “khi một người giỏi giải được một bài mới, trong đầu họ thực sự diễn ra chuyện gì”. Bốn mươi năm sau, Alan Schoenfeld làm một việc tàn nhẫn hơn: ông đặt camera quay lại quá trình giải toán của sinh viên và của các nhà toán học, rồi đối chiếu[2]. Kết quả của cả hai người, cộng với nghiên cứu về luyện tập có chủ đích của Ericsson[3], tạo thành nền cho chương này. Chúng ta sẽ dùng chúng theo nghĩa rất thực dụng: đây là quy trình để chạy khi không biết làm gì.

1.1 Bốn bước của Pólya, dịch sang ngôn ngữ lập trình

Pólya chia quá trình giải thành bốn bước: hiểu bài toán, lập kế hoạch, thực hiện kế hoạch, và nhìn lại. Nghe đơn giản đến mức dễ bị coi thường. Nhưng cái giá trị không nằm ở bốn cái tên mà nằm ở *những câu hỏi cụ thể* thuộc về mỗi bước, và ở việc bạn biết mình đang ở bước nào — vì phần lớn thất bại đến từ việc nhảy cóc sang bước 3 khi bước 1 còn chưa xong.

Bước 1 — Hiểu bài toán, kỹ hơn bạn tưởng là cần

Câu hỏi của bước này không phải “đề nói gì” mà là bốn câu rất cụ thể: *cái gì là dữ liệu vào, cái gì là kết quả cần ra, ràng buộc là gì, và tôi có thể phát biểu lại đề bằng lời của mình mà không nhìn vào đề không*. Câu cuối là bài kiểm tra thật sự. Nếu chưa phát biểu lại được, bạn chưa hiểu đề, và mọi phút suy nghĩ tiếp theo đều là phí.

Trong lập trình có ba việc thuộc bước này mà người mới hay bỏ qua. Thứ nhất, *đọc giới hạn*. Dòng “ $1 \leq n \leq 2 \cdot 10^5$ ” không phải chi tiết kỹ thuật, nó là một nửa gợi ý: nó nói lời giải cần cỡ $O(n \log n)$, tức là loại ngay mọi hướng $O(n^2)$. Chương A/03 có bảng chuyển từ giới hạn sang độ phức tạp cho phép; bảng đó nên thuộc lòng.

Thứ hai, *làm tay ví dụ nhỏ*. Không phải ví dụ trong đề — ví dụ của bạn, nhỏ hơn, và bạn tự tính đáp án. Với $n = 1, n = 2, n = 3$. Việc này nghe tầm thường nhưng nó là công cụ sinh ý tưởng mạnh nhất mà chương này có, vì cấu trúc của bài toán thường chỉ lộ ra khi bạn nhìn thấy các trường hợp cạnh nhau.

Thứ ba, *tìm trường hợp biên và trường hợp suy biến*. Mảng rỗng, tất cả phần tử bằng nhau, đồ thị không liên thông, số âm, tràn số. Danh sách này sẽ cứu bạn ở bước 4, nhưng nghĩ ra nó ở bước 1 thì rẻ hơn nhiều.

CẠM BẮY

Người mới thường coi bước 1 là bước “đọc đề”, mất hai phút, rồi lao vào nghĩ thuật toán. Người có kinh nghiệm dành *nhiều hơn* thời gian cho bước 1 chứ không ít hơn — vì đa số bài khó không khó ở thuật toán, mà khó ở chỗ bạn chưa nhìn ra bài toán thật sự đang hỏi gì. Một dấu hiệu chắc chắn rằng bạn bỏ dở bước 1: bạn đang viết code mà vẫn phải cuộn lên đọc lại đề.

Bước 2 – Lập kế hoạch: tìm mối nối

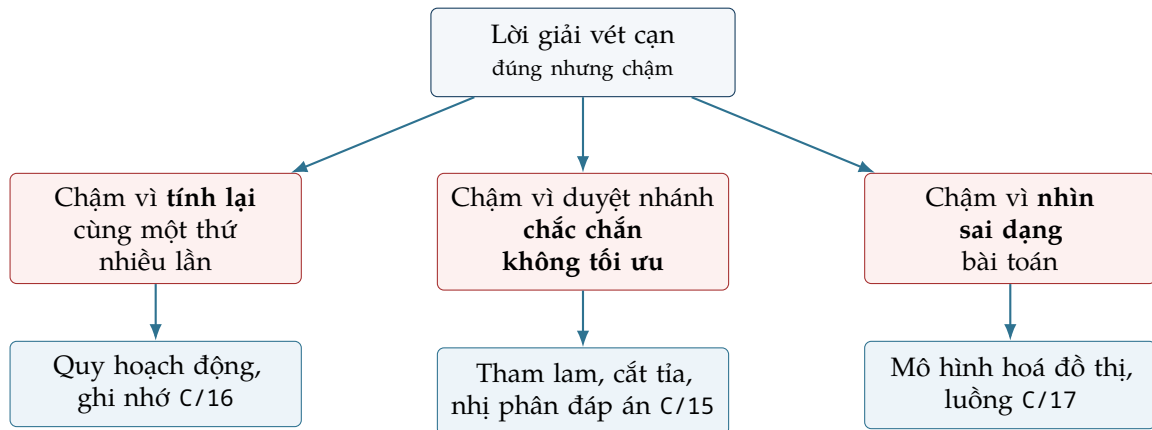
Đây là bước khó nhất và cũng là bước Pólya viết nhiều nhất. Câu hỏi trung tâm của ông rất đơn giản: *bạn đã gặp bài nào giống bài này chưa?* Không phải giống hệt — giống ở một khía cạnh nào đó. Cùng dạng dữ liệu vào, cùng dạng kết quả, cùng cấu trúc ràng buộc.

Trong lập trình, bước 2 có một hình thức rất cụ thể mà tôi sẽ nhắc lại nhiều lần trong quyển sách này: **luôn bắt đầu bằng vết cạn**. Viết ra — bằng lời hoặc bằng mã giả — một lời giải chắc chắn đúng dù chậm. Việc này có ba tác dụng, và cả ba đều lớn.

Tác dụng thứ nhất: nó buộc bạn phát biểu chính xác cái gì đang được tìm. Rất nhiều lúc bí là do bài toán trong đầu bạn vẫn còn mờ, và vết cạn làm nó rõ.

Tác dụng thứ hai: nó cho bạn một *lời giải đúng để đối chiếu*. Sau này, khi có lời giải nhanh, bạn cho hai chương trình chạy trên hàng nghìn test ngẫu nhiên nhỏ và so kết quả — kỹ thuật stress test ở chương E/25, và là kỹ thuật gỡ lỗi hiệu quả nhất trong nghề.

Tác dụng thứ ba, và là tác dụng quan trọng nhất về mặt tư duy: nó cho bạn một câu hỏi tốt hơn để suy nghĩ. Thay vì câu hỏi mù mờ “làm sao giải bài này”, bạn có câu hỏi sắc bén “*vì sao lời giải vết cạn này chậm, và công việc nào trong đó là thừa*”. Hình 1.1 cho thấy vì sao câu hỏi thứ hai dễ trả lời hơn hẳn: ba câu trả lời phổ biến dẫn thẳng tới ba nhóm kỹ thuật lớn của phần C.



Hình 1.1. Cây quyết định đi từ lời giải vết cạn tới nhóm kỹ thuật cần dùng. Câu hỏi “vì sao vết cạn chậm” có ít câu trả lời hơn hẳn câu hỏi “giải bài này thế nào”, nên nó dễ trả lời hơn — đó là toàn bộ giá trị của việc viết vết cạn ra trước.

Bước 3 – Thực hiện, và kiểm tra từng bước

Pólya nhấn mạnh một điều mà dân lập trình hay quên: khi thực hiện kế hoạch, phải kiểm tra từng bước có đúng không, và phải phân biệt được giữa “tôi thấy nó đúng” với “tôi chứng minh được nó đúng”. Trong lập trình, bước này gồm ba việc: chuyển ý tưởng thành mã, viết được các bất biến ra thành lời, và chạy thử trên ví dụ tự làm tay ở bước 1.

Có một quy tắc thực dụng đáng nhớ: *đừng gõ code khi bạn chưa nói được lời giải bằng ba câu*. Nếu bạn không tóm tắt được ý tưởng thành ba câu, tức là kế hoạch chưa xong, và gõ code lúc này chỉ chuyển sự mơ hồ từ đầu bạn vào trình soạn thảo — nơi nó khó sửa hơn nhiều.

Bước 4 – Nhìn lại: bước bị bỏ nhiều nhất, mất mát lớn nhất

Sau khi bài chạy đúng, gần như mọi người đóng bài và làm bài tiếp theo. Đây là lỗi tốn kém nhất trong toàn bộ việc luyện thuật toán, vì bước 4 chính là bước biến một lần giải bài thành một mẫu kiến thức dùng lại được.

Bước 4 gồm ba câu hỏi. Thứ nhất: *lời giải có thể ngắn hơn, nhanh hơn, hay đơn giản hơn không*. Thứ hai: *ý tưởng cốt lõi ở đây là gì, phát biểu bằng một câu, không nhắc tới chi tiết đề bài*. Thứ ba: *tôi đã bí ở đâu, và dấu hiệu nào lẽ ra đã giúp tôi thoát sớm hơn*. Câu thứ hai tạo ra thứ mà người luyện lâu năm gọi là “mẫu” — ví dụ “khi phải trả lời nhiều truy vấn tổng đoạn trên mảng tĩnh, hãy tính trước tổng tiền tố”. Câu thứ ba tạo ra nhật ký lỗi, công cụ được nói kỹ ở chương E/28.

CẦN NHỚ

Bốn bước không phải một đường thẳng mà là một vòng lặp có phản hồi. Bí ở bước 2 thì quay lại bước 1 (thường bạn hiểu sai đề); sai ở bước 3 thì quay lại bước 2 (thường kế hoạch có lỗ hổng, không phải code có lỗi gõ). Chỗ đáng ngờ nhất luôn là bước *ngay trước* chỗ bạn đang mắc.

1.2 Mười hai câu hỏi để hỏi khi bí

Phần này là công cụ thực dụng nhất của chương. Khi bí, đừng cố “nghĩ mạnh hơn” — đó là lời khuyên vô nghĩa vì nó không thay đổi thông tin bạn đang có. Thay vào đó, chạy qua danh sách dưới đây và tự hỏi từng câu. Mỗi câu hỏi có khả năng làm lộ ra một cấu trúc mà cách nhìn hiện tại của bạn đang che mắt. Danh sách này tổng hợp từ Pólya[1], Zeitz[4], Engel[5] và kinh nghiệm lập trình thi đấu.

1. **Làm ví dụ nhỏ nhất ra giấy.** $n = 1, 2, 3, 4$, tự tính đáp án bằng tay, xếp thành bảng, nhìn xem có quy luật gì. Đây là câu hỏi số một vì nó có tỉ lệ thành công cao nhất và chi phí thấp nhất.
2. **Bài này giống bài nào tôi từng giải?** Giống ở dữ liệu vào, ở kết quả ra, hay ở cấu trúc? Nếu giống một phần, phần khác biệt nằm ở đâu?
3. **Bỏ bớt một ràng buộc thì bài có dễ đi không?** Nếu mọi số đều dương thì sao? Nếu đồ thị là cây thì sao? Giải bài dễ hơn trước, rồi hỏi cách bù lại ràng buộc đã bỏ. Pólya gọi đây là “bài toán phụ”.
4. **Đảo ngược hướng nhìn.** Thay vì tìm cách tốt nhất, hãy đếm cách xấu và trừ đi. Thay vì đi từ đầu tới cuối, đi từ cuối về đầu. Thay vì hỏi “đáp án là bao nhiêu”, hỏi “với giá trị x cho trước, đáp án có $\geq x$ không” — câu này chính là cửa vào kỹ thuật nhị phân trên đáp án ở chương C/19.
5. **Sắp xếp lại dữ liệu thì sao?** Rất nhiều bài trở nên hiển nhiên sau khi sắp xếp, vì sắp xếp áp một thứ tự lên dữ liệu và thứ tự là thứ mà thuật toán tham lam cần.
6. **Có đại lượng nào không đổi không?** Tổng, chẵn lẻ, tổng theo modulo, số nghịch thế. Bất biến là công cụ mạnh nhất để chứng minh *không thể* làm được điều gì đó.
7. **Trường hợp cực đoan nói gì?** Cái nhỏ nhất, cái lớn nhất, cái ở biên. Lời giải tham lam thường sống ở đó.
8. **Có thể chia bài thành hai nửa độc lập không?** Nếu có thì chia để trị hoặc gặp nhau ở giữa. Nếu không, chỗ “đính” giữa hai nửa chính là trạng thái của quy hoạch động.
9. **Phát biểu lại bằng ngôn ngữ khác.** Bài mảng phát biểu lại thành bài đồ thị, bài đồ thị thành bài ma trận, bài đếm thành bài xác suất. Đổi ngôn ngữ hay làm hiện ra thuật toán có sẵn.
10. **Cái gì thực sự thay đổi giữa hai bước liên tiếp?** Nếu đáp án của bước $i + 1$ chỉ khác đáp án bước i một ít, bạn có lời giải tăng dần thay vì tính lại từ đầu — đó là ý tưởng nền của hai con trỏ, cửa sổ trượt và cấu trúc dữ liệu động.
11. **Nếu tôi được biết trước một đại lượng thì bài có dễ không?** Nếu “giá mà tôi biết phần tử lớn nhất nằm ở đâu” làm bài trở nên dễ, thì hãy duyệt qua mọi vị trí có thể của nó — kỹ thuật cố định một biến.

12. **Giới hạn đang mách gì?** $n \leq 20$ mách vét cạn 2^n ; $n \leq 10^5$ mách $O(n \log n)$; $n \leq 500$ mách $O(n^3)$; tổng n qua các test được chặn thì mách lời giải tuyến tính theo tổng. Chương A/03 có bảng đầy đủ.

TRỰC GIÁC

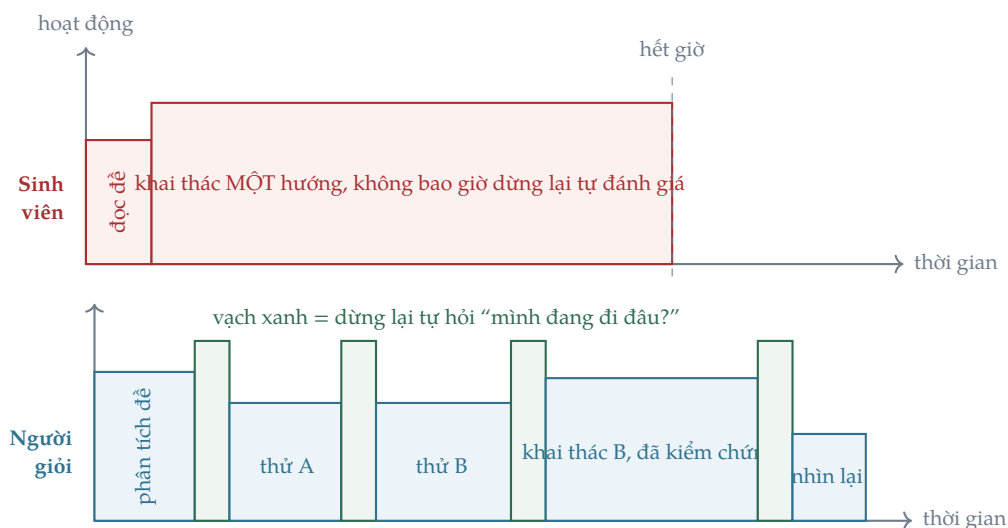
Vì sao một danh sách câu hỏi lại hiệu quả hơn “suy nghĩ tự do”? Vì bí không phải trạng thái thiếu năng lực tư duy, mà là trạng thái *lập lại một cách nhìn*. Đầu bạn cứ chạy lại đúng biểu diễn cũ của bài toán và tất nhiên thu được đúng kết luận cũ. Mỗi câu hỏi trong danh sách buộc bạn thay đổi biểu diễn — vẽ ra, đảo chiều, bỏ ràng buộc, đổi ngôn ngữ. Ý tưởng mới gần như luôn đến từ biểu diễn mới, chứ không từ việc nhìn chăm chăm biểu diễn cũ lâu hơn.

1.3 Điều Schoenfeld quay được trên video

Đến đây có một câu hỏi đáng đặt: nếu bốn bước và mười hai câu hỏi đơn giản như vậy, vì sao không phải ai đọc xong cũng giải bài giỏi lên? Alan Schoenfeld có câu trả lời dựa trên dữ liệu, và nó thay đổi cách nhìn của tôi về việc luyện tập.

Ông quay video các nhóm sinh viên giải toán và so với video các nhà toán học chuyên nghiệp giải cùng loại bài[2]. Điểm khác biệt lớn nhất không nằm ở kiến thức, cũng không nằm ở việc ai biết nhiều heuristic hơn. Nó nằm ở *cách phân bổ thời gian*. Sinh viên điển hình đọc đề, chọn một hướng trong vòng vài phút, rồi đi theo hướng đó cho tới hết giờ — kể cả khi hướng đó không dẫn tới đâu. Nhà toán học thì liên tục dừng lại tự hỏi: “mình đang làm gì đây? cái này có dẫn tới đâu không? đã bao lâu rồi? có nên thử hướng khác không?”.

Schoenfeld gọi tầng đó là *control* — tạm dịch là tự giám sát. Ông tổng kết rằng năng lực giải bài gồm bốn thành phần: kiến thức nền, các heuristic, khả năng tự giám sát, và hệ niềm tin về việc giải toán là gì. Thành phần thứ ba là thứ tạo ra khác biệt lớn nhất giữa hai nhóm, và cũng là thành phần dễ luyện nhất — vì nó chỉ đòi hỏi bạn đặt đồng hồ và tự hỏi vài câu.



Hình 1.2. Hai kiểu phân bổ thời gian mà Schoenfeld quan sát được. Người giải kém dành gần như toàn bộ thời gian để khai thác hướng đầu tiên nghĩ ra. Người giải giỏi cắt quá trình bằng những lần dừng ngắn để tự đánh giá, nhờ đó bỏ được hướng sai sớm. Điểm mấu chốt: các vạch xanh gần như không tốn thời gian, nhưng chúng quyết định phần thời gian còn lại được dùng vào việc gì.

Bài học thực hành rất cụ thể. Khi luyện, hãy đặt một mốc thời gian — ví dụ 10 phút — và mỗi khi tới mốc thì dừng lại 20 giây để trả lời ba câu: *tôi đang thử hướng gì, hướng này đã cho tôi tiến bộ nào chưa, nếu 10 phút nữa vẫn thế thì tôi sẽ chuyển sang hướng nào*. Ba câu này nghe máy móc, nhưng chúng chính là thứ mà người giỏi làm tự động và người mới không làm bao giờ.

1.4 Một ví dụ đi hết bốn bước

Lý thuyết trên sẽ mờ nếu không có một ví dụ chạy hết. Hãy lấy một bài kinh điển, đủ nhỏ để làm tay và đủ giàu để lộ ra cách nghĩ: *cho một mảng số nguyên có cả âm lẫn dương, tìm đoạn con liên tiếp có tổng lớn nhất*. Đây là bài mà Jon Bentley kể lại trong *Programming Pearls*[6] như một ví dụ mẫu mực về việc một bài toán có thể được giải lại nhiều lần, mỗi lần tốt hơn.

Bước 1 — hiểu. Dữ liệu vào là mảng $a_1, \dots, a_n$, kết quả là số lớn nhất trong các tổng $a_i + a_{i+1} + \dots + a_j$. Phát biểu lại bằng lời của mình: “chọn một đoạn liên nhau sao cho tổng lớn nhất”. Trường hợp biên đáng ngờ ngay: nếu mọi phần tử đều âm thì sao? Tùy quy ước — hoặc lấy phần tử lớn nhất, hoặc cho phép đoạn rỗng với tổng 0. Đây chính là loại câu hỏi mà bước 1 phải làm rõ, vì nó quyết định code.

Bước 2 — kế hoạch. Vết cặn trước: duyệt mọi cặp (i, j) , tính tổng, lấy lớn nhất. Chi phí $O(n^3)$ nếu tính tổng lại từ đầu mỗi lần, $O(n^2)$ nếu cộng dồn. Bây giờ dùng câu hỏi trung tâm: *công việc nào đang bị lặp?* Rõ ràng khi ta cố định j và thử mọi i , phần lớn công sức lặp lại giữa j và $j + 1$. Đó là dấu hiệu của câu hỏi số 10 trong danh sách: *cái gì thay đổi giữa hai bước liên tiếp?*

Hãy gọi B_j là tổng lớn nhất của một đoạn *kết thúc đúng tại j* . Câu hỏi trở thành: biết B_j , tính B_{j+1} thế nào? Đoạn tốt nhất kết thúc tại $j + 1$ hoặc là chỉ gồm mình a_{j+1} , hoặc là đoạn tốt nhất kết thúc tại j nối thêm a_{j+1} . Vậy

$$B_{j+1} = \max(a_{j+1}, B_j + a_{j+1}) = a_{j+1} + \max(0, B_j),$$

và đáp án là $\max_j B_j$. Công thức nhỏ này nói một điều rất trực giác: *nếu phần đuôi tích lũy được cho tới giờ đang âm, hãy vứt nó đi và bắt đầu lại*. Chi phí xuống $O(n)$, bộ nhớ $O(1)$.

Bước 3 — thực hiện. Bất biến cần giữ khi viết vòng lặp: sau khi xử lý xong vị trí j , biến cur bằng B_j và biến best bằng $\max_{k \leq j} B_k$. Viết được bất biến ra thành lời như vậy thì code gần như tự viết, và mọi câu hỏi kiểu “khởi tạo bằng 0 hay bằng $-\infty$ ” đều có câu trả lời từ bất biến chứ không phải từ việc thử.

Bước 4 — nhìn lại. Ý tưởng cốt lõi, phát biểu không nhắc đề bài: *khi tìm cực trị trên mọi đoạn, hãy cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó*. Câu này là một mẫu dùng lại được — nó giải luôn các bài tích lớn nhất, đoạn có tổng chia hết cho k , và là bước đầu tiên của rất nhiều bài quy hoạch động một chiều. So sánh: nếu bạn chỉ nhớ “bài tổng đoạn lớn nhất dùng thuật toán Kadane” thì bạn có một cái tên; nếu bạn nhớ câu vừa rồi thì bạn có một công cụ.

CẦN NHỚ

Bước 4 là chỗ biến một lần giải bài thành một mẫu kiến thức. Cách làm cụ thể: viết ra *một câu* nói ý tưởng cốt lõi mà không được nhắc tới chi tiết đề bài. Nếu không viết được câu đó, bạn chưa thực sự hiểu vì sao lời giải chạy được.

1.5 Vòng lặp bí — thoát bằng cách nào

Còn một tình huống chưa nói: bạn đã chạy qua danh sách câu hỏi, vẫn bí. Lúc này có ba chiến lược, xếp theo thứ tự nên thử.

Thứ nhất, giải một bài dễ hơn. Không phải bỏ cuộc mà là đổi mục tiêu tạm thời: giải bài với n nhỏ, hoặc với một ràng buộc thêm vào cho dễ, hoặc chỉ giải một trường hợp riêng. Rất thường xuyên, lời giải của bài dễ hơn chứa đựng ý tưởng cần cho bài gốc.

Thứ hai, viết ra chính xác chỗ bạn kẹt. Một câu, dạng “tôi biết cách làm nếu như ... nhưng tôi không biết cách xử lý khi ...”. Việc viết ra buộc sự mơ hồ phải thành hình, và một nửa số lần thì chính lúc viết bạn nhìn ra hướng đi.

Thứ ba, rời bài ra. Đây không phải lời khuyên tinh thần suông — nhiều người giải bài chuyên nghiệp nói về hiện tượng lời giải xuất hiện khi đang đi bộ hoặc tắm. Điều kiện để nó xảy ra là bạn phải đã nạp đủ vào đầu ở giai đoạn trước; rời bài mà chưa hiểu đề thì không có gì để ấp ủ.

Và một điều cuối, nói thẳng vì nó bị hiểu sai nhiều: *xem lời giải không phải điều xấu, xem lời giải quá sớm mới xấu.* Ngưỡng hợp lý khi luyện là 30–60 phút bí thật sự — bí sau khi đã chạy hết danh sách câu hỏi, chứ không phải bí sau khi đọc đề năm phút. Và khi đã xem, quy tắc bắt buộc là đóng lời giải lại rồi tự cài từ đầu, sau đó ghi vào nhật ký lỗi: *ý tưởng nào tôi đã không nghĩ ra, và dấu hiệu nào trên đề lẽ ra đã gợi ra nó.* Chương E/28 biến việc này thành một quy trình cụ thể.

1.6 Gốc rễ: quy trình này từ đâu ra

Bốn bước của Pólya trông như một danh sách hiển nhiên, nhưng nó là kết tinh của một truyền thống dài, và biết gốc rễ giúp bạn dùng nó đúng chỗ hơn.

Ý tưởng “có phương pháp để tìm ra lời giải, không chỉ để trình bày lời giải” xuất hiện từ Descartes trong *Quy tắc hướng dẫn tư duy* thế kỷ 17, và chính Pólya nhắc tới Descartes cùng Leibniz như những người đi trước. Điều thúc ép Pólya viết *How to Solve It* vào năm 1945 là một quan sát về dạy học: toán học được dạy như một *sản phẩm đã hoàn thành* — định lý, chứng minh, xong — trong khi quá trình thật sự tạo ra nó, gồm mò mẫm, thử ví dụ, đoán sai rồi sửa, thì bị giấu đi hoàn toàn. Người học vì thế tưởng rằng người giỏi “nhìn phát ra ngay”, và khi bản thân không nhìn ra ngay thì kết luận mình không có năng khiếu. Pólya viết sách để phơi bày phần bị giấu đó[1].

Đến thập niên 1970, ngành khoa học máy tính tiếp cận cùng câu hỏi từ hướng khác. Allen Newell và Herbert Simon nghiên cứu quá trình giải bài của con người bằng cách yêu cầu người ta *nói to suy nghĩ* khi giải, rồi mô hình hoá thành chương trình. Kết quả là khái niệm **không gian bài toán** (*problem space*) — giải bài tức là tìm đường trong một đồ thị các trạng thái — và **phân tích phương tiện–mục đích** (*means-ends analysis*): đo khoảng cách giữa trạng thái hiện tại và đích, rồi chọn hành động làm khoảng cách giảm. Đây là chỗ nổi rất đáng chú ý: cái mà ta gọi là “heuristic để nghĩ” trong chương này, và cái mà ta gọi là “hàm heuristic $h(n)$ ” trong thuật toán tìm kiếm A^* ở chương C/17, là *cùng một khái niệm*, ra đời từ cùng một dòng nghiên cứu. Tư duy của người và tìm kiếm của máy được mô hình hoá bằng cùng một ngôn ngữ.

Nhánh thứ ba là tâm lý học về sáng tạo. Graham Wallas mô tả bốn giai đoạn — chuẩn bị, ấp ủ, loé sáng, kiểm chứng — và giai đoạn “ấp ủ” chính là cơ sở cho lời khuyên “rời bài ra” ở mục trước, kèm điều kiện quan trọng: phải nạp đủ ở giai đoạn chuẩn bị thì mới có gì để ấp. Cuối cùng, Schoenfeld làm việc mà Pólya không làm được vì thiếu công cụ: đo thực nghiệm quá trình giải, và phát hiện ra rằng heuristic thôi chưa đủ, còn phải có tầng tự giám sát[2].

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thế kỷ 17 — Descartes, Leibniz	Tìm một “phương pháp phổ quát” để dẫn dắt tư duy, thay vì mỗi bài mỗi mẹo.	Đặt ra tham vọng có phương pháp; nhưng quá tổng quát nên không dùng trực tiếp được.
1945 — Pólya	Toán được dạy như sản phẩm hoàn chỉnh; người học không thấy quá trình mò mẫm nên tưởng mình thiếu năng khiếu.	Bốn bước và hệ thống câu hỏi; lần đầu quá trình giải bài trở thành thứ dạy được.
1972 — Newell & Simon	Muốn mô hình hoá tư duy đủ chính xác để viết thành chương trình.	Không gian bài toán và phân tích phương tiện–mục đích; heuristic trở thành khái niệm hình thức, dùng chung cho cả người lẫn máy ($\rightarrow A^*, C/17$).
1985 — Schoenfeld	Dạy heuristic Pólya cho sinh viên mà họ vẫn không giải được bài mới.	Phát hiện tăng tự giám sát; giải thích vì sao “biết heuristic” không đủ.
1993 — Ericsson	Vì sao cùng số giờ luyện mà người tiến người không.	Bốn điều kiện của luyện tập có chủ đích; nối chương này với chương A/02 .

1.7 Liên hệ ngang: cùng một quy trình ở những nơi khác

Điều đáng để ý là quy trình bốn bước không phải đặc sản của toán hay của lập trình. Nó xuất hiện gần như y hệt ở mọi nghề mà công việc chính là *chẩn đoán trong điều kiện thiếu thông tin*, và nhìn thấy sự trùng khớp đó giúp bạn mượn được kỹ thuật từ nghề khác.

Ví dụ rõ nhất là gỡ lỗi phần mềm. Khi một chương trình sai, người có kinh nghiệm không đọc code từ đầu tới cuối; họ *chia đôi vùng nghi ngờ*: đặt một điểm kiểm tra ở giữa, xem trạng thái ở đó đúng hay sai, và loại bỏ một nửa. Đó chính là tìm kiếm nhị phân, áp lên không gian nguyên nhân thay vì lên mảng — và chương E/25 sẽ chỉ ra rằng git *bisect* là đúng thuật toán đó, đóng gói thành công cụ.

Ví dụ thứ hai đến từ chính công việc của người viết dòng này: gỡ một hệ thống *SLAM* chạy sai. Quy trình hiệu quả không phải đọc lại toàn bộ mã ước lượng, mà là: dựng *ví dụ nhỏ nhất* (một đoạn dữ liệu vài giây thay vì cả chuỗi), tìm một *bất biến* phải luôn đúng (hướng trọng lực không được trôi, hiệp phương sai phải xác định dương), rồi tìm bước đầu tiên mà bất biến bị vi phạm. Ba từ khoá — ví dụ nhỏ nhất, bất biến, bước hỏng đầu tiên — là đúng ba công cụ của chương này và chương [A/04](#), chỉ đổi miền áp dụng.

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Khoa học thực nghiệm	Giả thuyết $\rightarrow$ thí nghiệm có khả năng bác bỏ $\rightarrow$ sửa giả thuyết	Giống: đều tiến bằng cách loại trừ, và một phản ví dụ mạnh hơn nhiều ví dụ ủng hộ. Khác: nhà khoa học không bao giờ có “đáp án đúng” để đối chiếu, còn bạn thì có — nên bạn được phép dùng vết cặn làm trọng tài.
Chẩn đoán y khoa	Chẩn đoán phân biệt: liệt kê mọi khả năng rồi loại bằng xét nghiệm rẻ nhất trước	Giống hệ heuristic “bỏ bớt ràng buộc” và “trường hợp cực đoan”. Khác: chi phí sai lệch bất đối xứng — bỏ sót bệnh nặng đắt hơn nhiều so với dương tính giả, nên thứ tự xét nghiệm không tối ưu theo thông tin thuần túy.
Gỡ lỗi phần mềm	Chia đôi vùng nghi ngờ, git <i>bisect</i>	Cùng thuật toán với tìm kiếm nhị phân, áp lên không gian nguyên nhân. Điều kiện áp dụng cũng giống: cần tính đơn điệu — trước điểm đó thì đúng, sau điểm đó thì sai (E/25).

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Ước lượng trạng thái, SLAM	Ví dụ nhỏ nhất + bất biến (trọng lực, hiệp phương sai) + tìm bước hỏng đầu tiên	Cùng bộ ba công cụ với chương này và A/04. Khác: hệ thực có nhiều, nên “bất biến” trở thành “nằm trong khoảng tin cậy”, và phải phân biệt sai số hợp lệ với lỗi thật.
Trí tuệ nhân tạo cổ điển	Không gian trạng thái, hàm heuristic $h(n)$, A^*	Không phải chỉ giống — cùng một khái niệm, cùng nguồn gốc từ Newell & Simon. Bạn dùng heuristic để cắt bớt nhánh trong đầu, A^* dùng nó để cắt bớt nhánh trong hàng đợi ưu tiên (C/17).
Cờ vua, cờ vây	Tính toán biến (calculation) xen kẽ với đánh giá thể (evaluation)	Giống tăng tự giám sát của Schoenfeld: kỹ thủ mạnh không tính sâu hơn nhiều, họ biết <i>khi nào nên ngừng tính một biến</i> . Xem thêm A/02.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì kiến thức chỉ là một trong bốn thành phần của năng lực giải bài. Người giải được bài mới có thêm ba thứ: một quy trình để chạy khi chưa biết làm gì, một bộ heuristic để đổi cách nhìn bài toán, và quan trọng nhất là khả năng tự giám sát — biết mình đang đi hướng nào, đi bao lâu rồi, và khi nào phải bỏ. Schoenfeld chỉ ra thành phần thứ ba tạo ra khác biệt lớn nhất trong dữ liệu thực nghiệm của ông.
- A2. Vì bí không phải trạng thái thiếu sức nghĩ, mà là trạng thái lặp lại một biểu diễn. “Nghĩ kỹ hơn” giữ nguyên biểu diễn nên cho ra đúng kết luận cũ. “Vẽ ví dụ $n = 4$ ” thì đổi biểu diễn từ ký hiệu trừu tượng sang hình ảnh cụ thể, và cấu trúc thường chỉ lộ ra ở biểu diễn mới.
- A3. Vì vết cạn cho ba thứ mà bạn không có nếu bỏ qua nó: một phát biểu chính xác về cái đang tìm, một lời giải đúng để đối chiếu khi stress test, và quan trọng nhất là một câu hỏi tốt hơn để suy nghĩ — “vì sao nó chậm” thay cho “giải thể nào”. Hình 1.1 cho thấy câu hỏi đó chỉ có vài câu trả lời khả dĩ, mỗi câu dẫn thẳng tới một nhóm kỹ thuật.
- A4. Bạn mất khả năng dừng lại. Bước 4 là chỗ ý tưởng được tách khỏi đề bài cụ thể và phát biểu thành một câu tổng quát — “cố định điểm cuối, duy trì đáp án tốt nhất kết thúc tại đó”. Không có bước 4, bạn tích lũy được những lần giải bài chứ không tích lũy được công cụ, nên bài sau vẫn phải nghĩ lại từ đầu.
- A5. Khi hướng đó đã chạy đủ lâu mà không tạo ra *tiến bộ kiểm chứng được* — không có bỏ đề mới, không có ví dụ nhỏ nào được giải, không có ràng buộc nào bị loại. Cách biết trước khi hết giờ là đặt mốc kiểm tra định kỳ, ví dụ 10 phút một lần, và trả lời ba câu ở mục trước. Không có mốc kiểm tra thì bạn chỉ phát hiện ra hướng sai khi đồng hồ đã hết.

NÓI LẠI THẬT ĐƠN GIẢN

Giải một bài lạ giống như tìm đường trong một thành phố lạ mà không có bản đồ. Việc đầu tiên không phải là chạy nhanh, mà là đứng lại xem mình đang ở đâu và cần tới đâu — đó là bước hiểu đề. Việc thứ hai là tìm một con đường bạn từng đi qua có nối tới đó không — đó là bước tìm mối nối, và mẹo hiệu quả nhất là đi bộ thử một đoạn ngắn bằng cách làm tay ví dụ nhỏ. Cách chậm nhưng chắc chắn tới nơi là đi thử hết mọi ngõ, tức vết cạn; bạn luôn nên hình dung cách đó trước, vì khi biết nó chậm ở chỗ nào thì bạn biết cần rẽ tắt ở đâu. Khi đã tới nơi rồi, hãy dành một phút nhìn lại con đường và ghi nhớ khúc quan trọng nhất — lần sau bạn sẽ không phải dò lại. Còn khi lạc, đừng đi nhanh hơn theo hướng cũ; hãy dừng lại và tự hỏi một câu hỏi khác.

Một câu: Khi bí, đừng nghĩ mạnh hơn — hãy đổi câu hỏi mà bạn đang tự hỏi.

Chỗ không rút gọn được: Không có cách nào biết chắc một hướng là sai trước khi thử; mọi thứ ta làm được chỉ là đặt mốc kiểm tra để phát hiện sớm và trả giá rẻ hơn.

Thuật ngữ / câu hỏi	Trả lời
●●● Bốn bước của Pólya	Hiểu bài toán → lập kế hoạch → thực hiện → nhìn lại. Không phải đường thẳng mà là vòng lặp: mắc ở bước sau thì quay lại bước ngay trước nó.
●●● Kiểm tra “đã hiểu đề”	Phát biểu lại đề bằng lời của mình, không nhìn vào đề. Chưa làm được thì mọi phút suy nghĩ tiếp theo đều phí.
●●● Vì sao luôn viết vết cạnh trước	Nó cho phát biểu chính xác bài toán, cho lời giải đúng để stress test, và đổi câu hỏi từ “giải thế nào” thành “vì sao chậm” — câu sau dễ trả lời hơn nhiều.
●●● Ba lý do vết cạnh chậm	Tính lại thứ đã tính (→ quy hoạch động); duyệt nhánh chắc chắn tẻ (→ tham lam, cắt tỉa); nhìn sai dạng bài (→ mô hình hoá lại).
●●● Tầng “control” của Schoenfeld	Khả năng tự giám sát: đang làm gì, có tiến bộ không, khi nào bỏ hướng. Đây là khác biệt lớn nhất giữa người giải giỏi và kém, và là thành phần dễ luyện nhất.
●● Heuristic hiệu quả nhất	Làm tay ví dụ nhỏ $n = 1, 2, 3, 4$ và xếp kết quả thành bảng. Chi phí thấp nhất, tỉ lệ ra ý tưởng cao nhất.
●● Heuristic “đảo ngược”	Đếm cái xấu rồi trừ; đi từ cuối về đầu; đổi “đáp án là bao nhiêu” thành “đáp án có $\geq x$ không” — câu cuối là cửa vào nhị phân trên đáp án.
●● Heuristic “bỏ bớt ràng buộc”	Giải bài dễ hơn trước (mọi số dương, đồ thị là cây, n nhỏ), rồi tìm cách bù lại ràng buộc đã bỏ.
●● Câu hỏi “cái gì đổi giữa hai bước”	Nếu đáp án bước $i + 1$ chỉ khác bước i một chút thì có lời giải tăng dần: hai con trỏ, cửa sổ trượt, cấu trúc dữ liệu động.
●● Quy tắc trước khi gõ code	Nói được lời giải bằng ba câu. Không nói được nghĩa là kế hoạch chưa xong, và gõ code chỉ chuyển sự mơ hồ vào trình soạn thảo.
●● Nội dung bước 4	Có ngắn/nhanh/đơn giản hơn không; ý tưởng cốt lõi phát biểu một câu không nhắc đề bài; tôi bí ở đâu và dấu hiệu nào lẽ ra giúp tôi thoát sớm.
●● Ba cách thoát khi hết heuristic	Giải một bài dễ hơn; viết ra chính xác chỗ kẹt dạng “tôi biết làm nếu... nhưng không biết khi...”; rời bài ra sau khi đã nạp đủ.
● Ngưỡng xem lời giải	30–60 phút bí thật sự, sau khi đã chạy hết danh sách câu hỏi. Xem xong bắt buộc đóng lại, tự cài từ đầu, và ghi nhật ký lỗi.
● Bốn thành phần của Schoenfeld	Kiến thức nền, heuristic, tự giám sát, và hệ niềm tin về việc giải toán là gì.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

- Vì sao Pólya nói “nếu bạn không giải được bài toán này, hãy tìm một bài dễ hơn liên quan tới nó”? Nghe như lời khuyên né tránh.

| Vì bài dễ hơn thường chứa *cùng cấu trúc* với bài gốc, chỉ bớt đi phần gây nhiễu. Khi giải được bài dễ, bạn có một lời giải cụ thể để soi, và câu hỏi trở thành “ràng buộc tôi đã bỏ đi làm hỏng chỗ nào của lời giải này” — một câu hỏi hẹp và cụ thể, dễ hơn hẳn câu hỏi mở ban đầu. Đây không phải né tránh mà là chia bài toán khó thành hai bước dễ hơn.

- Tại sao bảo “đọc giới hạn n là một nửa gợi ý”? Chẳng phải giới hạn chỉ là chi tiết kỹ thuật?

| Vì giới hạn loại bỏ phần lớn không gian tìm kiếm ý tưởng. Với $n \leq 20$, người ra đề gần như chắc chắn muốn một lời giải mũ, thường là duyệt 2^n tập con hoặc quy hoạch động trên bitmask — và bạn không cần phí thời gian tìm lời giải đa thức thông minh. Với $n \leq 2 \cdot 10^5$, lời giải phải cỡ $O(n)$ hoặc $O(n \log n)$, nên mọi ý tưởng $O(n^2)$ bị loại ngay và bạn tiết kiệm được nửa giờ.

- Nếu tôi đã biết bài này là quy hoạch động, tôi có cần bước 1 và bước 2 không?

| Có, và bước 1 lúc này còn quan trọng hơn. Biết “là quy hoạch động” chưa nói được trạng thái là gì — mà chọn sai trạng thái là cách hỏng phổ biến nhất của quy hoạch động. Trạng thái đúng thường lộ ra từ việc làm tay ví dụ nhỏ và từ câu hỏi “cái gì thực sự thay đổi giữa hai bước liên tiếp”, tức là từ bước 1 và bước 2.

4. Trong một kỳ thi có giới hạn thời gian, bốn bước của Pólya có xa xỉ quá không?

| Ngược lại, dưới sức ép thời gian thì bốn bước lại càng đáng giá, chỉ là mỗi bước ngắn lại. Kinh nghiệm thi đấu cho thấy chi phí lớn nhất không phải nghĩ chậm mà là *cài xong rồi mới phát hiện hiểu sai đề*, hoặc *đi một hướng sai 40 phút*. Bước 1 rút gọn còn hai phút và bước “dừng lại tự đánh giá” chỉ tốn 20 giây, nhưng chúng chặn đúng hai lỗi tốn kém nhất đó.

5. Làm sao phân biệt “bí lành mạnh” (nên tiếp tục) với “bí vô ích” (nên xem lời giải)?

| Bằng dấu hiệu tiến bộ, không bằng cảm giác. Bí lành mạnh là khi mỗi 10–15 phút bạn vẫn thu được thứ mới: một ví dụ nhỏ giải được, một hướng bị loại có lý do, một bất biến được phát hiện. Bí vô ích là khi bạn đọc lại đề lần thứ năm và nghĩ đúng những ý đã nghĩ. Dấu hiệu thứ hai xuất hiện thì hoặc đổi heuristic, hoặc chuyển bài, hoặc xem gợi ý — gợi ý một dòng, không phải toàn bộ lời giải.

6. Vì sao chương này nhấn mạnh “phát biểu ý tưởng bằng một câu không nhắc tới đề bài”?

| Vì đó là cách kiểm tra xem bạn đã trừu tượng hoá được chưa. Câu nói kèm chi tiết đề bài (“với mảng số trong bài này, ta duyệt từ trái sang”) chỉ dùng được cho đúng bài đó. Câu nói tổng quát (“khi tìm cực trị trên mọi đoạn, cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó”) dùng được cho hàng chục bài khác. Kiến thức chỉ dùng lại được ở dạng thứ hai.

7. Người ta hay nói “giải nhiều bài thì tự khắc giỏi”. Chương này có phản đối điều đó không?

| Có, một phần. Số lượng chỉ có tác dụng nếu mỗi bài đều đi qua bước 4 và nếu độ khó được chọn hơi cao hơn mức hiện tại của bạn — đúng định nghĩa luyện tập có chủ đích của Ericsson[3]. Giải một trăm bài vừa sức, không nhìn lại, không ghi lỗi, thì bạn luyện tốc độ gõ chứ không luyện năng lực giải bài. Chương E/28 biến nhận xét này thành một lịch luyện cụ thể.

8. Bất biến xuất hiện ở đây làm gì, chẳng phải nó thuộc về phần chứng minh?

| Bất biến có hai vai trò, và vai trò sinh ý tưởng thường bị bỏ quên. Khi bạn hỏi “đại lượng nào không đổi trong quá trình này”, bạn vừa đang tìm công cụ chứng minh, vừa đang tìm chính cấu trúc để thiết kế thuật toán — ví dụ bất biến “cur là tổng lớn nhất của đoạn kết thúc tại j ” vừa là lời giải vừa là chứng minh của bài tổng đoạn lớn nhất. Chương A/04 khai thác cả hai vai trò.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tổng đoạn con lớn nhất	CSES “Maximum Subarray Sum”	Đi lại đúng bốn bước của mục 4; viết bất biến ra thành lời trước khi gõ code.
● Đảo ngược một số nguyên, xử lý tràn	LeetCode “Reverse Integer”	Bước 1 quyết định tất cả: liệt kê trường hợp biên (số âm, tận cùng bằng 0, tràn 32 bit) trước khi viết.
● Hai số có tổng bằng x	CSES “Sum of Two Values”	Heuristic “sắp xếp lại dữ liệu” hoặc “đổi sang tra cứu”; so sánh hai lời giải và giải thích khi nào chọn cái nào.
●● Bài toán tám quân hậu	CSES “Chessboard and Queens”	Vết cạn có cắt tỉa; luyện thói quen ước lượng kích thước không gian tìm kiếm <i>trước</i> khi cài.
●● Tìm phần tử xuất hiện quá nửa	LeetCode “Majority Element”	Heuristic “đại lượng bất biến”: thuật toán bỏ phiếu Boyer–Moore. Hãy tự tìm bất biến trước khi tra.
●● Tích đoạn con lớn nhất	LeetCode “Maximum Product Subarray”	Cùng mẫu với bài mở đầu nhưng số âm phá cấu trúc: cần giữ đồng thời cực đại và cực tiểu. Ví dụ nhỏ lộ ra ngay.
●● Trò chơi Nim đơn giản	Bài tập kinh điển	Heuristic “bất biến” và “trường hợp cực đoan”: làm tay các thế $n \leq 3$ trước khi tìm quy luật XOR.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Số cách leo cầu thang với bước 1 và 2	AtCoder DP “Frog 1”	Bài mẫu để tập bước 4: phát biểu ý tưởng bằng một câu không nhắc tới cầu thang.
●●● Bài toán 3-SUM	Bài tập kinh điển	Từ $O(n^3)$ xuống $O(n^2)$: cố định một biến rồi dùng hai con trỏ. Luyện heuristic “giá mà tôi biết trước một đại lượng”.
●●● Xếp lịch cuộc họp tối đa	Bài tập kinh điển	Bí thì hỏi “trường hợp cực đoan nói gì”: chọn theo thời điểm kết thúc sớm nhất. Thử tự chứng minh bằng lập luận đối chỗ (chương A/04).
●●● Bài toán “nước mưa mắc kẹt”	LeetCode “Trapping Rain Water”	Ba lời giải khác nhau (tiền tố cực đại, ngăn xếp, hai con trỏ). Giải cả ba rồi so, đây là bài tập bước 4 tốt nhất trong danh sách này.
●●● Một bài bạn từng bí và đã xem lời giải	Nhật ký của chính bạn	Tự cài lại không nhìn, rồi viết một câu về ý tưởng cốt lõi và một câu về dấu hiệu lẽ ra đã gợi ra nó.

Kết chương và đọc tiếp

Chương này cố ý không chứa thuật toán nào. Nó dựng một quy trình bốn bước, một danh sách mười hai câu hỏi để phá thế bí, và một thói quen tự giám sát bằng mốc thời gian. Ba thứ đó là khung để treo mọi kiến thức kỹ thuật của 27 chương còn lại; thiếu chúng, kiến thức chỉ nằm thành danh mục và không tự tìm được đường ra khi cần.

Còn bỏ ngỏ một câu hỏi lớn: quy trình thì có rồi, nhưng *luyện* thế nào để nó thành phản xạ, và những người giỏi nhất thực sự luyện ra sao? Đó là nội dung chương [A/02](#). Sau đó, chương [A/03](#) và [A/04](#) cấp hai cái thước — nhanh không, đúng không — mà bước 2 và bước 3 của Pólya luôn cần tới.

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, “The Role of Deliberate Practice in the Acquisition of Expert Performance,” *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [7] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [10] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [11] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [12] C. A. R. Hoare, “An Axiomatic Basis for Computer Programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] D. Gries, *The Science of Programming*. Springer, 1981.
- [14] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [15] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [16] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [17] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [18] e-maxx / cp-algorithms contributors. “cp-algorithms: Algorithms for Competitive Programming.” (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [19] C. A. R. Hoare, “Quicksort,” *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [20] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [21] R. E. Tarjan, “Amortized Computational Complexity,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [22] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [23] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [24] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.

- [25] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.
- [26] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [34] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [35] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [36] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [37] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [38] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [39] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [40] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Kinh nghiệm của những người giỏi nhất (What the Masters Actually Do)

PHỤ THUỘC A/01 — chương này nói cách luyện để quy trình ở chương 1 thành phản xạ. Đọc được độc lập, nhưng nhiều lời khuyên ở đây chỉ có nghĩa khi bạn đã biết bốn bước.

TÓM TẮT

Chương này khảo sát cách làm việc thật của những người đã định hình ngành thuật toán — Knuth, Dijkstra, Hoare, Tarjan, Bentley, Skiena, Sedgewick — cùng cách luyện của giới lập trình thi đấu hiện đại và của giới huấn luyện olympic toán. Mục đích không phải kể chuyện danh nhân mà là rút ra những thói quen *bắt buộc được*: viết chương trình cùng lúc với chứng minh, ước lượng trước khi cài, luyện ở vùng hơi quá sức, upsolving sau mỗi trận, giữ nhật ký lỗi, tự xây thư viện mẫu. Chương cũng cho một bản đồ sách: quyển nào đọc lúc nào và để làm gì. Nếu chỉ nhớ một điều: điểm chung của tất cả những người này không phải năng khiếu mà là *họ có một vòng phản hồi chặt* — làm, đo, phát hiện sai, sửa cách làm, lặp lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **luyện tập có chủ đích (deliberate practice)** — luyện có mục tiêu cụ thể, ở mức hơi quá sức hiện tại, có phản hồi ngay và có sửa sai — khác hẳn với “làm nhiều giờ”. Khái niệm của Ericsson[3].
- **upsolving** — giải lại những bài bạn không giải được trong một kỳ thi, *sau khi* thi xong, tự làm tới cùng. Đây là nguồn tiên bộ chính của dân thi đấu.
- **editorial** — bài phân tích lời giải chính thức của một đề thi lập trình.
- **chunk (khối tri thức)** — một cụm thông tin đã được ghép thành một đơn vị nhớ duy nhất, ví dụ “thế cờ này” thay vì hai mươi quân rời. Chuyên gia nhớ nhiều hơn không phải vì trí nhớ tốt hơn mà vì họ có nhiều khối lớn hơn.
- **thư viện mẫu (template library)** — tập hợp các đoạn mã đã kiểm chứng — DSU, segment tree, Dijkstra — được chuẩn bị sẵn để dán vào khi thi. KACTL của KTH là ví dụ nổi tiếng[7].
- **nhật ký lỗi** — sổ ghi lại mỗi lần sai: sai gì, vì sao, dấu hiệu nào lẽ ra đã phát hiện sớm. Công cụ đơn giản nhất và bị bỏ qua nhiều nhất.

CÂU HỎI MỞ ĐẦU

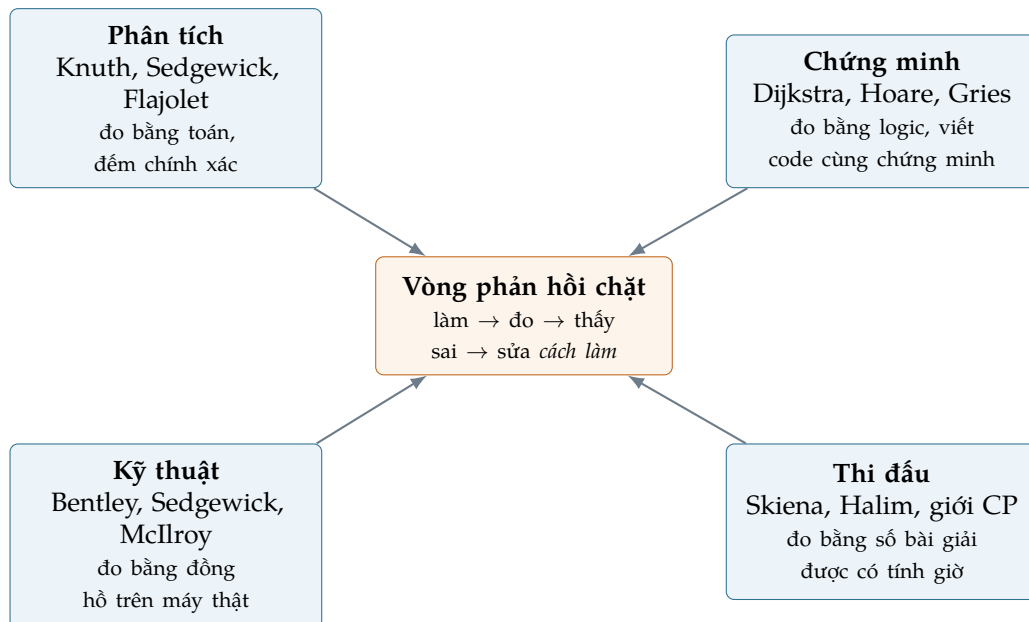
- Q1. Nếu tài năng là thứ quyết định, vì sao gần như mọi người giỏi thuật toán đều mô tả cùng một kiểu thói quen làm việc?
- Q2. Knuth ra bài tập có mức khó từ 0 đến 50 và bảo người đọc phải làm. Vì sao đọc mà không làm bài tập lại gần như vô ích?
- Q3. Dijkstra viết chương trình và chứng minh cùng lúc, không phải viết xong rồi kiểm thử. Điều đó thay đổi cái gì trong cách bạn ngồi trước bàn phím?
- Q4. Vì sao “giải thật nhiều bài” lại có thể khiến bạn dừng tiến bộ sau vài tháng, trong khi giải ít bài hơn nhưng chọn kỹ thì tiếp tục lên?
- Q5. Người thi đấu hàng đầu nói phần lớn tiến bộ đến từ upsolving chứ không từ chính kỳ thi. Cơ chế nào giải thích điều đó?

Có một cách đọc chương này sai ngay từ đầu, và đáng nói trước: đọc như đọc tiểu sử danh nhân, thấy hay, rồi không thay đổi gì. Vì vậy mỗi mục dưới đây được viết theo một khuôn cố định — người này làm gì cụ thể, vì sao nó hiệu quả, và bạn bắt chước được phần nào ngay tuần này. Những gì không bắt chước được, tôi sẽ nói thẳng là không bắt chước được.

Cũng cần nói rõ giới hạn: những phát biểu dưới đây dựa trên sách và tài liệu công khai của chính họ, không phải suy đoán về đời tư. Khi tôi mô tả thói quen của giới thi đấu, đó là những điều lặp đi lặp lại trong lời khuyên công khai của nhiều người giỏi, chứ không phải câu trích nguyên văn của một cá nhân cụ thể.

2.1 Bốn trường phái, một vòng phản hồi

Nếu xếp những người có ảnh hưởng nhất lên một sơ đồ, họ rơi vào bốn trường phái với bốn niềm tin khác nhau về việc “thế nào là làm thuật toán cho tốt”. Điều thú vị là bốn trường phái này không mâu thuẫn; chúng bổ sung nhau, và người mạnh thường lấy từ cả bốn.



Hình 2.1. Bốn trường phái khác nhau ở thứ họ dùng làm thước đo, nhưng giống nhau ở chỗ đều có một thước đo khách quan và đều sửa cách làm dựa trên kết quả đo. Người luyện một mình mà không có thước đo nào là người duy nhất không có vòng phản hồi — và đó là kiểu luyện phổ biến nhất.

Trường phái phân tích: Knuth và việc đếm cho chính xác

Donald Knuth xây cả sự nghiệp trên một niềm tin: muốn hiểu một thuật toán thì phải *đếm* được nó làm bao nhiêu phép toán, chứ không chỉ nói “cỡ $n \log n$ ”. Trong *The Art of Computer Programming* [8], [9], các thuật toán được phân tích tới hằng số, trên một máy ảo mà ông tự thiết kế để không phải phụ thuộc vào máy thật nào.

Hai thói quen của Knuth đáng bắt chước ngay. Thứ nhất là *bài tập có xếp hạng độ khó*: mỗi bài trong TAOCP có một số từ 0 đến 50, trong đó khoảng 20 là bài tập bình thường, còn 50 là bài toán mở nghiên cứu. Ý nghĩa của cách xếp hạng đó không phải để khoe độ khó mà để người đọc *chọn đúng vùng luyện* — ý tưởng mà mục sau sẽ nói kỹ. Thứ hai là thói quen coi việc phát hiện lỗi là có giá trị: Knuth nổi tiếng vì trả tiền thưởng cho mỗi lỗi được tìm thấy trong sách của ông. Thông điệp ẩn sau đó rất thực dụng: một người muốn đúng thì phải *tạo động lực cho việc tìm ra mình sai*, chứ không phòng thủ với nó.

Cũng của Knuth là câu bị trích sai nhiều nhất trong ngành: “tối ưu hoá sớm là gốc rễ của mọi tội lỗi”[10]. Nguyên văn nói rõ hơn nhiều — ý ông là trong phần lớn thời gian ta nên bỏ qua những cải thiện nhỏ, nhưng *không được* bỏ qua cơ hội ở phần mã thực sự quan trọng, và cách biết phần nào quan trọng là đo chứ không đoán. Đây chính là cầu nối sang trường phái kỹ thuật.

Trường phái chứng minh: Dijkstra, Hoare và kỷ luật viết code

Edsger Dijkstra có một quan điểm cực đoan và rất hữu ích: chương trình và chứng minh tính đúng của nó phải được phát triển *cùng lúc*, chứ không phải viết code trước rồi kiểm thử sau[11]. Ông cũng để lại câu nói chính xác về giới hạn của kiểm thử: kiểm thử cho thấy sự *hiện diện* của lỗi, không bao giờ cho thấy sự *vắng mặt* của lỗi.

Nghe có vẻ hàn lâm, nhưng hệ quả thực hành rất cụ thể và rất rẻ: trước khi viết một vòng lặp, hãy viết ra — bằng lời, trong một dòng chú thích — *bất biến* mà vòng lặp đó duy trì. Chỉ một dòng đó thôi đã loại được phần lớn lỗi lệch một đơn vị và lỗi khởi tạo sai, vì mọi câu hỏi kiểu “chạy tới n hay n-1” đều có câu trả lời suy ra từ bất biến. Tony Hoare cấp nền hình thức cho cách làm này[12], còn David Gries biến nó thành giáo trình dạy được[13]. Chương A/04 lấy đúng ý này làm trung tâm.

Cần nói cả mặt trái. Chứng minh hình thức đầy đủ cho mọi chương trình là không thực tế trong công việc hằng ngày, và Dijkstra bị phê phán chính ở điểm đó. Cách dùng hợp lý là dùng *ý thức về bất biến* chứ không phải bộ máy hình thức: viết bất biến ra thành lời, và chỉ hình thức hoá ở những chỗ thật sự tinh vi.

Trường phái kỹ thuật: Bentley và thói quen đo

Jon Bentley viết *Programming Pearls*[6] như những chuyên mục ngắn, mỗi bài xoay quanh một bài toán thật gặp trong công việc. Ba thói quen của ông dùng được ngay.

Thứ nhất là *ước lượng phong bì* — tính nhằm quy mô trước khi cài: bao nhiêu phần tử, mỗi phần tử bao nhiêu byte, một giây làm được cỡ bao nhiêu phép toán. Ước lượng thô loại được những hướng sai hằn với chi phí gần bằng không, và nó là kỹ năng dùng suốt đời chứ không chỉ trong thi đấu.

Thứ hai là quan sát nổi tiếng của ông về *tìm kiếm nhị phân*: ai cũng nghĩ mình viết đúng, còn thực tế thì rất nhiều lập trình viên chuyên nghiệp viết sai khi được yêu cầu viết từ đầu không tra cứu. Bài học không phải “tìm kiếm nhị phân khó” mà là: một thuật toán *đơn giản về ý tưởng* vẫn có thể rất khó viết đúng, nên phần cài đặt xứng đáng được luyện riêng như một kỹ năng độc lập.

Thứ ba là kỷ luật *đo trước khi tối ưu*. Trong bài viết cùng McIlroy về việc kỹ thuật hoá hàm sắp xếp của thư viện C[14], hai người mô tả cách một thuật toán sách vở phải chỉnh sửa nhiều lần mới sống được trong môi trường thật — với dữ liệu thù địch, với mảng đã sắp sẵn, với nhiều phần tử bằng nhau. Đây là ví dụ mẫu mực cho chương E/26.

Trường phái thi đấu: nhận diện bài toán quan trọng hơn thuộc thuật toán

Steven Skiena tổ chức *The Algorithm Design Manual*[15] thành hai nửa rất khác nhau: nửa đầu dạy kỹ thuật, nửa sau là một *danh mục bài toán* — bạn tra “tôi đang có bài toán trông như thế này” và sách chỉ cho bạn nó tên là gì, đã được giải chưa, và nên dùng thư viện nào. Cách tổ chức đó phản ánh một quan sát cốt lõi: trong thực tế, phần khó nhất thường không phải nghĩ ra thuật toán mà là *nhận ra bài toán của mình chính là bài toán kinh điển nào*.

Giới lập trình thi đấu đầy quan sát này tới cùng. Bộ sách *Competitive Programming*[16] và *Guide to Competitive Programming*[17] tổ chức kiến thức thành các mẫu nhận dạng kèm bài luyện; các nguồn cộng đồng như cp-algorithms[18] và thư viện KACTL[7] cấp phần cài đặt đã kiểm chứng. Điểm chung trong lời khuyên luyện tập của những người ở nhóm dẫn đầu, lặp lại rất nhất quán

qua nhiều nguồn công khai, gồm bốn điểm: giải bài *hơi quá sức*, upsolving sau mỗi trận, tự cài lại thay vì đọc code người khác, và luyện có tính giờ.

I Kinh nghiệm người đi trước

Người / trường phái	Điều họ thực sự làm	Nguồn
Donald Knuth	Phân tích tới hằng số, không chỉ tới bậc tiệm cận. Bài tập xếp hạng 0–50 để người đọc chọn đúng vùng luyện. Thường tiền cho ai tìm ra lỗi trong sách của mình.	TAOCP[8], [9]
Edsger Dijkstra	Phát triển chương trình và chứng minh cùng lúc; viết bất biến trước khi viết vòng lặp; coi kiểm thử là công cụ phát hiện lỗi chứ không phải bằng chứng đúng.	<i>A Discipline of Programming</i> [11]
Tony Hoare	Đặt nền logic cho lập luận về chương trình (tiền điều kiện, hậu điều kiện); Quicksort ra đời từ việc tìm cách sắp xếp trước khi tra từ điển — ý tưởng đến từ ràng buộc thực tế.	[12], [19]
Robert Tarjan	Thiết kế cấu trúc dữ liệu song song với công cụ phân tích của nó: khẩu hao và hàm thể năng ra đời để phân tích chính những cấu trúc ông thiết kế.	[20], [21]
Jon Bentley	Ước lượng phong bì trước khi cài; đo trước khi tối ưu; coi việc viết đúng một thuật toán đơn giản là kỹ năng phải luyện riêng.	<i>Programming Pearls</i> [6]
Robert Sedgewick	Kết hợp phân tích toán học với thực nghiệm: cài đặt, đo, rồi đối chiếu với dự đoán lý thuyết; nếu lệch thì một trong hai phía sai.	[22], [23]
Steven Skiena	Xây danh mục bài toán để tra ngược từ “bài của tôi trông thế này”; nhấn mạnh nhận diện bài toán hơn là sáng tạo thuật toán mới.	<i>The Algorithm Design Manual</i> [15]
George Pólya	Bốn bước và một hệ thống câu hỏi để tự hỏi khi bí; dạy <i>quá trình</i> chứ không chỉ kết quả.	<i>How to Solve It</i> [1]
Alan Schoenfeld	Đo thực nghiệm quá trình giải bài; chỉ ra rằng tự giám sát là khác biệt lớn nhất giữa người giỏi và người kém.	[2]
Anders Ericsson	Luyện tập có chủ đích: mục tiêu hẹp, mức hơi quá sức, phản hồi ngay, sửa sai lập tức. Số giờ không tự sinh ra năng lực.	[3], [24]
Richard Hamming	Chọn bài toán quan trọng để làm; giữ thói quen suy nghĩ có hệ thống về việc mình đang làm gì và vì sao.	<i>The Art of Doing Science and Engineering</i> [25]
Peter Norvig	Không có đường tắt mười ngày; năng lực đến từ nhiều năm luyện có phản hồi, đọc code người khác, và làm dự án thật.	[26]
Giới lập trình thi đấu	Bốn thói quen lặp lại nhất quán: giải bài hơi quá sức, upsolving sau mỗi trận, tự cài lại thay vì chép, luyện có tính giờ; kèm một thư viện mẫu tự viết và đã kiểm chứng.	[7], [16], [17]

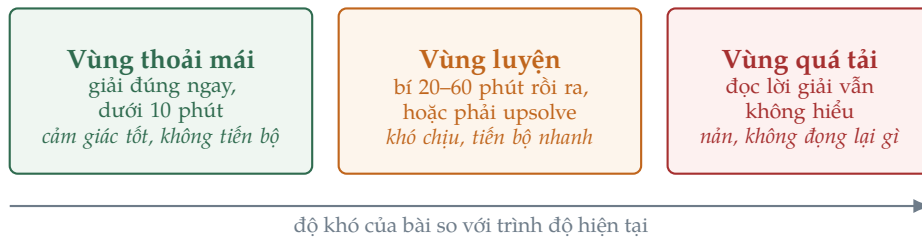
I 2.2 Luyện tập có chủ đích, dịch sang việc giải thuật

Nghiên cứu của Ericsson và cộng sự[3] đưa ra một kết luận đơn giản nhưng khó nuốt: số giờ luyện tập chỉ dự báo được năng lực khi việc luyện có bốn tính chất — mục tiêu hẹp và cụ thể, mức khó hơi vượt khả năng hiện tại, phản hồi gần như tức thì, và có sửa sai chứ không chỉ làm lại. Thiếu bất kỳ tính chất nào thì thời gian bỏ ra vẫn trôi, nhưng năng lực đứng yên.

Dịch sang việc luyện thuật toán, bốn tính chất trở thành bốn quy tắc rất cụ thể.

Mục tiêu hẹp. Không luyện kiểu “hôm nay học quy hoạch động”. Luyện kiểu “hôm nay làm năm bài quy hoạch động mà trạng thái là hai chiều, và với mỗi bài tôi phải viết ra định nghĩa trạng thái trước khi gõ code”. Mục tiêu hẹp cho phép biết mình có đạt hay không.

Vùng khó hơi vượt. Đây là quy tắc bị vi phạm nhiều nhất. Giải bài dễ cho cảm giác tốt — giải nhanh, đúng ngay, tự tin. Nhưng nếu bạn giải đúng ngay trong mười phút thì bài đó không dạy bạn gì. Vùng luyện đúng là vùng mà bạn giải được khoảng một nửa số bài, và nửa còn lại phải vật lộn rồi mới ra hoặc phải upsolve. Hình 2.2 vẽ ba vùng.



Hình 2.2. Ba vùng độ khó. Sai lầm phổ biến nhất khi tự luyện là ở lì trong vùng thoải mái vì nó cho cảm giác thành công; sai lầm ngược lại là nhảy thẳng vào vùng quá tải vì nghĩ rằng làm bài khó thì tiến bộ nhanh. Vùng luyện nằm giữa: bạn giải được khoảng một nửa, và luôn hơi khó chịu.

Phản hồi ngay. Đây là lý do các trang chấm bài tự động có giá trị lớn hơn vẻ ngoài của chúng: bạn biết đúng hay sai trong vài giây. Nhưng phản hồi “sai ở test 12” vẫn còn nghèo; phản hồi tốt hơn là stress test tự viết (chương E/25), vì nó chỉ thẳng ra *ca dữ liệu* nào làm hỏng.

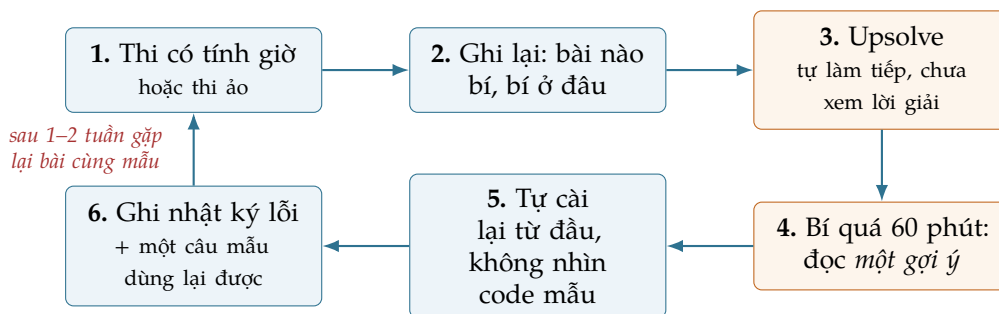
Sửa sai, không chỉ làm lại. Đây là chỗ nhét ký lỗi vào cuộc. Sau mỗi lần sai, ghi ba dòng: tôi đã hiểu sai điều gì, dấu hiệu nào trên đề lẽ ra đã gợi ý đúng, lần sau tôi sẽ kiểm tra gì. Sau vài tháng, nhét ký này sẽ cho bạn thấy một sự thật khó chịu và hữu ích: bạn sai đi sai lại đúng vài kiểu.

TRỰC GIÁC

Vì sao vùng thoải mái không tạo tiến bộ? Vì học xảy ra khi *dự đoán của bạn bị sai*. Giải một bài mà bạn biết chắc cách làm thì không có dự đoán nào bị vi phạm, nên không có gì để cập nhật. Cảm giác khó chịu khi bí chính là dấu hiệu của việc mô hình trong đầu bạn đang được sửa. Đó là lý do luyện đúng cách gần như luôn thấy không thoải mái, và vì sao cảm giác “buổi luyện hôm nay trôi chảy quá” thường là tin xấu.

2.3 Vòng upsolving — cỗ máy tiến bộ của dân thi đấu

Người mới thường nghĩ tiến bộ đến từ việc dự thi. Thực tế thì kỳ thi chỉ là phép đo; phần tạo ra tiến bộ nằm ở việc bạn làm gì trong vài ngày sau đó. Vòng lặp mà giới thi đấu dùng gần như thống nhất được vẽ ở hình 2.3.



Hình 2.3. Vòng upsolving. Hai ô tô hổ phách là phần tạo ra phần lớn tiến bộ, và cũng là phần hay bị cắt nhất vì nó không vui. Bước 5 — tự cài lại thay vì chép — là bước phân biệt “hiểu lời giải” với “làm được”; rất nhiều người tưởng mình đã học xong sau bước 4.

Cơ chế đằng sau vòng này khá rõ. Trong lúc thi, bạn ở trạng thái sức ép và đã tự mình đào rất sâu vào bài — tức là đã nạp đủ ngữ cảnh. Ngay sau đó, mỗi mẫu thông tin mới (một gợi ý, một ý tưởng) sẽ dính vào đúng chỗ trong đầu bạn, vì bạn biết chính xác nó lắp vào lỗ hổng nào. Đọc lời giải khi *chưa* vật lộn thì không có lỗ hổng nào để lắp, và thông tin trôi qua.

Một biến thể đáng dùng khi không có kỳ thi nào: *thi ảo*. Lấy một đề cũ, đặt đồng hồ đúng thời lượng thật, và làm nghiêm túc. Điểm quan trọng là tính giờ — vì kỹ năng phân bổ thời gian và kỹ năng chịu sức ép chỉ luyện được khi có sức ép.

CẠM BẮY

Cạm bẫy phổ biến nhất trong upsolving: đọc lời giải, gật gù “à hoá ra thế, dễ mà”, rồi chuyển bài. Cảm giác hiểu khi đọc là *cảm giác quen thuộc*, không phải năng lực tái tạo. Bài kiểm tra duy nhất đáng tin: đóng lời giải lại và tự cài từ trang trắng. Rất nhiều người phát hiện ra mình không làm được ở đúng bước này — và đó là thông tin quý, không phải điều đáng xấu hổ.

2.4 Đọc sách thế nào cho có tác dụng

Sách thuật toán bị đọc sai cách nhiều hơn bất kỳ loại sách kỹ thuật nào. Có ba cách đọc, và chỉ một cách tạo ra năng lực.

Đọc như tiểu thuyết — đọc từ đầu tới cuối, thấy hiểu. Cách này gần như không đọng lại gì sau ba tuần, vì không có lần nào bạn phải tự truy xuất kiến thức từ trí nhớ.

Đọc như từ điển — chỉ mở khi cần tra. Cách này hợp với nửa sau sách của Skiena hoặc với cp-algorithms, nhưng không xây được nền.

Đọc như phòng tập — đọc một mục, gấp sách, tự viết lại ý chính, rồi làm bài tập của mục đó *trước* khi đọc mục sau. Đây là cách duy nhất tạo ra năng lực, và nó chậm hơn hẳn: một chương có thể mất cả tuần. Knuth thiết kế TAOCP theo đúng giả định này — bài tập là một phần của nội dung, không phải phụ lục.

Bảng dưới đây là bản đồ sách: quyển nào cho vai trò nào. Đừng đọc song song quá hai quyển; chọn một quyển làm trục chính và dùng phần còn lại để tra.

Sách	Vai trò	Dùng khi nào
Pólya, <i>How to Solve It</i> [1]	Quy trình nghĩ	Đọc trước tất cả; mỏng, đọc trong một buổi
Zeitz[4], Engel[5]	Heuristic giải toán	Khi muốn luyện tư duy chứ không luyện thuật toán
Skiena[15]	Nhận diện bài toán	Trực chính tốt nhất cho người tự học; nửa sau là danh mục tra
CLRS[27]	Tham chiếu chuẩn	Tra định nghĩa, chứng minh chuẩn; <i>không</i> nên đọc tuần tự
Kleinberg–Tardos[28]	Dạy cách thiết kế	Khi muốn học cách <i>nghĩ ra</i> thuật toán, đặc biệt tham lam và luồng
Erickson[29]	Đệ quy và chứng minh	Miễn phí; phần quay lui và quy hoạch động rất tốt
Sedgewick–Wayne[22]	Cài đặt và đo	Khi muốn code chạy được, có thực nghiệm
Bentley[6]	Kỹ thuật thực dụng	Đọc rải rác, mỗi lần một chương ngắn
Laaksonen[17], Halim[16]	Thi đấu	Trực chính nếu mục tiêu là contest
Dijkstra[11], Gries[13]	Tính đúng	Khi bạn liên tục viết code gần đúng
Knuth[8]	Phân tích sâu	Tra một chủ đề cụ thể; không đọc tuần tự trừ khi có nhiều năm
Graham–Knuth–Patashnik[30]	Toán rời rạc	Khi phân tích độ phức tạp đệ quy tổng và truy hồi khó
Cormen <i>et al.</i> , Garey–Johnson[31]	Độ khó	Khi nghi ngờ bài của mình là NP-khó
McDowell[32]	Phỏng vấn	Bốn tới sáu tuần trước phỏng vấn, không sớm hơn

2.5 Những gì không bắt buộc được, và những gì bắt buộc được ngay

Trung thực mà nói, có những thứ trong danh sách trên bạn không nên cố bắt buộc. Knuth dành hàng chục năm cho một bộ sách; kiểu đầu tư đó chỉ hợp lý với một người có đúng mục tiêu của ông. Dijkstra làm việc phần lớn bằng bút và giấy, gần như không dùng máy tính — cách đó phù hợp với thời của ông và với tính chất công việc của ông, không phù hợp với người cần giao sản phẩm hàng tuần. Một số kỳ thủ lập trình bắt đầu từ rất sớm và luyện với cường độ mà người đi làm không thể có; so sánh bản thân với họ theo thang thời gian là cách nhanh nhất để nản vô ích.

Nhưng có sáu thứ bắt buộc được ngay trong tuần này, và chúng chiếm phần lớn giá trị:

1. Trước mỗi vòng lặp, viết một dòng bất biến bằng lời (Dijkstra).
2. Trước khi cài, ước lượng phong bì: n bao nhiêu, mỗi thao tác bao nhiêu, tổng cỡ bao nhiêu (Bentley).
3. Chọn bài ở vùng bạn giải được khoảng một nửa, không dễ hơn (Ericsson).
4. Sau mỗi trận hoặc mỗi buổi luyện, upsolve ít nhất một bài tới nơi tới chốn, tự cài lại từ trang trắng.
5. Giữ nhật ký lỗi ba dòng cho mỗi lần sai; đọc lại mỗi tháng.
6. Sau mỗi bài giải xong, viết một câu về ý tưởng cốt lõi không nhắc tới đề bài (Pólya, bước 4).

Sáu việc này cộng lại tốn chừng mười phút mỗi buổi luyện. Điểm chung của chúng: cả sáu đều là cách *đóng vòng phản hồi* — đúng thứ mà hình 2.1 nói là mẫu số chung của cả bốn trường phái.

2.6 Gốc rễ: vì sao ta biết những điều này về luyện tập

Những lời khuyên ở trên nghe như kinh nghiệm dân gian, nhưng phần lớn chúng có nguồn thực nghiệm, và nguồn ấy không đến từ ngành máy tính mà từ nghiên cứu về cờ vua.

Năm 1946, Adriaan de Groot — vừa là nhà tâm lý học vừa là kỳ thủ — đặt các kỳ thủ mạnh và yếu trước cùng một thế cờ và yêu cầu họ nói to suy nghĩ. Kết quả trái với kỳ vọng: kiện tướng *không* tính sâu hơn người trung bình bao nhiêu. Cái họ hơn là ngay từ vài giây đầu, họ đã nhìn ra một nhóm nhỏ nước đi đáng xét, trong khi người yếu hơn phải rà nhiều nước vô nghĩa. Nói cách khác, ưu thế nằm ở *nhận dạng*, không ở sức tính.

Đến 1973, William Chase và Herbert Simon — chính Simon của *Human Problem Solving* nhắc ở chương A/01 — làm thí nghiệm quyết định. Họ cho kỳ thủ nhìn một thế cờ trong vài giây rồi dựng lại. Kiện tướng dựng lại gần như hoàn hảo, người mới thì không. Nhưng khi các quân được đặt *ngẫu nhiên* thay vì theo thế cờ thật, ưu thế biến mất gần hết. Kết luận: chuyên gia không có trí nhớ tốt hơn; họ nhìn thấy *đơn vị lớn hơn* — những cụm quân có nghĩa, gọi là chunk. Trí nhớ của họ chỉ chứa vài chục đơn vị như mọi người, nhưng mỗi đơn vị đóng gói nhiều thông tin hơn.

Hai kết quả đó giải thích trực tiếp vì sao chương A/01 nhấn mạnh việc “viết một câu về ý tưởng cốt lõi không nhắc tới đề bài”. Câu đó chính là một chunk đang được tạo ra. Người nói “bài này dùng hai con trỏ” và người nói “khi cả hai đầu đều đơn điệu thì quét từ hai phía” đang lưu hai thứ rất khác nhau: một cái tên, và một điều kiện kích hoạt. Chỉ cái thứ hai mới tự nhận ra chính nó khi gặp bài mới.

Nhánh nghiên cứu thứ hai bắt đầu từ Benjamin Bloom, người phỏng vấn những cá nhân đạt đỉnh cao trong nhiều lĩnh vực và thấy một mẫu chung: nhiều năm luyện có hướng dẫn, tăng dần độ khó, với người dạy đóng vai trò cấp phản hồi. Ericsson và cộng sự hình thức hoá mẫu đó thành khái niệm luyện tập có chủ đích[3], [24]. Cần nói thẳng phần bị tranh cãi: con số “10 000 giờ” là cách phổ biến hoá của người khác, không phải kết luận của nghiên cứu, và mức độ mà luyện tập giải thích được khác biệt năng lực vẫn còn đang tranh luận trong giới nghiên cứu. Phần lỗi thì vững: luyện có mục tiêu, có phản hồi, ở mức hơi quá sức hiệu quả hơn hẳn lặp lại thụ động — và đó là toàn bộ phần ta dùng.

Còn bản thân môi trường luyện thuật toán hiện đại thì trẻ hơn nhiều. Các kỳ thi lập trình đại học bắt đầu từ thập niên 1970 và lớn dần thành ICPC; kỳ thi tin học quốc tế dành cho học sinh phổ thông (IOI) bắt đầu năm 1989; các nền tảng chấm bài trực tuyến xuất hiện từ cuối thập niên 1990 và bùng nổ trong thập niên 2010. Điều mà những nền tảng này thay đổi không phải nội dung kiến thức mà là *độ trễ phản hồi*: từ vài tuần chờ thầy chấm xuống còn vài giây. Nếu tin vào Ericsson, thì đó chính là biến số quan trọng nhất, và nó giải thích vì sao trình độ trung bình của người luyện nghiêm túc ngày nay cao hơn hẳn hai mươi năm trước.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1946 — de Groot	Muốn biết kiện tướng cờ nghĩ khác người thường ở đâu.	Kiến tướng không tính sâu hơn; họ nhận dạng nhanh hơn. Ưu thế nằm ở nhận dạng chứ không ở sức tính.
1973 — Chase & Simon	Kiểm chứng: đó là trí nhớ tốt hơn hay cách tổ chức tri thức khác?	Ưu thế biến mất với thế cờ ngẫu nhiên ⇒ chuyên gia lưu các <i>khối có nghĩa</i> . Giải thích vì sao phải phát biểu ý tưởng ở dạng tổng quát.
1985 — Bloom	Phỏng vấn người đạt đỉnh cao ở nhiều ngành để tìm mẫu chung.	Nhiều năm luyện có hướng dẫn và tăng dần độ khó; vai trò của người cấp phản hồi.
1993 — Ericsson và cộng sự	Vì sao cùng số giờ mà kết quả khác nhau.	Bốn điều kiện của luyện tập có chủ đích. Phần “10 000 giờ” là diễn giải của người khác và còn tranh cãi.
1989 và sau đó — IOI, chấm bài trực tuyến	Muốn đánh giá được kỹ năng lập trình ở quy mô lớn.	Độ trễ phản hồi tụt từ vài tuần xuống vài giây — thay đổi lớn nhất với người tự học trong 30 năm qua.

2.7 Liên hệ ngang: nghề nào cũng phải giải bài toán luyện tập này

Nếu bốn điều kiện của luyện tập có chủ đích là đúng, thì mọi nghề đòi hỏi kỹ năng cao đều phải tự phát minh ra cách thoả chúng. Và quả thật là vậy — nhìn sang các nghề khác, bạn sẽ thấy những cơ chế trùng khớp đến bất ngờ với upsolving và nhật ký lỗi, chỉ khác tên gọi. Hai chỗ đáng học nhất là y khoa và hàng không, vì họ đã *thể chế hoá* việc học từ lỗi thay vì phó mặc cho ý chí cá nhân.

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Âm nhạc	Tập chậm từng đoạn khó, tách tay, dùng máy đếm nhịp tăng dần tốc độ	Giống: cô lập đúng phần yếu thay vì chơi lại cả bản — tương đương việc luyện riêng một dạng bài. Khác: nhạc công có phản hồi tức thì bằng tai, còn bạn phải tự tạo phản hồi bằng chấm bài và stress test.
Thể thao	Chu kỳ hoá: xen kẽ giai đoạn tăng khối lượng, tăng cường độ và phục hồi	Giống: cường độ cao liên tục không bền, cần xen kẽ. Gợi ý cho lộ trình ở E/28: xen tuần thi đấu với tuần học kiến thức mới. Khác: giải thuật không có giới hạn sinh lý, giới hạn là sự chú ý và động lực.
Y khoa	Hội chẩn tử vong và biến chứng (M&M): mỗi ca xấu được mổ xẻ công khai, không quy tội cá nhân	Đây chính là nhật ký lỗi được thể chế hoá, và nó hiệu quả hơn nhật ký cá nhân vì bài học được chia sẻ. Bắt buộc được: viết công khai bài phân tích lỗi của mình.
Hàng không	Bảng kiểm (checklist), báo cáo sự cố phi trường phạt, huấn luyện trên buồng lái mô phỏng	Bảng kiểm ứng với danh sách trường hợp biên trước khi nộp bài (E/25). Buồng lái mô phỏng ứng với thi ảo: rủi ro bằng không, lặp lại được, có thể cố tình tạo tình huống xấu.
Quân sự	Rà soát sau hành động (after-action review): điều dự định, điều đã xảy ra, vì sao khác, lần sau làm gì	Bốn câu này gần như trùng khít với bước 4 của Pólya và nội dung nhật ký lỗi. Điểm mạnh: nó là quy trình bắt buộc, không phụ thuộc tâm trạng.
Nghiên cứu khoa học	Số thí nghiệm, tái lập kết quả, bình duyệt	Số thí nghiệm chính là nhật ký lỗi. Bình duyệt là phản hồi từ người ngoài — thứ mà người tự học thiếu nhất; thay thế gần nhất là nhờ người khác đọc code hoặc viết editorial công khai.
Học máy	Vòng huấn luyện: mất mát trên tập kiểm tra, dừng sớm, học theo chương trình (curriculum learning)	Rất giống về cấu trúc: mẫu quá dễ cho gradient nhỏ, mẫu quá khó cho gradient nhiều; vùng học hiệu quả nằm giữa — đúng hình 2.2. Khác: mô hình không tự chọn được bài, còn bạn thì có.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì thói quen của họ không phải sở thích cá nhân mà là cách đóng vòng phản hồi, và vòng phản hồi thì chỉ có vài cách đóng. Bốn trường phái ở hình 2.1 khác nhau ở thước đo — toán, logic, đồng hồ, số bài giải được — nhưng ai cũng phải có một thước đo khách quan và phải sửa cách làm theo kết quả đo. Người luyện không có thước đo nào là người duy nhất không tiến bộ.
- A2. Vì đọc tạo ra *cảm giác quen thuộc*, còn làm bài tập tạo ra *khả năng truy xuất*. Hai thứ đó khác nhau: bạn có thể thấy mọi dòng chứng minh đều hiển nhiên khi đọc mà vẫn không tái tạo được nó khi gấp sách. Bài tập ép bạn truy xuất, và chính lúc truy xuất khó khăn thì trí nhớ mới được củng cố. Cách xếp hạng 0–50 của Knuth còn giúp bạn chọn đúng vùng khó thay vì làm hết mọi bài.
- A3. Nó đổi thứ tự công việc. Thay vì gõ code rồi tìm ca dữ liệu làm nó sai, bạn viết ra điều kiện cần giữ đúng *trước*, và code trở thành cách thực hiện điều kiện đó. Cụ thể nhất là một dòng bắt biên trước mỗi vòng lặp: khi có nó, các câu hỏi “bắt đầu từ 0 hay 1”, “chạy tới n hay n-1”, “khởi tạo bằng 0 hay $-\infty$ ” đều được suy ra thay vì đoán rồi thử.

- A4. Vì “nhiều bài” thường có nghĩa là nhiều bài *vừa sức*, mà vừa sức thì không tạo ra sai lệch dự đoán để học. Sau vài tháng bạn thành thạo hơn ở những gì đã biết và không mở rộng được gì mới. Điều kiện Ericsson đòi hỏi là mức hơi quá sức cộng phản hồi cộng sửa sai; giải ít bài hơn nhưng chọn ở vùng luyện, có upsolve và có nhật ký lỗi thì thoả cả ba.
- A5. Vì trong lúc thi bạn đã tự đào rất sâu vào bài, tức là đã tạo sẵn những “lỗ hổng” rất cụ thể trong hiểu biết. Ngay sau đó, một gợi ý nhỏ sẽ rơi đúng vào lỗ hổng và dính lại. Nếu đọc lời giải khi chưa vật lộn thì không có lỗ hổng nào, thông tin trôi qua và bạn chỉ thu được cảm giác đã hiểu. Đó cũng là lý do bước “tự cài lại từ trang trắng” không được bỏ.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn học chơi đàn. Có bốn kiểu thầy: người bắt bạn đếm nhịp thật chính xác, người bắt bạn hiểu vì sao hợp âm này đi với hợp âm kia, người bắt bạn thu âm lại rồi nghe xem chỗ nào sai, và người bắt bạn lên sân khấu đúng giờ. Bốn kiểu ấy tương ứng với bốn nhóm người trong chương này. Điều họ giống nhau là ai cũng có một cách để *biết mình đang sai ở đâu* — đếm nhịp, lý thuyết, bản thu, khán giả. Người tập đàn một mình trong phòng, không thu âm, không ai nghe, thì tập bao nhiêu năm cũng chỉ giỏi lên chút ít, vì không có gì chỉ cho họ chỗ sai.

Việc luyện thuật toán y hệt. Bạn cần một cái “bản thu”: chấm bài tự động, stress test, đồng hồ bấm giờ, hay đơn giản là quyển sổ ghi mỗi lần bạn sai. Và bạn cần chọn bài ở mức hơi khó — dễ quá thì như tập lại bài đã thuộc, khó quá thì như ngồi trước bản nhạc không đọc nổi nốt nào. Cuối cùng, sau mỗi lần tập, hãy quay lại đúng chỗ bạn vấp và tự chơi lại từ đầu, chứ đừng nghe người khác chơi rồi tưởng mình chơi được.

Một câu: Người giỏi không luyện nhiều hơn bạn nhiều đến thế; họ luyện ở mức khó hơn một chút và luôn biết mình vừa sai ở đâu.

Chỗ không rút gọn được: Có phần thật sự không rút gọn được: thời gian. Vòng phản hồi tốt làm mỗi giờ luyện đáng giá hơn, nhưng không có cách nào bỏ qua vài trăm giờ đầu tiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn điều kiện của luyện tập có chủ đích	Mục tiêu hẹp; mức hơi quá sức; phản hồi gần như tức thì; có sửa sai chứ không chỉ làm lại. Thiếu một điều thì thời gian trôi mà năng lực đứng.
••• Vùng luyện đúng	Vùng bạn giải được khoảng một nửa số bài; luôn hơi khó chịu. Dễ quá thì không có dự đoán nào bị sai để mà học.
••• Vòng upsolving	Thi tính giờ → ghi chỗ bí → tự làm tiếp → chỉ đọc <i>một</i> gợi ý khi bí quá 60 phút → tự cài lại từ trang trắng → ghi nhật ký lỗi.
••• Kiểm tra “đã học được” đáng tin duy nhất	Đóng lời giải, tự cài từ trang trắng. Cảm giác hiểu khi đọc chỉ là cảm giác quen thuộc.
••• Câu của Dijkstra về kiểm thử	Kiểm thử cho thấy sự hiện diện của lỗi, không bao giờ cho thấy sự vắng mặt của lỗi — nên tính đúng phải được xây vào lúc viết, không thêm vào lúc kiểm.
•• Câu Knuth bị trích sai	“Tối ưu hoá sớm là gốc rễ mọi tội lỗi” — nguyên ý là bỏ qua cái thiện nhỏ trong phần lớn thời gian, nhưng phải đo để tìm ra phần thực sự quan trọng và tối ưu ở đó.
•• Đóng góp phương pháp của Tarjan	Thiết kế cấu trúc dữ liệu cùng với công cụ phân tích của nó; khẩu hao và hàm thể năng ra đời từ nhu cầu đó.
•• Ba thói quen của Bentley	Ước lượng phong bì trước khi cài; đo trước khi tối ưu; coi việc viết đúng thuật toán đơn giản là kỹ năng phải luyện riêng.
•• Vì sao Skiena tổ chức sách thành danh mục	Vì trong thực tế, phần khó nhất thường là nhận ra bài của mình chính là bài kinh điển nào, chứ không phải nghĩ ra thuật toán mới.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Ba cách đọc sách và cách nào có tác dụng	Đọc như tiểu thuyết (trôi), như từ điển (tra được, không xây nền), như phòng tập (gấp sách, viết lại, làm bài tập trước khi sang mục sau) — chỉ cách thứ ba tạo năng lực.
●● Nội dung ba dòng của nhật ký lỗi	Tôi hiểu sai điều gì; dấu hiệu nào lẽ ra đã gợi ý đúng; lần sau tôi sẽ kiểm tra gì.
● Thi ảo	Lấy đề cũ, đặt đồng hồ đúng thời lượng thật, làm nghiêm túc — dùng khi không có kỳ thi thật, để luyện phân bổ thời gian dưới sức ép.
● Bốn trường phái và thước đo của họ	Phân tích (toán), chứng minh (logic), kỹ thuật (đồng hồ trên máy thật), thi đấu (số bài giải được có tính giờ).

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi đi làm, mỗi ngày chỉ có một giờ. Nên phân bổ thế nào cho hiệu quả nhất?

| Một cách phân bổ dùng được: 10 phút ôn lại (đọc nhật ký lỗi và bảng active recall của chương đang học), 40 phút giải một bài ở vùng luyện có bầm giờ, 10 phút cho bước 4 — viết một câu về ý tưởng cốt lõi và ghi nhật ký nếu sai. Điều quan trọng hơn tỉ lệ là tính đều đặn: một giờ mỗi ngày trong sáu tháng hơn hẳn tám giờ mỗi cuối tuần, vì trí nhớ củng cố theo số lần truy xuất chứ không theo tổng thời gian.

2. Tôi nên đọc hết CLRS không?

| Gần như chắc chắn là không, trừ khi bạn đang học một khoá có giáo trình đó. CLRS được thiết kế làm tham chiếu chuẩn: định nghĩa chính xác, chứng minh đầy đủ, bao phủ rộng. Đọc tuần tự thì rất chậm và bạn sẽ dừng ở khoảng chương 8. Cách dùng đúng: lấy một quyển khác làm trục chính (Skiena nếu tự học, Laaksonen nếu thi đấu), và mở CLRS khi cần một chứng minh chuẩn hoặc một chi tiết mà trục chính nói lướt.

3. Đọc code của người giỏi có ích không, hay chỉ nên tự viết?

| Cả hai, nhưng đúng thứ tự. Đọc code trước khi tự vật lộn thì bạn chép được kiểu viết mà không hiểu vì sao; đọc sau khi đã tự cài xong thì rất có giá trị, vì bạn thấy ngay chỗ họ ngắn hơn, ít trường hợp riêng hơn, hoặc chọn biểu diễn dữ liệu khôn hơn. Một bài tập tốt: cài xong rồi so với thư viện tham chiếu như KACTL[7] và tự hỏi vì sao họ chọn cách đó.

4. Xây thư viện mẫu cho riêng mình có đáng không, khi đã có thư viện công khai?

| Đáng, vì lợi ích chính không nằm ở đoạn code mà ở quá trình viết. Khi tự cài DSU hay segment tree tới mức đúng và gọn, bạn buộc phải hiểu từng chi tiết, và về sau bạn sửa được nó cho biến thể lạ — điều không làm được với code chép về. Nguyên tắc: mỗi đoạn trong thư viện riêng của bạn phải do bạn viết và đã qua stress test.

5. Làm sao biết mình đang ở vùng luyện chứ không phải vùng thoải mái?

| Bằng tỉ lệ giải được. Nếu bạn giải đúng ngay gần như mọi bài trong dưới 15 phút, bạn đang ở vùng thoải mái — nâng độ khó. Nếu bạn không giải được gì trong hai buổi liên tiếp và đọc lời giải vẫn thấy mù mờ, bạn đang ở vùng quá tải — hạ xuống. Điểm cân bằng là khoảng một nửa số bài giải được sau khi vật lộn, nửa còn lại phải upsolve.

6. Nghiên cứu về luyện tập có chủ đích có bị thổi phồng không?

| Có phần. Kết luận mạnh kiểu “chỉ cần 10 000 giờ là thành chuyên gia” là cách diễn giải phổ biến hoá, và bị tranh luận nhiều trong giới nghiên cứu; các yếu tố khác như điểm xuất phát và điều kiện học tập đều có vai trò. Nhưng phần lõi thì vững và cũng là phần ta dùng ở đây: luyện có mục tiêu, có phản hồi, ở mức hơi quá sức thì hiệu quả hơn hẳn luyện lặp lại thụ động. Ta không cần phần bị tranh cãi để dùng được phần lõi.

7. Chuyên gia cờ vua nhớ thể cờ giỏi hơn người thường — điều đó liên quan gì tới thuật toán?

| Liên quan trực tiếp, và đây là một trong những kết quả kinh điển của tâm lý học nhận thức: ưu thế trí nhớ đồ biến mất khi các quân được đặt ngẫu nhiên. Nghĩa là chuyên gia không nhớ tốt hơn, họ *nhìn thấy đơn vị lớn hơn* — những cụm có nghĩa. Với thuật toán cũng vậy: người có kinh nghiệm không nhớ nhiều dòng code hơn, họ nhìn một đề bài và thấy “đây là bài ghép đôi trá hình”. Xây các khối có nghĩa đó chính là việc mà bước 4 của Pólya và câu “phát biểu ý tưởng không nhắc đề bài” đang làm.

8. Tôi hay bỏ dở lộ trình sau vài tuần. Có mẹo nào không?

| Ba điều chỉnh có hiệu quả rõ. Thứ nhất, hạ mục tiêu hằng ngày xuống mức bạn chắc chắn làm được kể cả ngày bận — một bài, hoặc thậm chí 20 phút; tính liên tục quan trọng hơn khối lượng. Thứ hai, làm cho tiến bộ nhìn thấy được: một bảng đánh dấu ngày, hoặc danh sách bài đã upsolve. Thứ ba, ghép việc luyện vào một khung giờ cố định thay vì “khi nào rảnh”. Chương E/28 có lộ trình cụ thể theo tuần đã tính tới ba điều này.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tự viết lại tìm kiếm nhị phân từ trang trắng, không tra cứu	Bài tập của Bentley[6]	Viết bất biến trước; sau đó stress test với mảng rỗng, một phần tử, nhiều phần tử trùng nhau.
● Đọc mục 1 của <i>How to Solve It</i> và áp bốn bước vào một bài bạn đang bí	[1]	Mục tiêu là tập nhận ra mình đang ở bước nào, đặc biệt phát hiện lúc bạn nhảy cóc sang bước 3.
● Lập nhật ký lỗi và ghi ba dòng cho năm lần sai gần nhất	Bài tập tự tổ chức	Sau năm mục, tự phân loại: sai do hiểu đề, sai do ý tưởng, sai do cài đặt. Tỷ lệ này quyết định bạn nên luyện gì.
●● Chọn một đề contest cũ và làm thi ảo đúng thời lượng	Codeforces / AtCoder / CSES[33]	Ghi lại mốc thời gian mỗi lần đổi bài; sau đó đối chiếu với hình phân bố thời gian ở chương A/01.
●● Upsolve một bài trong đề vừa làm, tự cài lại từ trang trắng	Cùng đề trên	Bước 5 của vòng upsolving; đây là bài tập quan trọng nhất trong cả danh sách này.
●● Cài DSU và segment tree cho thư viện riêng, có stress test	[7] để đối chiếu	So sánh code của bạn với thư viện tham chiếu và giải thích từng khác biệt — đừng chép, hãy giải thích.
●● Chọn một chương của Skiena, làm hết bài tập trước khi đọc chương sau	[15]	Luyện cách đọc “như phòng tập”; ghi lại thời gian thật để biết tốc độ đọc bền vững của bạn.
●●● Viết editorial cho một bài bạn vừa giải, như thể dạy người khác	Bài tập kiểu Feynman	Chỗ bạn viết không trôi chảy là chỗ bạn chưa hiểu; đó là mục đích duy nhất của bài tập này.
●●● Lấy một thuật toán bạn “đã biết” và tự chứng minh tính đúng bằng bất biến	Chương A/04	Chọn Dijkstra hoặc thuật toán tham lam xếp lịch; nếu không viết được chứng minh, bạn mới chỉ nhớ chứ chưa hiểu.
●●● Thiết kế lộ trình 12 tuần cho riêng bạn, có mục tiêu đo được mỗi tuần	Chương E/28 làm mẫu	Mỗi tuần phải có một tiêu chí kiểm tra khách quan, không phải “học xong chương X”.

Kết chương và đọc tiếp

Chương này rút gọn được thành một câu: những người giỏi nhất khác nhau ở thước đo họ chọn, nhưng giống nhau ở chỗ ai cũng có một thước đo và ai cũng sửa cách làm theo kết quả đo. Bạn không sao chép được hoàn cảnh của họ, nhưng sáu thói quen ở mục cuối thì sao chép được ngay, và chúng chính là phần tạo ra phần lớn giá trị.

Còn bỏ ngỏ hai câu hỏi thực dụng mà chương này liên tục giả định đã có câu trả lời: thước đo “đủ nhanh” là gì, và thước đo “chắc chắn đúng” là gì. Hai chương tiếp theo cấp đúng hai thước đo đó — A/03 cho chi phí, A/04 cho tính đúng. Sau đó, E/28 sẽ biến toàn bộ chương này thành một lịch luyện cụ thể theo tuần.

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, "The Role of Deliberate Practice in the Acquisition of Expert Performance," *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [7] KTH Royal Institute of Technology. "KACTL: KTH Algorithm Competition Template Library." (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [10] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [11] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [12] C. A. R. Hoare, "An Axiomatic Basis for Computer Programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] D. Gries, *The Science of Programming*. Springer, 1981.
- [14] J. L. Bentley and M. D. McIlroy, "Engineering a Sort Function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [15] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [16] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [17] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [18] e-maxx / cp-algorithms contributors. "cp-algorithms: Algorithms for Competitive Programming." (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [19] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [20] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [21] R. E. Tarjan, "Amortized Computational Complexity," *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [22] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [23] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [24] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.

- [25] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.
- [26] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [34] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [35] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [36] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [37] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [38] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [39] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [40] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Mô hình máy và độ phức tạp (Cost Models and Complexity)

PHỤ THUỘC A/01 cho vai trò của chương này trong bước 2 của Pólya. Phần giải hệ thức truy hồi dùng vài công cụ tổng ở A/05, nhưng đọc được trước.

TÓM TẮT

Chương này cấp cái thước thứ nhất: *ý tưởng của tôi có đủ nhanh không*. Nó bắt đầu từ mô hình máy mà mọi phân tích ngầm giả định, đi qua ký hiệu tiệm cận, rồi tới ba công cụ dùng hằng ngày — bảng chuyển từ giới hạn đề bài sang độ phức tạp cho phép, cách giải hệ thức truy hồi, và phân tích khấu hao. Phần cuối phá bỏ sự ngây thơ của mô hình: hằng số, bộ nhớ và cache có thể lật ngược kết quả tiệm cận ở kích thước thật. Nếu chỉ nhớ một điều: độ phức tạp không phải để khoe mà để *loại bỏ ý tưởng trước khi gõ code*, và bảng ở mục 3 là công cụ đáng thuộc nhất chương.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình RAM (random access machine)** — mô hình máy giả định: mỗi phép toán số học, so sánh, gán, và truy cập một ô nhớ bất kỳ đều mất một đơn vị thời gian. Mọi ký hiệu $O(\cdot)$ trong sách vở đều ngầm nói về mô hình này.
- **kích thước dữ liệu vào n** — đại lượng mà ta đo chi phí theo nó. Phải nói rõ n là gì — số phần tử, số đỉnh, số bit — vì cùng một thuật toán có thể là đa thức theo cách đo này và mũ theo cách đo kia.
- **trường hợp xấu nhất / trung bình** — chi phí lớn nhất trên mọi dữ liệu vào cỡ n , so với chi phí trung bình theo một phân phối giả định. Nói “trung bình” mà không nói phân phối là nói vô nghĩa.
- **hệ thức truy hồi (recurrence)** — phương trình kiểu $T(n) = 2T(n/2) + n$, mô tả chi phí của thuật toán đệ quy theo chi phí của chính nó trên dữ liệu nhỏ hơn.
- **khấu hao (amortized)** — chi phí trung bình mỗi thao tác trong một *dãy* thao tác xấu nhất. Khác với trung bình xác suất: ở đây không có ngẫu nhiên nào cả.
- **phân cấp bộ nhớ** — thanh ghi, L1, L2, L3, RAM, SSD, đĩa — mỗi tầng chậm hơn tầng trên hàng chục tới hàng nghìn lần. Mô hình RAM giả vờ chúng như nhau, và đó là chỗ nó sai nhiều nhất.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao ta phân tích thuật toán trên một mô hình máy mà không máy thật nào giống, và vì sao điều đó vẫn hữu ích?
- Q2. Hai thuật toán cùng $O(n \log n)$. Trong thực tế một cái nhanh gấp năm lần cái kia. Ký hiệu tiệm cận đã che mất cái gì?
- Q3. Vì sao mảng động tăng gấp đôi kích thước lại cho chi phí thêm phần tử là $O(1)$ khấu hao, trong khi có những lần thêm tốn tới $O(n)$?
- Q4. Đề cho $n \leq 2 \cdot 10^5$ và thời gian 1 giây. Vì sao từ hai con số đó bạn suy ra được gần như toàn bộ dạng lời giải cần tìm?
- Q5. Khi nào một thuật toán $O(n^2)$ là lựa chọn *đúng* thay vì $O(n \log n)$?

Trước khi gõ dòng code đầu tiên, có một câu hỏi cần trả lời và nó chỉ tốn ba mươi giây: *ý tưởng này chạy trong bao lâu với dữ liệu lớn nhất mà đề cho phép?* Người mới bỏ qua câu hỏi này và trả giá bằng cách cài xong một lời giải $O(n^2)$ rồi mới phát hiện n lên tới một triệu. Người có kinh nghiệm trả lời nó trước, và nhờ vậy loại được ba phần tư số ý tưởng trong đầu mà không mất một phút gõ phím nào.

Chương này dựng công cụ cho câu hỏi đó. Nhưng cũng cần nói ngay giới hạn của nó: độ phức tạp tiệm cận là một mô hình, và mô hình nào cũng có chỗ nói dối. Nửa cuối chương dành cho những chỗ nói dối ấy — hằng số, bộ nhớ, cache — vì trong công việc thật, chúng thường quyết định nhiều hơn cả bậc tiệm cận.

3.1 Mô hình máy: giả định mà mọi phân tích đều ngầm dùng

Khi ta nói “thuật toán này chạy $O(n \log n)$ ”, ta đang nói về một cái máy tưởng tượng gọi là *mô hình RAM*: bộ nhớ là một dãy ô đánh số, truy cập ô nào cũng mất một đơn vị thời gian, và mỗi phép toán cơ bản trên một số cũng mất một đơn vị.

Mô hình này sai ở ít nhất ba chỗ. Truy cập bộ nhớ *không* đồng giá — đọc một ô đang nằm trong cache nhanh hơn đọc một ô phải lấy từ RAM cỡ hai bậc. Phép toán *không* đồng giá — phép chia chậm hơn phép cộng nhiều lần, và phép nhân số lớn thì chi phí phụ thuộc số bit. Và số nguyên trong máy thật có kích thước hữu hạn, nên “một phép cộng” chỉ đúng khi số vừa trong một từ máy.

Vậy vì sao vẫn dùng? Vì mô hình này giữ được thứ ta cần và bỏ đi thứ ta không kiểm soát được. Nó cho phép so sánh hai thuật toán mà không phụ thuộc ngôn ngữ, trình biên dịch hay đời máy, và nó dự đoán đúng điều quan trọng nhất: *khi dữ liệu lớn gấp đôi thì thời gian tăng bao nhiêu lần*. Với dữ liệu đủ lớn, bậc tiệm cận luôn thắng hằng số. Sai lầm không nằm ở việc dùng mô hình, mà ở việc quên rằng mình đang dùng mô hình — mục 6 sẽ nói về đúng những lúc phải nhớ ra.

CẠM BẤY

Một chỗ mà mô hình RAM đánh lừa thường xuyên trong lập trình thi đấu: coi số nguyên lớn tùy ý là “một đơn vị”. Nếu bài toán làm việc với số có hàng nghìn chữ số, mỗi phép cộng đã tốn $O(\text{số chữ số})$, và độ phức tạp thật khác hẳn con số bạn vừa tính. Tương tự với chuỗi: so sánh hai chuỗi *không* phải $O(1)$ mà là $O(\text{độ dài})$ — đây là lý do một lời giải dùng `map<string,int>` có thể chậm hơn dự đoán hẳn một bậc.

3.2 Ký hiệu tiệm cận, nói cho đúng

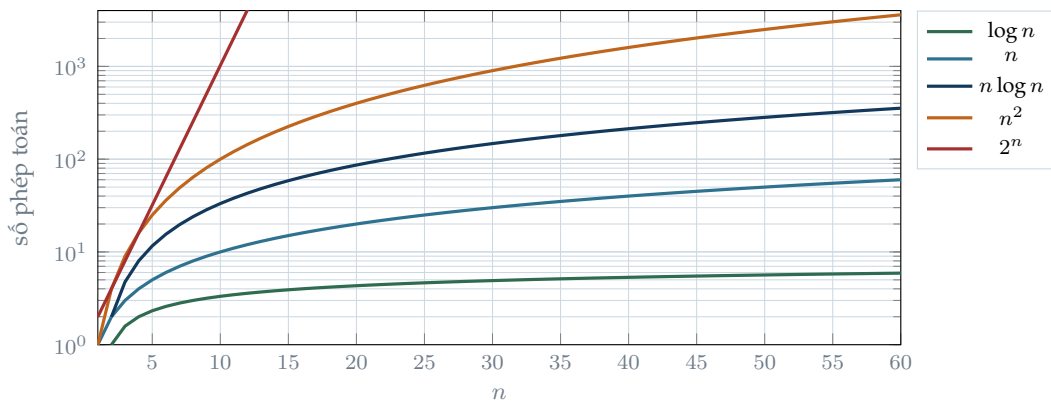
Ba ký hiệu cần phân biệt, và cách nhớ đơn giản nhất là coi chúng như dấu so sánh cho hàm số khi n lớn.

$O(f(n))$ là chặn trên: thuật toán chạy *không chậm hơn* $c \cdot f(n)$ với n đủ lớn. $\Omega(f(n))$ là chặn dưới: *không nhanh hơn*. $\Theta(f(n))$ là cả hai: đúng bậc đó. Trong thực tế người ta nói “thuật toán này $O(n \log n)$ ” và ý là $\Theta(n \log n)$; sự lỏng lẻo này vô hại, trừ khi bạn đang nói về *cận dưới của một bài toán* — lúc đó phải chính xác, vì “mọi thuật toán so sánh cần $\Omega(n \log n)$ ” là một phát biểu hoàn toàn khác với “thuật toán này chạy $O(n \log n)$ ”. Chương D/24 bàn kỹ.

Bậc tăng của các hàm hay gặp, xếp từ chậm tới nhanh:

$$1 \prec \log^* n \prec \log \log n \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n.$$

Điều đáng nhớ hơn danh sách này là *khoảng cách* giữa các bậc, và hình 3.1 cho thấy nó bằng hình.



Hình 3.1. Các bậc tăng trên trục dọc lôgarit. Hai điều đáng nhìn kỹ: n và $n \log n$ gần như song song — khoảng cách giữa chúng nhỏ tới mức trong thực tế hiếm khi đáng đánh đổi tính đơn giản để bỏ thừa số log; còn 2^n cất lên phía trên mọi đường khác trước khi n kịp đạt 20. Đó là toàn bộ lý do vì sao giới hạn $n \leq 20$ trên đề bài lại là một gợi ý mạnh.

Ba quy tắc rút gọn dùng hằng ngày: bỏ hằng số nhân ($3n^2 \rightarrow n^2$); chỉ giữ số hạng lớn nhất ($n^2 + n \log n \rightarrow n^2$); và với vòng lặp lồng nhau thì nhân, với các đoạn nối tiếp thì lấy cái lớn nhất. Riêng cơ sở của lôgarit thì bỏ được, vì đối cơ sở chỉ là nhân với hằng số.

3.3 Từ giới hạn đề bài suy ra độ phức tạp cho phép

Đây là mục thực dụng nhất chương. Một máy chấm hiện đại thực hiện cỡ 10^8 tới 10^9 phép toán đơn giản mỗi giây — con số thô, phụ thuộc ngôn ngữ và loại phép toán, nhưng đủ tốt để ước lượng. Lấy 10^8 làm mốc an toàn cho giới hạn một giây, ta có bảng sau. Bảng này nên thuộc lòng; nó là thứ biến dòng “ $n \leq \dots$ ” trên đề thành một gợi ý về dạng lời giải.

Giới hạn n	Độ phức tạp cho phép	Dạng lời giải thường gặp
$n \leq 11$	$O(n!)$	duyệt mọi hoán vị
$n \leq 20\text{--}25$	$O(2^n \cdot n)$	duyệt tập con, quy hoạch động bitmask, gặp nhau ở giữa
$n \leq 100$	$O(n^4)$	bốn vòng lặp, quy hoạch động bốn chiều
$n \leq 500$	$O(n^3)$	Floyd–Warshall, nhân ma trận, quy hoạch động khoảng
$n \leq 5000$	$O(n^2)$	quy hoạch động hai chiều, mọi cặp
$n \leq 10^5\text{--}10^6$	$O(n \log n)$	sắp xếp, cây chỉ số, chia để trị, tập hợp
$n \leq 10^7\text{--}10^8$	$O(n)$	một lượt quét, hai con trỏ, sàng
$n \leq 10^{18}$	$O(\log n)$ hoặc $O(1)$	nhị phân, lũy thừa nhanh, công thức đóng

Cách dùng bảng này ngược với cách người ta hay nghĩ. Bạn không dùng nó để kiểm tra lời giải *sau khi* nghĩ ra, mà để *định hướng* việc nghĩ. Thấy $n \leq 20$ thì đừng phí thời gian tìm lời giải đa thức tinh vi — người ra đề đã nói với bạn rằng lời giải là mũ. Thấy $n \leq 5000$ mà bạn đang cố nghĩ ra $O(n \log n)$ thì bạn đang làm khó mình vô ích; $O(n^2)$ được chấp nhận và thường đơn giản hơn nhiều.

Ba lưu ý làm bảng chính xác hơn. Thứ nhất, để ý *tổng* n qua các bộ test: đề ghi “tổng n không vượt quá $2 \cdot 10^5$ ” nghĩa là bạn được phép làm tuyến tính trên mỗi test mà không lo số test lớn. Thứ hai, hằng số của thao tác quan trọng: 10^8 phép cộng số nguyên thì nhanh, nhưng 10^8 lần chèn vào map thì chắc chắn quá giờ, vì mỗi thao tác đó tốn hàng chục tới hàng trăm lần hơn. Thứ

ba, bộ nhớ cũng có giới hạn: một mảng long long kích thước 10^8 cần 800 MB, vượt xa mức cho phép thông thường — nên đôi khi ràng buộc bộ nhớ mới là ràng buộc quyết định.

CẦN NHỚ

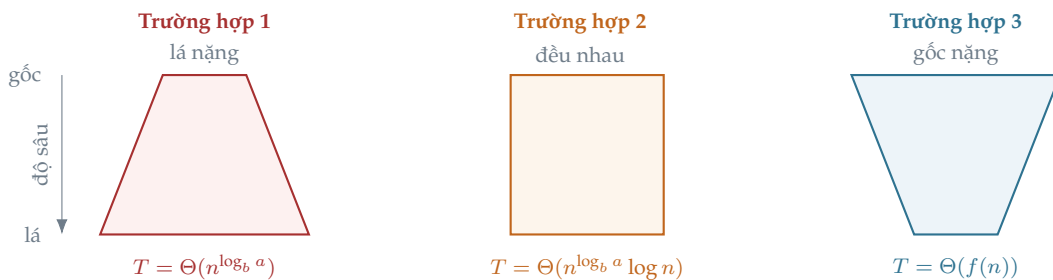
Đọc giới hạn *trước* khi nghĩ, không phải *sau*. Hai con số — n lớn nhất và giới hạn thời gian — thu hẹp không gian ý tưởng xuống còn một hoặc hai dạng. Đây là kỹ thuật rẻ nhất trong cả quyển sách này: tốn ba mươi giây, tiết kiệm hàng giờ.

3.4 Hệ thức truy hồi: chi phí của thuật toán đệ quy

Khi thuật toán gọi lại chính nó trên dữ liệu nhỏ hơn, chi phí được mô tả bằng hệ thức truy hồi. Có ba cách giải, xếp theo thứ tự nên thử.

Cách thứ nhất — vẽ cây đệ quy. Đây là cách trực giác nhất và hầu như luôn đủ. Vẽ cây các lời gọi, tính tổng chi phí ở mỗi tầng, rồi cộng các tầng lại. Với $T(n) = 2T(n/2) + n$: tầng gốc tốn n ; tầng hai có hai nút, mỗi nút tốn $n/2$, tổng vẫn là n ; mọi tầng đều tốn n ; cây sâu $\log_2 n$ tầng; vậy $T(n) = \Theta(n \log n)$. Toàn bộ merge sort nằm trong ba câu đó.

Cách thứ hai — định lý thợ (master theorem). Với dạng $T(n) = aT(n/b) + f(n)$, so sánh $f(n)$ với $n^{\log_b a}$: nếu f nhỏ hơn hẳn thì lá chiếm ưu thế và $T = \Theta(n^{\log_b a})$; nếu bằng nhau thì $T = \Theta(n^{\log_b a} \log n)$; nếu f lớn hơn hẳn (và thoả điều kiện đều đặn) thì gốc chiếm ưu thế và $T = \Theta(f(n))$. Hình 3.2 minh hoạ ba trường hợp bằng hình dáng cây.



Hình 3.2. Ba trường hợp của định lý thợ, nhìn theo “tầng nào tốn nhiều nhất”. Bề rộng mỗi tầng biểu diễn tổng chi phí của tầng đó. Nếu chi phí dồn về lá thì tổng do lá quyết định; nếu mọi tầng bằng nhau thì tổng bằng chi phí một tầng nhân số tầng; nếu dồn về gốc thì tổng do gốc quyết định. Nhớ hình này thì không cần nhớ công thức.

Cách thứ ba — đoán và chứng minh bằng quy nạp. Dùng khi hai cách trên không áp được, ví dụ với truy hồi không đều như $T(n) = T(n/5) + T(7n/10) + n$ của thuật toán chọn trung vị [34]. Đoán $T(n) \leq cn$, thay vào, và tìm c làm bất đẳng thức đúng: $cn/5 + 7cn/10 + n = 9cn/10 + n \leq cn$ khi $c \geq 10$. Vậy $T(n) = O(n)$, và cái đẹp của lời giải là ở chỗ tổng hai phân số nhỏ hơn 1.

Ba truy hồi đáng thuộc: $T(n) = T(n/2) + O(1)$ cho $\Theta(\log n)$ (tìm kiếm nhị phân); $T(n) = 2T(n/2) + O(n)$ cho $\Theta(n \log n)$ (merge sort); $T(n) = T(n-1) + O(n)$ cho $\Theta(n^2)$ (quicksort trường hợp xấu nhất).

3.5 Phân tích khấu hao: khi một thao tác đắt trả cho nhiều thao tác rẻ

Xét mảng động: khi đầy thì cấp phát mảng mới gấp đôi và sao chép toàn bộ. Một lần thêm phần tử có thể tốn $O(n)$. Nhưng nói “thêm phần tử là $O(n)$ ” thì sai lệch nghiêm trọng, vì lần đắt đó

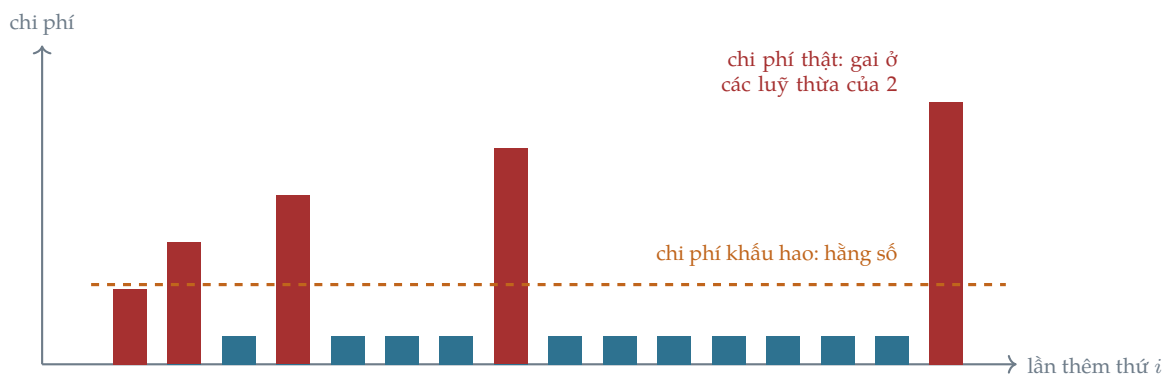
rất hiếm. Cách nói đúng: *chi phí khấu hao* của mỗi lần thêm là $O(1)$ — nghĩa là m lần thêm liên tiếp tốn tổng cộng $O(m)$, dù có vài lần lẻ tẻ tốn nhiều.

Chú ý một điểm bị nhầm rất nhiều: khấu hao *không* phải trung bình xác suất. Không có ngẫu nhiên nào ở đây; phát biểu đúng cho mọi dãy thao tác, kể cả dãy tệ nhất. Đây là công cụ Tarjan phát triển thành hệ thống[21], và nó có ba cách dùng.

Phương pháp tổng gộp: tính tổng chi phí của cả dãy rồi chia cho số thao tác. Với mảng động, tổng chi phí sao chép khi lớn tới n là $1 + 2 + 4 + \dots + n < 2n$, nên trung bình mỗi lần thêm là $O(1)$. Chuỗi hình học ấy là toàn bộ lý do vì sao phải nhân đôi chứ không phải cộng thêm một hằng số — nếu mỗi lần chỉ nới thêm 10 ô, tổng chi phí sao chép thành $\Theta(n^2)$ và chi phí khấu hao thành $\Theta(n)$.

Phương pháp kế toán: gán cho mỗi thao tác một “giá vé” cố định cao hơn chi phí thật, phần dư gửi vào tài khoản, và những thao tác đắt rút từ tài khoản ra. Với mảng động: mỗi lần thêm trả 3 đơn vị — 1 cho việc ghi, 2 gửi tiết kiệm để sau này trả cho việc sao chép chính nó và một phần tử cũ. Nếu chứng minh được tài khoản không bao giờ âm thì giá vé chính là chi phí khấu hao.

Phương pháp thế năng: tổng quát hơn, gán cho cấu trúc một hàm thế năng Φ đo “độ lộn xộn tích lũy”; chi phí khấu hao của một thao tác bằng chi phí thật cộng độ tăng của Φ . Thao tác đắt được phép vì nó làm Φ giảm mạnh. Đây là cách dùng cho những cấu trúc tinh vi như splay tree[35] và Fibonacci heap[36].



Hình 3.3. Chi phí thật của thao tác thêm vào mảng động (cột) so với chi phí khấu hao (đường ngang). Các gai xuất hiện đúng ở lũy thừa của 2 và cao gấp đôi lần trước, nhưng cũng thừa gấp đôi — hai hiệu ứng triệt tiêu nhau, để lại tổng tuyến tính. Đây là hình đáng nhớ nhất về khấu hao: *gai cao hơn thì phải thưa hơn tương ứng*.

CẠM BÃY

Khấu hao chỉ hợp lệ khi bạn quan tâm tới *tổng* thời gian. Trong hệ thống thời gian thực — điều khiển robot, xử lý âm thanh — một thao tác đột ngột tốn $O(n)$ có thể làm hỏng hạn kỳ, dù chi phí khấu hao rất đẹp. Ở đó người ta chọn cấu trúc có đảm bảo *trường hợp xấu nhất* cho từng thao tác, chấp nhận hằng số lớn hơn. Chương E/26 nói kỹ tình huống này.

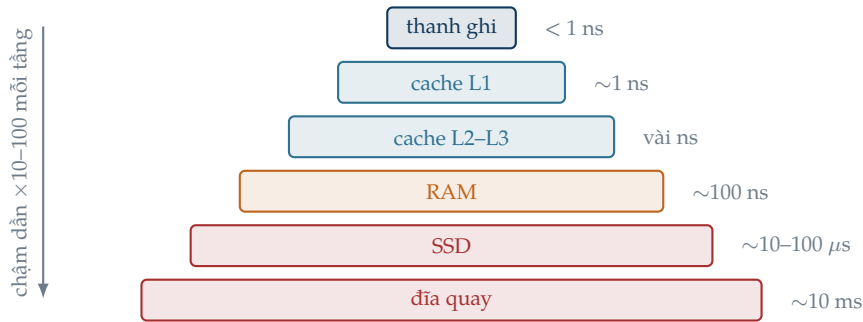
3.6 Chỗ mô hình nói dối: hằng số, bộ nhớ và cache

Đến đây ta có công cụ tiệm cận. Bây giờ là phần mà giáo trình hay nói lướt còn thực tế thì phạt rất nặng nếu bỏ qua.

Hằng số không nhỏ như ký hiệu gợi ý. Sắp xếp $O(n \log n)$ bằng `std::sort` nhanh hơn hẳn một segment tree $O(n \log n)$ tự viết, vì hằng số khác nhau cả chục lần. Ở kích thước nhỏ, thuật toán

“kém” thường thắng: đó là lý do các thư viện sắp xếp thật đều chuyển sang sắp xếp chèn khi đoạn còn dưới vài chục phần tử[14], [37].

Truy cập bộ nhớ mới là chi phí thật. Trên máy hiện đại, đọc một ô trong cache L1 nhanh hơn đọc từ RAM khoảng hai bậc độ lớn. Hệ quả rất cụ thể: duyệt một mảng liên tục nhanh hơn nhiều lần so với duyệt một danh sách liên kết cùng số phần tử, dù cả hai đều $O(n)$. Vì vậy mảng phẳng thường thắng cấu trúc con trỏ ở kích thước thật, và một segment tree cài bằng mảng nhanh hơn hẳn cài bằng con trỏ.



Hình 3.4. Phân cấp bộ nhớ với độ trễ theo bậc độ lớn (con số chỉ mang tính bậc, khác nhau theo máy). Mô hình RAM coi mọi tầng như nhau. Trong thực tế, một thuật toán “chậm hơn” về tiệm cận nhưng truy cập tuần tự có thể thắng một thuật toán “nhanh hơn” nhưng nhảy ngẫu nhiên khắp bộ nhớ — chương D/24 và E/26 khai thác đúng hiện tượng này.

Bộ nhớ cũng là một ngân sách. Một mảng hai chiều $10^4 \times 10^4$ kiểu int cần 400 MB — thường là vượt giới hạn. Khi gặp tình huống đó, hai lối thoát quen thuộc: giảm chiều của trạng thái quy hoạch động bằng cách chỉ giữ lại hàng trước (chương C/16), hoặc đổi sang cấu trúc thưa nếu dữ liệu thưa.

Và cuối cùng: đo, đừng đoán. Trong công việc thật, khoảng cách giữa dự đoán và thực tế đủ lớn để đáng bỏ hai phút chạy thử. Bài học của McSherry và cộng sự[38] rất đáng nhớ: nhiều hệ thống song song “có khả năng mở rộng” bị một chương trình đơn luồng viết cẩn thận đánh bại — vì chi phí điều phối lớn hơn phần được tăng tốc. Bậc tiệm cận không nói gì về những chi phí đó.

3.7 Đo bậc tăng trưởng thay vì đoán: phép thử nhân đôi

Cho tới đây ta luôn đi một chiều: từ mã nguồn suy ra chi phí. Chiều ngược lại — chạy chương trình rồi *đọc ngược* ra bậc tăng trưởng — rẻ hơn nhiều so với vẽ ngoài của nó và gần như không ai làm. Sedgewick và Wayne gọi kỹ thuật này là *phép thử tỉ lệ nhân đôi* (doubling ratio test), và khuyên chạy nó cho mọi chương trình mà hiệu năng có ý nghĩa[22].

Cách làm gồm ba bước. Một, viết một bộ sinh dữ liệu vào giống dữ liệu bạn thật sự gặp. Hai, chạy chương trình với n , rồi $2n$, rồi $4n$, ghi lại thời gian mỗi lần. Ba, lấy tỉ số giữa thời gian của hai lần liên tiếp. Nếu tỉ số ấy tiến tới một hằng số r , thì thời gian chạy xấp xỉ $a n^b$ với $b = \log_2 r$ — bậc của thuật toán hiện ra từ số đo, không cần đọc lại một dòng mã nào.

Vì sao tỉ số lại hội tụ? Chỉ cần một dòng tính. Nếu $T(n) \sim a n^b \lg n$ thì

$$\frac{T(2n)}{T(n)} = \frac{a (2n)^b \lg(2n)}{a n^b \lg n} = 2^b \left(1 + \frac{\lg 2}{\lg n}\right) \sim 2^b.$$

Nói cách khác, thừa số lôgarit — thứ luôn gây phiền khi phân tích bằng tay — gần như biến mất khỏi *tỉ số*. Đó là lý do phép đo này ổn định hơn ta tưởng, và cũng chính là lý do nó *không* phân biệt được $O(n)$ với $O(n \log n)$: cả hai đều cho tỉ số tiến về 2.

Ví dụ trong chính quyển sách của Sedgewick và Wayne đáng nhớ vì nó cho thấy sức mạnh dự đoán. Chương trình ThreeSum có ba vòng lặp lồng nhau; tỉ số đo được hội tụ về 8, tức $b = 3$, đúng bằng n^3 . Từ số đo 51,1 giây tại $n = 8000$, họ dự đoán 408,8 giây cho $n = 16\,000$ và 26 163 giây cho $n = 64\,000$ — chỉ bằng cách nhân liên tiếp với 8, không cần chạy thật.

Phép thử này trả lời ba câu hỏi mà lập luận trên giấy hay trả lời sai.

Chương trình có đúng như tôi nghĩ không? Nếu bạn tin mình viết $O(n \log n)$ mà tỉ số đo được là 4, thì ở đâu đó có một vòng lặp ẩn — thường là một phép sao chép chuỗi trong vòng lặp, một lần chèn vào giữa mảng, hoặc một lần tìm tuyến tính trong danh sách. Đây là cách rẻ nhất để bắt *lỗi hiệu năng*: loại lỗi không làm sai kết quả nên không phép kiểm tính đúng nào phát hiện được (E/25).

Chạy tới kích thước thật thì mất bao lâu? Nhân liên tiếp tỉ số cho một dự đoán đủ tốt, và nó biến câu hỏi “có kịp không” ở mục 3 từ ước lượng trong đầu thành một con số có cơ sở.

Ngưỡng hoà vốn nằm ở đâu? Khi hai lời giải khác bậc nhưng cũng khác hằng số, hai đường thời gian cắt nhau tại một điểm, và chỉ có đo mới tìm được điểm đó. Đây đúng là con số mà mọi thư viện sắp xếp phải chọn khi quyết định lúc nào chuyển sang sắp xếp chèn (C/14, E/26).

Ba điều kiện phải nhớ kèm, nếu không phép đo sẽ nói dối. Thứ nhất, tỉ số chỉ hội tụ với các bậc dạng lũy thừa; với thuật toán mũ nó tăng mãi — và chính việc *không* hội tụ đã là một câu trả lời. Thứ hai, bậc đo được là bậc *trên phân phối dữ liệu bạn sinh ra*: một bảng băm cho tỉ số 2 rất đẹp trên dữ liệu ngẫu nhiên vẫn có thể tệ đi n lần trên dữ liệu do đối thủ dựng (B/08). Thứ ba, ở kích thước nhỏ, thời gian bị chi phối bởi khởi động, cache lạnh và nhiễu đo; hãy bỏ vài điểm đầu và chỉ tin phần đuôi của bảng.

3.8 Gốc rễ: ký hiệu này mượn từ đâu và vì sao lại cần

Ký hiệu $O(\cdot)$ không sinh ra trong ngành máy tính. Nó xuất hiện cuối thế kỷ 19 trong lý thuyết số giải tích — Paul Bachmann dùng nó năm 1894, Edmund Landau phổ biến nó — để nói “sai số của xấp xỉ này không lớn hơn cỡ này” khi nghiên cứu phân bố số nguyên tố. Ở đó, biến chạy ra vô cùng là một đại lượng toán học thuần túy, và mối quan tâm là đáng điệu tiệm cận của hàm.

Ngành máy tính mượn ký hiệu ấy vì gặp đúng nhu cầu đó dưới hình thức khác: muốn so sánh hai thuật toán mà không phải nói “trên máy IBM 7090 với trình biên dịch này thì...”. Knuth là người chuẩn hoá cách dùng cho thuật toán và cũng là người kéo $\Omega(\cdot)$ và $\Theta(\cdot)$ vào để phân biệt chặn trên với chặn dưới — một phân biệt mà giới toán trước đó không cần tới, nhưng ngành thuật toán thì rất cần, vì ở đây ta phải nói được cả “thuật toán của tôi nhanh cỡ này” lẫn “mọi thuật toán đều không thể nhanh hơn cỡ này”.

Điều lý thú là mô hình máy cũng có gốc rễ tương tự: nó ra đời vì một mô hình cũ không dùng được cho mục đích mới. Máy Turing (1936) là mô hình chuẩn của lý thuyết tính toán, nhưng nó tính chi phí theo số bước trên băng, nên ngay cả việc đọc phần tử thứ i của mảng cũng tốn thời gian tỉ lệ i . Với câu hỏi “bài toán này có giải được không” thì chi tiết đó không quan trọng; với câu hỏi “thuật toán nào nhanh hơn” thì nó bóp méo mọi thứ. Mô hình RAM ra đời trong thập niên 1970 chính để lấp chỗ đó: cho phép truy cập ngẫu nhiên với giá một đơn vị, gần với máy thật hơn nhiều.

Còn từ “khấu hao” thì mượn thẳng từ kế toán, và nghĩa gốc soi sáng nghĩa kỹ thuật rất rõ. Trong kế toán, khi mua một cái máy đắt tiền dùng được mười năm, người ta không ghi toàn bộ chi phí vào tháng mua mà *phân bổ* nó ra suốt vòng đời. Tarjan mượn đúng ý đó: lần cấp phát lại tốn kém của mảng động không nên bị tính hết cho một thao tác thêm, mà nên phân bổ cho tất cả các thao tác đã hưởng lợi từ nó[21]. Khi đã thấy nghĩa gốc, phương pháp kế toán ở mục 5 không còn là mẹo mà là chính bút toán trả trước.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1894 — Bachmann, Landau	Cần nói gọn về sai số xấp xỉ trong lý thuyết số giải tích.	Ra đời ký hiệu O ; ban đầu không liên quan gì tới thuật toán.
1936 — Turing	Định nghĩa chính xác “tính được”.	Máy Turing: hoàn hảo cho câu hỏi <i>giải được hay không</i> , nhưng bóp méo chi phí truy cập nên không hợp để so tốc độ.
Thập niên 1960–70 — mô hình RAM	Cần một mô hình chi phí gần máy thật để phân tích và so sánh thuật toán.	Truy cập ngẫu nhiên giá một đơn vị; trở thành mô hình ngầm định của mọi ký hiệu $O(\cdot)$ trong sách vở.
1976 — Knuth	Trong thuật toán cần phân biệt “nhanh cỡ này” với “không thể nhanh hơn cỡ này”.	Chuẩn hoá bộ ba O, Ω, Θ ; mở đường cho lý thuyết cận dưới (Ω).
1985 — Tarjan	Các cấu trúc dữ liệu mới (splay tree, Fibonacci heap) có thao tác lẻ tẻ rất đắt nhưng tổng thể rất rẻ.	Hệ thống hoá phân tích khấu hao, mượn khái niệm phân bổ chi phí của kế toán.
1999 — Frigo và cộng sự	Khoảng cách tốc độ giữa CPU và bộ nhớ ngày càng lớn, mô hình RAM ngày càng nói dối.	Mô hình cache-oblivious: tính chi phí theo số lần chuyển khối bộ nhớ thay vì số phép toán (Ω).

I 3.9 Liên hệ ngang: so sánh với các cách đo chi phí khác

Mô hình RAM chỉ là một trong nhiều mô hình chi phí, và biết các mô hình khác giúp bạn nhận ra lúc nào mô hình quen thuộc đang nói dối. Bảng dưới đặt chúng cạnh nhau theo cùng một tiêu chí: đơn vị đếm, giả định chính, và chỗ hỏng.

Mô hình	Đếm cái gì	Hợp khi nào, hỏng khi nào
Máy Turing	Bước trên băng	Hợp cho câu hỏi giải được / độ khó lớp bài toán; hỏng khi so tốc độ vì truy cập không phải $O(1)$
RAM (mặc định)	Phép toán và truy cập	Hợp cho hầu hết phân tích; hỏng khi số rất lớn, khi chuỗi dài, khi cache quyết định
Độ phức tạp bit	Phép toán trên từng bit	Bắt buộc với số học số lớn, mật mã (Ω); nặng nề khi không cần
Bộ nhớ ngoài	Số lần đọc/ghi khối đĩa	Hợp với cơ sở dữ liệu, dữ liệu lớn hơn RAM; thừa khi mọi thứ nằm gọn trong bộ nhớ
Cache-oblivious	Số lần chuyển khối, không cần biết kích thước khối	Giải thích vì sao mảng thẳng con trở; khó phân tích hơn (Ω)
Song song (PRAM)	Số bước với p bộ xử lý	Hợp để tìm giới hạn song song hoá; bỏ qua chi phí đồng bộ và truyền tin, vốn thường mới là nút cổ chai
Luồng dữ liệu	Bộ nhớ dùng, số lượt quét	Hợp khi dữ liệu chảy qua một lần và không lưu được (Ω)
Năng lượng	Joule mỗi thao tác	Quan trọng trên thiết bị nhúng và robot; gần như vắng mặt trong sách giáo khoa thuật toán

Ngoài các mô hình tính toán, ý tưởng “chỉ quan tâm tốc độ tăng, bỏ hằng số” còn là một công cụ tư duy dùng khắp nơi ngoài ngành máy tính — và biết điều đó làm cho việc phân tích tiệm cận bớt giống một nghi thức học thuật.

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Vật lý	Phân tích thứ nguyên và luật tỉ lệ: bỏ hằng số, giữ số mũ	Cùng một động tác tư duy: muốn biết cái gì chi phối khi hệ lớn lên thì bỏ chi tiết và giữ số mũ. Khác: trong vật lý số mũ đến từ định luật bảo toàn; trong thuật toán nó đến từ cấu trúc đệ quy của lời giải.
Sinh học	Luật tỉ lệ trao đổi chất theo khối lượng cơ thể	Cùng dạng phát biểu “đại lượng này tăng theo lũy thừa của kích thước”. Bài học chung: khi hệ lớn gấp mười, thứ chi phối thường đổi hẳn — đúng như $O(n^2)$ thua $O(n \log n)$ ở quy mô lớn.
Kế toán, tài chính	Khấu hao tài sản, trả góp	Không phải chỉ giống mà là nguồn gốc trực tiếp của thuật ngữ: phân bổ một chi phí lớn cho nhiều kỳ hưởng lợi. Phương pháp kế toán ở mục 5 chính là bút toán trả trước.
Kinh tế	Chi phí biên, lợi thế quy mô	Chi phí khấu hao chính là chi phí biên trung bình của một thao tác. Cả hai đều nhắc: đừng đánh giá một hệ thống bằng chi phí của một sự kiện đơn lẻ.
Kỹ thuật hệ thống	Đo hiệu năng bằng hồ sơ chạy thật (profiling)	Bổ sung chứ không thay thế: tiệm cận nói đáng điều khi n lớn, profiling nói sự thật ở n hiện tại. Người chỉ dùng một trong hai đều sai — chi tiết ở E/26.
Ước lượng trạng thái, robot	Ngân sách tính toán theo chu kỳ thời gian thực	Ở đây cái quan trọng là <i>trường hợp xấu nhất mỗi khung hình</i> , không phải trung bình — nên cấu trúc có chi phí khấu hao đẹp có thể lại là lựa chọn sai. Xem cạm bẫy ở mục 5.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mô hình giữ lại thứ ta cần và bỏ đi thứ ta không kiểm soát. Điều ta cần là *tốc độ tăng khi dữ liệu lớn lên* — thứ quyết định lời giải có sống nổi ở giới hạn của đề hay không — và điều đó không phụ thuộc máy, ngôn ngữ hay trình biên dịch. Cái giá phải trả là mô hình mù với hằng số và với phân cấp bộ nhớ, nên nó chỉ đúng khi n đủ lớn; ở n nhỏ nó có thể sai hoàn toàn.
- A2. Ký hiệu tiệm cận cố ý bỏ hằng số nhân và các số hạng bậc thấp. Chênh lệch năm lần thường đến từ ba nguồn: hằng số của thao tác cơ bản (so sánh số nguyên rẻ hơn nhiều so với so sánh chuỗi hoặc phép chia), kiểu truy cập bộ nhớ (tuần tự so với nhảy ngẫu nhiên), và chi phí cấp phát. Với dữ liệu đủ lớn thì bậc luôn thắng; nhưng “đủ lớn” đôi khi lớn hơn dữ liệu thật của bạn.
- A3. Vì các lần đắt vừa cao hơn vừa thưa hơn theo đúng cùng một tỉ lệ, và hai hiệu ứng triệt tiêu nhau. Chi phí sao chép khi mảng lớn tới n là $1 + 2 + 4 + \dots + n < 2n$, một cấp số nhân bị chặn bởi hai lần số hạng cuối. Chia cho n lần thêm ta được hằng số. Điều then chốt là nhân đôi: nếu chỉ nói thêm một lượng cố định thì tổng chi phí thành $\Theta(n^2)$ và khấu hao thành $\Theta(n)$.
- A4. Vì hai con số đó xác định số phép toán bạn được tiêu — cỡ 10^8 — và từ đó ra bậc cho phép, còn bậc cho phép thì gần như xác định họ kỹ thuật. Với $n \leq 2 \cdot 10^5$ và một giây, bậc cho phép là $O(n \log n)$: điều này loại mọi ý tưởng “xét mọi cặp”, và trở thẳng tới sắp xếp, hai con trỏ, cây chỉ số, chia để trị hoặc tập hợp có thứ tự. Bạn vừa cắt bỏ phần lớn không gian tìm kiếm ý tưởng chỉ bằng cách đọc một dòng của đề.
- A5. Ít nhất bốn tình huống. Khi n nhỏ và luôn nhỏ — hằng số nhỏ và code đơn giản thắng. Khi $O(n^2)$ truy cập tuần tự còn $O(n \log n)$ nhảy ngẫu nhiên — cache có thể lật ngược kết quả. Khi thuật toán phức tạp có nguy cơ sai — một lời giải đúng chậm hơn vẫn hơn một lời giải nhanh mà sai. Và khi thời gian của bạn là tài nguyên khan hiếm hơn thời gian máy, ví dụ trong kỳ thi còn 15 phút hoặc trong đoạn mã chạy mỗi tháng một lần.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới việc dọn một thư viện. Nếu có gấp đôi số sách thì bạn mất gấp đôi thời gian để xếp chúng lên kệ, nhưng chỉ mất thêm một chút thời gian để *tìm* một quyển nếu kệ đã sắp theo thứ tự — tìm kiếm nhị phân là thế. Còn nếu bạn phải so từng cặp sách với nhau thì số việc tăng gấp bốn

khi số sách tăng gấp đôi; đó là n^2 , và nó là lý do những việc kiểu “xét mọi cặp” hỏng rất nhanh khi dữ liệu lớn.

Ký hiệu $O(\cdot)$ chỉ là cách nói “khi mọi thứ lớn gấp đôi thì công việc tăng bao nhiêu lần”. Nó cố tình không quan tâm bạn xếp sách nhanh hay chậm, vì cái đó đổi theo người, còn tỉ lệ tăng thì không. Khấu hao là chuyện khác và cũng rất đời thường. Bạn không đi chợ mỗi lần cần một quả trứng; bạn đi một chuyến và mua cả vỉ. Chuyến đi đó tốn thời gian, nhưng chia cho số quả trứng thì rẻ. Mảng động cũng vậy: thỉnh thoảng nó dọn nhà một lần thật tốn, nhưng vì mỗi lần dọn nó tăng gấp đôi sức chứa nên các lần dọn thừa dần đúng bằng mức chúng đất lên.

Cuối cùng, cái bảng “ n tới đâu thì được phép chậm tới đâu” chỉ là câu hỏi: tôi có ngân sách khoảng một trăm triệu việc nhỏ, dữ liệu chừng này, vậy mỗi phần tử tôi được làm bao nhiêu việc?

Một câu: Độ phức tạp là để loại bỏ ý tưởng trước khi gõ code, không phải để trang trí lời giải sau khi xong.

Chỗ không rút gọn được: Ranh giới “ n đủ lớn để bậc tiệm cận thắng hằng số” không có công thức chung; chỉ đo trên máy thật mới biết.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Mô hình RAM giả định gì	Mỗi phép toán cơ bản và mỗi truy cập ô nhớ bất kỳ tốn một đơn vị thời gian. Sai ở ba chỗ: cache, chi phí phép toán khác nhau, và số nguyên có kích thước hữu hạn.
••• Bảng giới hạn $\rightarrow$ độ phức tạp	$n \leq 20$: 2^n ; $n \leq 500$: n^3 ; $n \leq 5000$: n^2 ; $n \leq 10^5$: $n \log n$; $n \leq 10^7$: n ; $n \leq 10^{18}$: $\log n$. Ngân sách khoảng 10^8 phép toán mỗi giây.
••• $O()$, $\Omega()$, $\Theta()$ khác nhau	Chặn trên, chặn dưới, và cả hai. Nói lỏng $O()$ thay cho $\Theta()$ thì vô hại, trừ khi đang phát biểu cận dưới của một bài toán.
••• Ba truy hồi phải thuộc	$T(n) = T(n/2) + O(1) \Rightarrow \Theta(\log n)$; $T(n) = 2T(n/2) + O(n) \Rightarrow \Theta(n \log n)$; $T(n) = T(n-1) + O(n) \Rightarrow \Theta(n^2)$.
••• Khấu hao khác trung bình xác suất thế nào	Khấu hao không có ngẫu nhiên: nó là trung bình trên một dãy thao tác xấu nhất, đúng cho mọi dãy. Trung bình xác suất cần một phân phối giả định.
••• Vì sao mảng động phải nhân đôi	Vì tổng chi phí sao chép $1 + 2 + 4 + \dots + n < 2n$ là cấp số nhân, cho khấu hao $O(1)$. Nếu nói thêm hằng số thì tổng thành $\Theta(n^2)$.
•• Ba trường hợp của định lý thợ	So $f(n)$ với $n^{\log_b a}$: lá nặng $\Rightarrow \Theta(n^{\log_b a})$; đều nhau $\Rightarrow$ nhân thêm $\log n$; gốc nặng $\Rightarrow \Theta(f(n))$.
•• Ba cách phân tích khấu hao	Tổng gộp (cộng cả dây rồi chia); kế toán (giá vé cố định, gửi phần dư vào tài khoản); thế năng (hàm Φ đo độ lộn xộn tích lũy).
•• Khi nào khấu hao không dùng được	Hệ thời gian thực có hạn kỳ cho từng thao tác; ở đó cần đảm bảo trường hợp xấu nhất, chấp nhận hằng số lớn hơn.
•• Bấy “một phép toán là $O(1)$ ”	Sai với số lớn tùy ý (chi phí theo số chữ số) và với chuỗi (so sánh tốn theo độ dài) — <code>map<string, int></code> chậm hơn dự đoán một bậc vì lý do này.
•• Vì sao mảng thẳng danh sách liên kết	Truy cập tuần tự thân thiện với cache; RAM chậm hơn L1 khoảng hai bậc. Cùng $O(n)$ nhưng khác nhau nhiều lần trên máy thật.
•• “Tổng n qua các test bị chặn” nghĩa là gì	Bạn được phép chạy tuyến tính trên mỗi test mà không lo số test lớn — đây là gợi ý rất mạnh về dạng lời giải.
• Bài học COST[38]	Nhiều hệ thống song song bị một chương trình đơn luồng viết tốt đánh bại, vì chi phí điều phối không xuất hiện trong phân tích tiệm cận.
•• Phép thử tỉ lệ nhân đôi	Chạy với $n, 2n, 4n, \dots$; nếu tỉ số thời gian tiến tới r thì bậc là $n^{\log_2 r}$. Không phân biệt được $O(n)$ với $O(n \log n)$, và chỉ đúng cho phân phối dữ liệu bạn tự sinh.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao nói “thuật toán này có độ phức tạp trung bình $O(n \log n)$ ” mà không nói gì thêm là chưa đủ?

| Vì trung bình luôn phải kèm phân phối. Quicksort có trung bình $\Theta(n \log n)$ với giả định thứ tự đầu vào ngẫu nhiên đều; nếu dữ liệu đến đã sắp sẵn và bạn chọn chốt là phần tử đầu, nó tụt xuống $\Theta(n^2)$. Trong hệ thống thật, dữ liệu hiếm khi ngẫu nhiên và đôi khi do kẻ tấn công chọn. Cách sửa đúng là tự tạo ra ngẫu nhiên — chọn chốt ngẫu nhiên — để đảm bảo không phụ thuộc phân phối đầu vào; chương C/20 bàn kỹ.

2. Tôi có lời giải $O(n\sqrt{n})$ và một lời giải $O(n \log^2 n)$. Chọn cái nào?

| Tính thử ở n thật. Với $n = 10^5$: $n\sqrt{n} \approx 3 \cdot 10^7$, còn $n \log^2 n \approx 10^5 \cdot 17^2 \approx 3 \cdot 10^7$ — gần như bằng nhau, nên yếu tố quyết định là hằng số và độ khó cài đặt. Với $n = 10^6$ thì $n\sqrt{n} = 10^9$ đã quá lớn còn $n \log^2 n \approx 4 \cdot 10^8$ vẫn có cửa. Bài học chung: khi hai bậc gần nhau, đừng so ký hiệu — thay số thật vào.

3. Định lý thợ không áp được cho truy hồi của tôi. Làm sao?

| Vẽ cây đệ quy, đó là công cụ tổng quát hơn và gần như luôn cho câu trả lời. Tính chi phí mỗi tầng, xem chi phí tăng, giảm hay giữ nguyên theo tầng. Tăng theo cấp số nhân thì tổng do lá quyết định; giảm theo cấp số nhân thì do gốc quyết định; giữ nguyên thì nhân với số tầng. Nếu cây không đều (như truy hồi của thuật toán chọn trung vị), đoán nghiệm rồi chứng minh bằng quy nạp.

4. Vì sao log trong độ phức tạp không cần ghi cơ số?

| Vì đổi cơ số chỉ nhân với hằng số: $\log_2 n = \log_{10} n / \log_{10} 2$. Ký hiệu tiện cận bỏ hằng số nhân, nên $\Theta(\log_2 n) = \Theta(\log_{10} n)$. Nhưng khi ước lượng số phép toán thật thì cơ số rất quan trọng: $\log_2 10^6 \approx 20$ còn $\log_{10} 10^6 = 6$, lệch hơn ba lần.

5. “Độ phức tạp giả đa thức” nghĩa là gì và vì sao nó quan trọng?

| Là độ phức tạp đa thức theo giá trị của số trong dữ liệu vào, chứ không theo số bit biểu diễn chúng. Quy hoạch động cho bài xếp ba lô chạy $O(nW)$ với W là sức chứa — nhìn thì đa thức, nhưng W được ghi bằng $\log W$ bit, nên theo kích thước dữ liệu vào thật thì đó là mũ. Đây chính là lý do bài ba lô vẫn là NP-khó dù có lời giải quy hoạch động; chương D/23 nói kỹ.

6. Tôi nên tính độ phức tạp bộ nhớ thế nào cho nhanh?

| Đếm mảng lớn nhất rồi nhân với kích thước phần tử: `int` 4 byte, `long` 8 byte và `con trỏ` 8 byte. Nhớ rằng các cấu trúc có `con trỏ` tồn thêm nhiều: mỗi nút của `map` hoặc `set` thường tốn cỡ 48 byte cho một phần tử vài byte. Đó là lý do một `set<int>` với 10^7 phần tử là không khả thi trong khi mảng `int` cùng kích thước chỉ tốn 40 MB.

7. Nếu thuật toán của tôi vượt giới hạn thời gian, tôi nên tìm thuật toán tốt hơn hay tối ưu hằng số?

| Trả lời bằng khoảng cách. Nếu bạn chậm gấp 2–3 lần, tối ưu hằng số thường đủ: bỏ cấu trúc có `con trỏ`, đổi `map` sang mảng, tránh cấp phát trong vòng lặp, đọc dữ liệu nhanh hơn. Nếu chậm gấp 50 lần trở lên, không có tối ưu hằng số nào cứu được — bậc của bạn sai và phải đổi ý tưởng. Ước lượng bằng bảng ở mục 3 trước khi quyết định, để khỏi tối ưu vi mô một lời giải chết từ trong ý tưởng.

8. Phân tích trường hợp xấu nhất có bị quan quá không, khi dữ liệu thật hiếm khi tệ như vậy?

| Có và không. Trong lập trình thi đấu, dữ liệu được cố ý làm tệ nhất có thể, nên phân tích xấu nhất là phân tích duy nhất đáng tin. Trong hệ thống thật, phân tích xấu nhất đôi khi quá bị quan — nhưng nó lại là điều bạn quan tâm nhất khi có hạn kỳ hoặc khi kẻ tấn công có thể chọn dữ liệu (băm là ví dụ kinh điển, xem chương B/08). Cách dùng thực dụng: dùng trường hợp xấu nhất để đảm bảo an toàn, và dùng đo đạc thực nghiệm để tối ưu.

I Bài tập (mức: ● khởi động ● ● chuẩn ● ● ● khó)

Bài	Nguồn	Điều cần nhận ra
• Với mỗi giới hạn $n = 10, 20, 500, 5000, 10^5, 10^7$, viết ra bậc cho phép và một thuật toán mẫu	Tự luyện	Mục tiêu là thuộc bảng mục 3 tới mức phản xạ; kiểm tra lại sau một tuần.
• Tính độ phức tạp của ba đoạn code có vòng lặp lồng nhau mà bước nhảy không đều	Bài tập kinh điển	Vòng lặp <code>for(i=1; i<=n; i*=2)</code> cho $\log n$; vòng lặp trong chạy tới i cho tổng $n^2/2$.
• Giải ba truy hồi cơ bản bằng cây đệ quy, không dùng định lý thợ	[27]	Vẽ cây thật ra giấy; mục tiêu là thấy được “tầng nào nặng” chứ không phải ra đáp số.
•• Chứng minh chi phí khấu hao của mảng động bằng cả ba phương pháp	[21]	Ba phương pháp cho cùng kết quả; điều cần rút ra là mỗi phương pháp dễ dùng ở tình huống khác nhau.
•• Phân tích chi phí khấu hao của bộ đếm nhị phân tăng dần	Bài tập kinh điển	Mỗi lần tăng lật trung bình 2 bit; dùng phương pháp kế toán với “một đơn vị gửi kèm mỗi bit 1”.
•• Đo thời gian thật: sắp xếp 10^6 số bằng <code>std::sort</code> và bằng cây tìm kiếm	Thực nghiệm	Cùng $O(n \log n)$ nhưng khác nhau nhiều lần; giải thích khác biệt bằng cache và cấp phát.
•• Duyệt mảng 10^7 phần tử theo thứ tự và theo hoán vị ngẫu nhiên, đo hai thời gian	Thực nghiệm	Đây là cách rẻ nhất để thấy phân cấp bộ nhớ bằng số liệu của chính máy bạn.
••• Giải truy hồi $T(n) = T(n/5) + T(7n/10) + n$ bằng quy nạp	[34]	Đoán $T(n) \leq cn$ và tìm c ; mấu chốt là $1/5 + 7/10 < 1$.
••• Tìm một trường hợp n nhỏ mà thuật toán $O(n^2)$ nhanh hơn $O(n \log n)$ trên máy bạn	Thực nghiệm	Sắp xếp chèn so với sắp xếp trộn; tìm điểm giao và giải thích vì sao thư viện thật cũng chuyển ngưỡng ở quanh đó.
••• Ước lượng bộ nhớ của quy hoạch động hai chiều $10^4 \times 10^4$ rồi tìm cách giảm chiều	Chương C/16	Chỉ giữ hàng trước; đây là kỹ thuật bạn sẽ dùng lại rất nhiều.
••• Đọc lại một lời giải cũ của bạn bị quá thời gian và phân loại: sai bậc hay sai hằng số	Nhật ký của bạn	Quy tắc ở mục hỏi đáp: chậm 2–3 lần thì sửa hằng số, chậm 50 lần thì phải đổi ý tưởng.

Kết chương và đọc tiếp

Chương này cho cái thước thứ nhất. Ba thứ đáng mang theo: bảng chuyển từ giới hạn đề bài sang độ phức tạp cho phép, hình ảnh “tầng nào nặng” để giải truy hồi mà không cần nhớ công thức, và ý niệm khấu hao — gai cao thì phải thưa tương ứng. Cùng với đó là một sự dè dặt lành mạnh: tiệm cận là mô hình, và ở kích thước thật thì hằng số cùng cache có tiếng nói riêng.

Nhưng nhanh chưa đủ. Một lời giải $O(n \log n)$ sai thì tệ hơn một lời giải $O(n^2)$ đúng. Chương A/04 cấp cái thước thứ hai — công cụ để biết lời giải *đúng* — và bạn sẽ thấy công cụ ấy, bất biến, không chỉ dùng để chứng minh mà còn dùng để *nghĩ ra* thuật toán.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Sedgewick & Wayne, <i>Algorithms</i> (4th ed.), chương 1.4	Addison-Wesley, 2011	Nguồn của phép thử nhân đôi và của cách viết $\sim$ cho hằng số. Đọc mục “Doubling ratio experiments” — ba trang, dùng được ngay.
nền tảng	Tarjan, “Amortized Computational Complexity”	SIAM Journal on Algebraic and Discrete Methods, 1985	Bài báo hệ thống hoá ba phương pháp khấu hao. Đọc phần thể năng nếu muốn hiểu vì sao splay tree và Fibonacci heap phải phân tích kiểu đó.
tra khi cần	Bentley & McIlroy, “Engineering a Sort Function”	Software: Practice and Experience, 1993	Tài liệu tốt nhất về khoảng cách giữa “thuật toán trong sách” và “hàm dùng được”: ngưỡng hoà vốn, chọn chốt, dữ liệu ác ý.
tra khi cần	Frigo, Leiserson, Prokop, Ramachandran, “Cache-Oblivious Algorithms”	FOCS, 1999	Đọc khi muốn một mô hình chi phí có tính tới phân cấp bộ nhớ mà không phải biết kích thước cache ($D/24$).
tra khi cần	McSherry, Isard, Murray, “Scalability! But at What COST?”	HotOS, 2015	Sáu trang, đổi cách bạn đọc mọi báo cáo hiệu năng song song. Đọc bảng so sánh với chương trình đơn luồng.

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, “The Role of Deliberate Practice in the Acquisition of Expert Performance,” *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [7] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [10] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [11] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [12] C. A. R. Hoare, “An Axiomatic Basis for Computer Programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] D. Gries, *The Science of Programming*. Springer, 1981.
- [14] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [15] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [16] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [17] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [18] e-maxx / cp-algorithms contributors. “cp-algorithms: Algorithms for Competitive Programming.” (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [19] C. A. R. Hoare, “Quicksort,” *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [20] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [21] R. E. Tarjan, “Amortized Computational Complexity,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [22] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [23] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [24] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.

- [25] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.
- [26] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [34] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [35] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [36] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [37] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [38] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [39] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [40] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Tính đúng và chứng minh (Correctness and Proof)

PHỤ THUỘC A/01 cho vai trò của chứng minh trong bước 3; A/03 cho ngôn ngữ chỉ phí. Chương này là điều kiện cần cho C/15 (tham lam) và C/16 (quy hoạch động), nơi “có vẻ đúng” hỏng nhiều nhất.

TÓM TẮT

Chương này cấp cái thước thứ hai: *lời giải của tôi có đúng không*. Công cụ trung tâm là **bất biến** (*invariant*) — một mệnh đề luôn đúng tại một điểm cố định trong thuật toán. Bất biến có hai vai trò và vai trò thứ hai hay bị bỏ quên: nó vừa để chứng minh, vừa để *nghe ra* thuật toán. Chương đi qua bốn kiểu lập luận dùng nhiều nhất — bất biến vòng lặp, quy nạp, lập luận đổi chỗ cho tham lam, lập luận cắt-dán cho quy hoạch động — cộng với cách tìm phản ví dụ và cách chứng minh thuật toán *dùng*. Nếu chỉ nhớ một điều: viết một dòng bất biến trước mỗi vòng lặp là việc rẻ nhất bạn có thể làm để loại bỏ phần lớn lỗi lệch một đơn vị và lỗi khởi tạo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bất biến vòng lặp (loop invariant)** — mệnh đề đúng ngay trước mỗi lần lặp và vẫn đúng sau mỗi lần lặp. Ba việc phải kiểm: khởi tạo đúng, được duy trì, và khi thoát thì cho ra kết luận cần.
- **quy nạp (induction)** — chứng minh mệnh đề đúng cho mọi n bằng cách chứng minh nó đúng cho trường hợp nhỏ nhất, rồi chứng minh “đúng tới $n - 1$ kéo theo đúng ở n ”.
- **đại lượng giảm (variant / measure)** — một số nguyên không âm giảm nghiêm ngặt sau mỗi vòng lặp. Dùng để chứng minh thuật toán *dùng*, thứ mà bất biến không đảm bảo.
- **lập luận đổi chỗ (exchange argument)** — kỹ thuật chứng minh tham lam: lấy một lời giải tối ưu bất kỳ, biến đổi nó từng bước cho giống lựa chọn tham lam mà không làm nó tệ đi.
- **tiền điều kiện / hậu điều kiện** — điều kiện phải đúng trước khi chạy một đoạn mã, và điều kiện được đảm bảo sau khi chạy xong. Ngôn ngữ của Hoare[12].
- **phản ví dụ** — một dữ liệu vào cụ thể làm thuật toán cho kết quả sai. Một phản ví dụ đủ để bác bỏ; một triệu ví dụ đúng không đủ để chứng minh.

CÂU HỎI MỞ ĐẦU

- Q1. Chạy đúng trên một nghìn bộ test ngẫu nhiên — vì sao vẫn chưa phải bằng chứng đúng, và loại lỗi nào sống sót qua kiểm thử ngẫu nhiên?
- Q2. Bất biến là công cụ chứng minh. Vì sao chương này lại nói nó cũng là công cụ *sinh* thuật toán?
- Q3. Vì sao chứng minh một thuật toán tham lam khó hơn hẳn chứng minh một thuật toán quy hoạch động, dù code tham lam ngắn hơn nhiều?
- Q4. Một thuật toán có thể luôn giữ đúng bất biến mà vẫn sai. Sai ở đâu, và cần thêm gì?
- Q5. Khi nào thì bỏ việc chứng minh là quyết định hợp lý, chứ không phải lười?

Có một trải nghiệm mà ai luyện thuật toán cũng gặp: bạn viết xong, chạy thử vài ví dụ, thấy đúng, nộp bài, và nhận về “sai ở test 47”. Bạn không biết test 47 là gì. Bạn nhìn code, mọi dòng đều hợp lý. Và bạn bắt đầu sửa mò — đổi $<$ thành $\leq$, đổi khởi tạo, nộp lại. Đó là trạng thái tệ nhất trong việc lập trình: sửa mà không hiểu, và không biết khi nào thì xong.

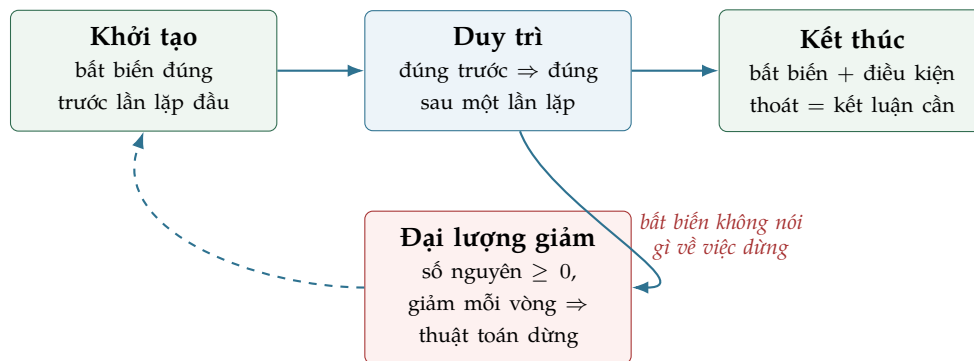
Chương này nói về cách thoát khỏi trạng thái đó, và câu trả lời không phải “kiểm thử nhiều hơn”. Dijkstra đã nói câu chính xác nhất về giới hạn của kiểm thử: nó cho thấy sự *hiện diện* của lỗi, không bao giờ cho thấy sự *vắng mặt* [11]. Muốn biết chương trình đúng, phải có một lập luận — và lập luận đó không nhất thiết phải hình thức, phần lớn thời gian nó chỉ là một hai câu viết ra cho rõ ràng.

Điều bất ngờ, và là luận điểm chính của chương: lập luận đó không phải việc làm *sau* khi có thuật toán. Làm đúng cách, nó chính là việc giúp bạn *tìm ra* thuật toán.

4.1 Bất biến vòng lặp: ba việc phải kiểm

Bất biến là một mệnh đề đúng tại một điểm cố định trong vòng lặp — thường là ngay trước mỗi lần lặp. Để dùng nó chứng minh một vòng lặp, cần kiểm ba việc, đúng như cấu trúc của phép quy nạp.

Khởi tạo: bất biến đúng trước lần lặp đầu tiên. *Duy trì*: nếu đúng trước một lần lặp thì vẫn đúng trước lần lặp kế tiếp. *Kết thúc*: khi vòng lặp thoát, bất biến cộng với điều kiện thoát cho ra đúng điều ta cần.



Hình 4.1. Bộ khung chứng minh một vòng lặp. Ba ô trên là bất biến; ô dưới là thứ thường bị quên: bất biến chỉ nói “nếu dừng thì đúng”, nó không hứa vòng lặp sẽ dừng. Muốn có tính dừng cần thêm một đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng.

Hãy làm một ví dụ đủ nhỏ để thấy hết. Tìm kiếm nhị phân trên mảng đã sắp tăng $a[0..n-1]$, tìm vị trí đầu tiên có $a[i] \geq x$. Bất biến chọn như sau: *mọi phần tử ở vị trí $< lo$ đều nhỏ hơn x , và mọi phần tử ở vị trí $\geq hi$ đều $\geq x$* . Nói cách khác, đáp án luôn nằm trong đoạn $[lo, hi]$.

Khởi tạo: đặt $lo = 0, hi = n$. Hai mệnh đề đúng một cách rỗng vì không có vị trí nào < 0 hay $\geq n$. **Duy trì**: lấy mid ở giữa; nếu $a[mid] < x$ thì mọi vị trí $\leq mid$ cũng nhỏ hơn x (vì mảng đã sắp), nên đặt $lo = mid + 1$ vẫn giữ bất biến; ngược lại đặt $hi = mid$. **Kết thúc**: khi $lo = hi$, bất biến nói mọi thứ bên trái nhỏ hơn x và mọi thứ từ lo trở đi $\geq x$ — đúng định nghĩa của vị trí cần tìm.

Điểm đáng chú ý: mọi câu hỏi cài đặt gây bối rối đều có câu trả lời rút ra từ bất biến, chứ không phải từ việc thử. hi khởi tạo bằng n hay $n - 1$? Bằng n , vì bất biến nói hi là vị trí đầu tiên đã biết là $\geq x$, mà có thể không có vị trí nào như vậy. Cập nhật $lo = mid + 1$ hay $lo = mid$? Bằng $mid + 1$, vì ta vừa chứng minh $a[mid] < x$ nên mid không thể là đáp án. Vòng lặp chạy khi $lo < hi$ hay $lo \leq hi$? Khi $lo < hi$, vì $lo = hi$ đã là kết luận.

CẦN NHỚ

Viết bất biến *trước* khi viết vòng lặp, dưới dạng một dòng chú thích bằng tiếng Việt. Đây là thói quen rẻ nhất và hiệu quả nhất trong cả quyển sách: nó biến các quyết định “< hay

<=" từ chỗ đoán mò thành chỗ suy ra được. Bentley[6] nhận xét rằng rất nhiều lập trình viên chuyên nghiệp viết sai tìm kiếm nhị phân khi phải viết từ đầu — gần như luôn vì họ viết code trước rồi mới nghĩ về ý nghĩa của các biến.

4.2 Tính dừng: thứ bất biến không hứa

Một điểm tinh tế mà người mới hay bỏ qua: bất biến chỉ chứng minh *nếu vòng lặp dừng thì kết quả đúng*. Nó hoàn toàn không nói vòng lặp sẽ dừng. Vòng lặp `while (true) { }` giữ mọi bất biến một cách hoàn hảo.

Công cụ cho tính dừng là *đại lượng giảm*: một số nguyên không âm, giảm ít nhất một đơn vị sau mỗi vòng. Vì số nguyên không âm không giảm được vô hạn lần, vòng lặp phải dừng. Với tìm kiếm nhị phân, đại lượng đó là $hi - lo$. Với thuật toán Euclid tính ước chung lớn nhất, là giá trị của số nhỏ hơn. Với thuật toán Ford–Fulkerson trên đồ thị có trọng số nguyên (chương C/18), là tổng luồng còn thiếu — và chính chỗ này lộ ra vì sao thuật toán đó có thể không dừng khi trọng số là số vô tỉ.

Trong đệ quy, đại lượng giảm chính là “kích thước bài toán con nhỏ hơn kích thước bài toán gốc”. Nghe hiển nhiên, nhưng đây đúng là chỗ hỏng của rất nhiều lời giải đệ quy sai: một nhánh nào đó gọi lại chính nó với cùng kích thước.

4.3 Quy nạp: khuôn của mọi chứng minh về cấu trúc đệ quy

Quy nạp và đệ quy là hai mặt của một đồng xu: đệ quy giải bài lớn bằng bài nhỏ, quy nạp chứng minh mệnh đề cho n bằng mệnh đề cho các giá trị nhỏ hơn. Vì vậy mọi thuật toán đệ quy đều được chứng minh tự nhiên bằng quy nạp, và hai phần khớp nhau từng dòng.

Jeff Erickson[29] có một cách diễn đạt rất hữu ích cho việc viết đệ quy: hãy giả định rằng mọi lời gọi đệ quy *đã* cho kết quả đúng — ông gọi đó là “bà tiên đệ quy” — và bạn chỉ cần lo đúng một việc: từ kết quả của các bài toán con, ghép ra kết quả của bài toán gốc. Đây không phải mẹo tâm lý mà chính là bước quy nạp, viết dưới dạng dễ dùng. Người mới hay tự làm khổ mình bằng cách cố hình dung toàn bộ cây đệ quy; điều đó vừa bất khả thi vừa không cần thiết.

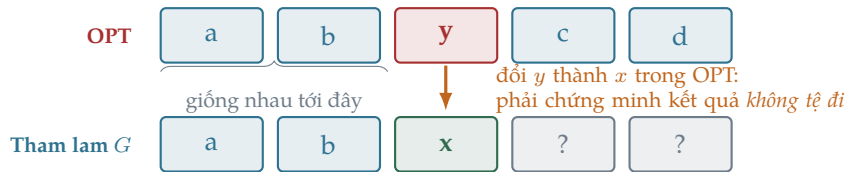
Hai biến thể đáng biết. *Quy nạp mạnh* giả định mệnh đề đúng cho *mọi* giá trị nhỏ hơn n , không chỉ $n - 1$ — cần thiết cho các thuật toán chia bài toán thành nhiều mảnh không đều. *Quy nạp cấu trúc* làm việc trên cây, đồ thị hoặc biểu thức thay vì trên số: chứng minh mệnh đề đúng ở lá, và đúng ở một nút nếu đúng ở mọi con của nó. Gần như mọi chứng minh về cây trong phần B đều theo khuôn này.

4.4 Lập luận đổi chỗ: vũ khí để chứng minh tham lam

Thuật toán tham lam là loại có khoảng cách lớn nhất giữa độ ngắn của code và độ khó của chứng minh. Code thường bốn dòng; chứng minh thì đòi hỏi thật sự, và nếu bỏ qua thì bạn sẽ nộp một lời giải sai với vẻ ngoài rất thuyết phục.

Khuôn chứng minh chuẩn là *lập luận đổi chỗ*: giả sử tồn tại một lời giải tối ưu OPT khác với lời giải tham lam G . Xét chỗ khác nhau *đầu tiên*. Chứng minh rằng ta có thể sửa OPT tại chỗ đó cho

giống G mà không làm nó tệ đi. Lặp lại, ta biến OPT thành G qua hữu hạn bước mà chất lượng không giảm — vậy G cũng tối ưu.



Hình 4.2. Sơ đồ lập luận đổi chỗ. Ta không cố chứng minh trực tiếp rằng tham lam tối ưu; ta chứng minh mọi lời giải tối ưu đều *biến đổi được* về lời giải tham lam mà không mất chất lượng. Toàn bộ công việc nằm ở mũi tên: chứng minh phép đổi tại vị trí khác nhau đầu tiên là hợp lệ và không làm tệ đi.

Ví dụ chuẩn: chọn nhiều nhất các cuộc họp không chồng lấn. Tham lam chọn cuộc họp *kết thúc sớm nhất*. Chứng minh: giả sử OPT chọn cuộc họp đầu tiên là y trong khi tham lam chọn x với thời điểm kết thúc $f(x) \leq f(y)$. Thay y bằng x trong OPT : mọi cuộc họp còn lại của OPT bắt đầu sau $f(y) \geq f(x)$, nên chúng vẫn không chồng lấn với x . Số cuộc họp không đổi, vậy lời giải mới vẫn tối ưu. Lặp lại cho vị trí thứ hai, thứ ba, và ta xong.

Một khuôn họ hàng là *tham lam luôn đi trước* (greedy stays ahead): chứng minh bằng quy nạp rằng sau k bước, lời giải tham lam tốt ít nhất bằng bất kỳ lời giải nào khác sau k bước. Với bài xếp lịch trên, sau k cuộc họp, thời điểm kết thúc của tham lam luôn sớm nhất có thể — và đó chính là lý do nó không bao giờ bị kẹt.

CẠM BÃY

Cạm bẫy chết người của tham lam: dùng “tôi thử vài ví dụ thấy đúng” làm chứng minh. Nhiều tiêu chí tham lam sai chỉ hổng ở những ví dụ khá đặc biệt. Ví dụ bài đổi tiền với các mệnh giá 1, 3, 4 và số tiền 6: tham lam lấy 4 rồi 1 rồi 1 (ba tờ), trong khi tối ưu là 3 + 3 (hai tờ). Nếu bạn chỉ thử với mệnh giá quen thuộc như 1, 5, 10 thì không bao giờ thấy lỗi. Quy tắc: hoặc chứng minh được bằng đổi chỗ, hoặc stress test với lời giải vét cạn (chương E/25); “thấy đúng” không nằm trong hai lựa chọn đó.

4.5 Lập luận cắt-dán: nền của quy hoạch động

Quy hoạch động đúng khi bài toán có *cấu trúc con tối ưu*: lời giải tối ưu của bài lớn chứa trong nó lời giải tối ưu của các bài con. Chứng minh chuẩn cho tính chất này là *cắt-dán*: giả sử lời giải tối ưu của bài lớn chứa một lời giải *không* tối ưu cho bài con nào đó; cắt phần đó ra, dán lời giải tối ưu của bài con vào, và ta được lời giải tốt hơn cho bài lớn — mâu thuẫn.

Nghe hình thức, nhưng nó có tác dụng thực hành rất cụ thể: khi cấu trúc con tối ưu *không* đúng, lập luận cắt-dán sẽ hổng ở một chỗ nhìn thấy được, và chỗ hổng đó cho bạn biết trạng thái đang thiếu chiều nào. Ví dụ kinh điển: đường đi *dài nhất* không có chu trình trong đồ thị tổng quát không có cấu trúc con tối ưu — ghép hai đường dài nhất có thể tạo ra đỉnh lặp. Nhận ra điều đó bằng cắt-dán nhanh hơn nhiều so với cài xong rồi mới thấy sai.

4.6 Tìm phản ví dụ: kỹ năng đối ngẫu với chứng minh

Chứng minh và phản ví dụ là hai mặt của cùng một việc, và người giải bài giỏi luôn làm cả hai xen kẽ: thử chứng minh vài phút, nếu chỗ nào không chứng minh được thì soi đúng chỗ đó để tìm phản ví dụ.

Ba nơi phản ví dụ hay ẩn nấp. *Trường hợp suy biến*: rỗng, một phần tử, mọi phần tử bằng nhau, đồ thị không liên thông, cạnh trọng số 0 hoặc âm. *Ranh giới của tiêu chí*: khi hai lựa chọn hoà nhau theo tiêu chí tham lam, cách phá hoà quyết định đúng sai. *Chỗ mà lập luận của bạn phải nói “hiển nhiên”*: mỗi lần bạn thấy mình viết chữ đó, hãy dừng lại kiểm tra — đó là nơi có mật độ lỗi cao nhất.

Và khi tìm bằng tay không ra, hãy để máy tìm: sinh dữ liệu nhỏ ngẫu nhiên, chạy song song lời giải nhanh và lời giải vét cạn, so kết quả. Kỹ thuật này — stress test — được nói kỹ ở chương E/25, và nó là cách tìm phản ví dụ hiệu quả nhất mà cũng ít tốn công nhất.

4.7 Chứng minh tới mức nào là đủ

Cần trung thực về liều lượng. Chứng minh hình thức đầy đủ cho mọi dòng code là điều không ai làm trong thực tế, kể cả người viết sách về chứng minh. Câu hỏi đúng không phải “có chứng minh hay không” mà là “chứng minh chặt tới mức nào cho phần nào”.

Có ba mức, và chọn mức là một quyết định kỹ thuật. *Mức một* — *một câu bằng lời*: dùng cho mọi vòng lặp, mọi lúc. Ví dụ “sau mỗi bước, best là đáp án tốt nhất trong $a[0..i]$ ”. Rẻ tới mức không có lý do gì để bỏ. *Mức hai* — *lập luận đầy đủ nhưng không hình thức*: dùng cho mỗi tiêu chí tham lam, mỗi công thức truy hồi quy hoạch động, mỗi chỗ bạn thấy “hình như đúng”. Đây là mức mà chương này chủ yếu dạy. *Mức ba* — *hình thức hoá*: dành cho những chỗ thật sự tinh vi, hiếm, và cho phần mềm mà lỗi có giá quá đắt.

Ngược lại, có những lúc bỏ chứng minh là hợp lý: trong kỳ thi khi bạn còn 10 phút và có thể stress test thay thế; khi thuật toán là một biến thể rất gần với một thuật toán đã được chứng minh; hoặc khi hậu quả của sai là nhỏ và phát hiện được ngay. Điều không hợp lý là bỏ chứng minh và bỏ luôn stress test, rồi tin vào cảm giác.

TRỰC GIÁC

Vì sao viết bất biến lại giúp *nghĩ ra* thuật toán chứ không chỉ kiểm chứng nó? Vì bất biến buộc bạn phát biểu “sau khi xử lý xong i phần tử đầu, tôi đang giữ đúng thông tin gì”. Câu hỏi đó chính là câu hỏi thiết kế trung tâm: nếu thông tin bạn giữ đủ để tính bước tiếp theo thì bạn có thuật toán; nếu chưa đủ thì bạn vừa phát hiện chính xác cái còn thiếu, và thứ còn thiếu đó thường trở thành một chiều mới của trạng thái. Đây là lý do người quen viết bất biến ít khi phải viết lại code từ đầu: họ phát hiện lỗ hổng ở giai đoạn suy nghĩ, chỗ mà sửa còn rẻ.

4.8 Máy trạng thái: một khung duy nhất cho ba loại bất biến

Chương này đã dùng bất biến ở ba vai trò trông rất khác nhau: bất biến vòng lặp để chứng minh một thuật toán đúng (mục 1), bất biến kiểu olympiad — chắn lẻ, tô màu — để chứng minh một cấu hình *không thể* đạt tới, và ở chương C/17 sẽ có thêm đồ thị trạng thái mà ta duyệt bằng tìm kiếm. Ba vai trò ấy là một, và Lehman, Leighton cùng Meyer[39] trình bày khung gộp chúng gọn hơn cả: **máy trạng thái** (*state machine*).

Máy trạng thái gồm ba thứ: một tập trạng thái, một trạng thái đầu, và một quan hệ chuyển. Một **bất biến bảo toàn** (*preserved invariant*) là một vị từ P trên trạng thái sao cho hễ $P(q)$ đúng và có bước chuyển $q \rightarrow r$ thì $P(r)$ cũng đúng. *Nguyên lý bất biến* phát biểu: nếu P đúng ở trạng thái đầu và được bảo toàn qua mọi bước chuyển, thì P đúng ở mọi trạng thái tới được. Nói cho đủ, đây chỉ là quy nạp theo số bước — nhưng viết ở dạng này thì dùng được ngay mà không phải dựng lại phép quy nạp mỗi lần.

Ví dụ nhỏ nhất trong sách của họ đáng nhớ vì nó gói cả ý tưởng trong ba dòng. Một robot xuất phát tại $(0, 0)$ trên lưới nguyên, mỗi bước đi chéo: $(m, n) \rightarrow (m \pm 1, n \pm 1)$. Nó có tới được $(1, 0)$ không? Xét vị từ “ $m + n$ chẵn”. Nó đúng ở trạng thái đầu, và mỗi bước làm tổng đổi đi $0, +2$ hoặc -2 nên tính chẵn được giữ. Mà $(1, 0)$ có tổng lẻ. Vậy robot *không bao giờ* tới được $(1, 0)$ — và chú ý loại kết luận: đây là điều mà không lượng thời gian chạy tìm kiếm nào chứng minh được, vì tìm kiếm chỉ có thể chạy mãi mà không tìm thấy. Bài Fifteen Puzzle trong cùng chương của họ là bản lớn hơn của đúng lập luận này.

Khung này đáng nhớ vì nó gộp ba việc thành một, và mỗi lần gộp là một lần bạn phải nhớ ít đi.

Vòng lặp là một máy trạng thái mà trạng thái là bộ giá trị các biến và bước chuyển là một lần chạy thân vòng lặp. Bất biến vòng lặp chính là bất biến bảo toàn, và “ba việc phải kiểm” ở mục 1 — đúng lúc đầu, được giữ qua mỗi vòng, kết luận được điều cần khi thoát — đúng là ba phần của nguyên lý trên.

Bài toán tìm kiếm trên không gian trạng thái cũng vậy, chỉ khác là ta duyệt thay vì lập luận. Và đây là chỗ khung này trả công nhiều nhất trong lúc thi: trước khi viết duyệt rộng trên hàng triệu trạng thái, hãy dành một phút hỏi có bất biến nào không. Nếu trạng thái đích vi phạm một bất biến thì khỏi chạy — đáp án là “không thể”. Nếu bất biến chia không gian thành nhiều lớp thì bạn chỉ phải duyệt lớp chứa trạng thái đầu, và không gian bài toán nhỏ đi theo đúng số lớp.

Còn tính dừng thì bất biến không nói gì, đúng như mục 2 đã cảnh báo. Máy trạng thái làm rõ chỗ khác nhau: bất biến là phát biểu về *tập trạng thái tới được*, còn đại lượng giảm là phát biểu về *độ dài của đường đi*. Hai câu hỏi khác nhau thì cần hai công cụ khác nhau, và lần chúng là lỗi phổ biến khi người ta tưởng đã chứng minh xong.

Một mẹo thực hành khấp lại mục: khi đã phát biểu được bất biến thành một câu, hãy cài luôn nó thành một phép kiểm assert chạy sau mỗi bước trong bản gỡ lỗi. Bất biến bị vi phạm ở đúng bước đầu tiên là thông tin đắt nhất mà bạn có thể có khi đi tìm lỗi (E/25).

4.9 Gốc rễ: chứng minh chương trình bắt đầu từ đâu

Ý tưởng lập luận về chương trình thay vì chỉ chạy thử nó xuất hiện sớm hơn nhiều người tưởng. Năm 1949, trong một bài viết ngắn về việc kiểm tra một chương trình lớn, Alan Turing đã trình bày gần như đầy đủ bộ khung của chương này: gắn vào từng điểm của chương trình một khẳng định về trạng thái, kiểm tra rằng mỗi bước bảo toàn các khẳng định đó, và — điểm đáng chú ý nhất — chứng minh riêng tính dừng bằng một đại lượng giảm. Nghĩa là ngay từ những năm đầu tiên của máy tính, người ta đã thấy *bất biến* và *đại lượng giảm* là hai thứ tách biệt và cùng cần thiết.

Ý tưởng đó bị bỏ quên gần hai mươi năm, cho tới khi Robert Floyd đưa ra cách gán ý nghĩa cho sơ đồ khối bằng các khẳng định (1967), và Tony Hoare biến nó thành hệ tiên đề với bộ ba tiên đề điều kiện–lệnh–hậu điều kiện (1969) [12]. Bối cảnh lịch sử giải thích vì sao chuyện này xảy ra vào đúng lúc đó: cuối thập niên 1960 là thời kỳ mà giới công nghiệp bắt đầu nói về “khủng hoảng phần mềm” — phần cứng mạnh lên nhanh, chương trình lớn lên theo, và số lỗi vượt quá khả năng kiểm soát bằng cách chạy thử. Dijkstra đẩy phản ứng ấy tới cực đoan với luận điểm rằng chương trình phải được *dẫn xuất* cùng với chứng minh của nó [11], và Gries biến nó thành sách dạy được [13].

Nhánh này về sau chia làm hai. Nhánh thực dụng thấm vào thói quen hằng ngày: khẳng định trong code, kiểm thử dựa trên tính chất, và chính lối viết bất biến bằng lời mà chương này khuyến nghị. Nhánh hình thức đi tới các công cụ kiểm chứng tự động — kiểm tra mô hình và trợ lý chứng minh — và tới thập niên 2000 đã đủ mạnh để chứng minh trọn vẹn những phần mềm thật: một trình biên dịch C có chứng minh và một nhân hệ điều hành có chứng minh là hai ví dụ được nhắc nhiều nhất. Cái giá là công sức lớn hơn viết code thông thường nhiều lần, nên nó chỉ dừng ở nơi mà một lỗi đắt tới mức đáng trả cái giá đó.

Với người luyện thuật toán, bài học từ lịch sử này rất cụ thể: bạn không cần nhánh hình thức, nhưng bạn cần đúng cái mà Turing đã dùng năm 1949 — một khẳng định ở mỗi điểm quan trọng, và một đại lượng giảm.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1949 — Turing	Kiểm tra một chương trình dài mà chạy thử không xuể.	Khẳng định tại từng điểm + đại lượng giảm cho tính dừng; bộ khung của cả chương này đã có mặt.
1967 — Floyd	Cần cách gán ý nghĩa chính xác cho sơ đồ khối.	Khẳng định gắn vào cạnh của sơ đồ; chính thức hoá ý tưởng bất biến.
1969 — Hoare	Muốn lập luận về chương trình như lập luận trong toán học.	Bộ ba $\{P\} S \{Q\}$; ngôn ngữ tiền điều kiện-hậu điều kiện dùng tới ngày nay.
Thập niên 1970 — Dijkstra, Gries	“Khủng hoảng phần mềm”: chương trình lớn hơn khả năng kiểm soát bằng chạy thử.	Chương trình và chứng minh phát triển cùng nhau; kiểm thử bị hạ xuống vai trò phát hiện lỗi, không phải bảo chứng.
Thập niên 1980–2000 — kiểm tra mô hình, trợ lý chứng minh	Hệ thống tương tranh và hệ thống trọng yếu mà lỗi có giá quá đắt.	Kiểm chứng tự động; đủ sức chứng minh trọn vẹn trình biên dịch và nhân hệ điều hành, với chi phí công sức rất lớn.

■ 4.10 Liên hệ ngang: bất biến là ý tưởng dùng chung của nhiều ngành

Đây có lẽ là chỗ liên hệ đẹp nhất trong cả phần A. Khái niệm “một đại lượng không đổi trong khi mọi thứ khác thay đổi” không phải phát minh của khoa học máy tính; nó là một trong những công cụ tư duy phổ quát nhất mà con người có, và nó xuất hiện dưới những cái tên khác nhau ở khắp nơi.

Trong vật lý, đó là các định luật bảo toàn: năng lượng, động lượng, điện tích. Ta không giải phương trình chuyển động cho từng hạt trong một vụ va chạm — ta viết ra đại lượng được bảo toàn và suy ngay ra kết quả. Đó chính xác là cách bất biến được dùng để chứng minh một thuật toán mà không phải theo dõi từng vòng lặp.

Trong lý thuyết điều khiển, đó là hàm Lyapunov: một hàm không âm của trạng thái, luôn giảm theo thời gian, dùng để chứng minh hệ hội tụ về điểm cân bằng. So sánh với mục 2 của chương này thì thấy ngay: hàm Lyapunov *chính là* đại lượng giảm, chỉ khác ở chỗ nó nhận giá trị thực và giảm liên tục thay vì nguyên và giảm rời rạc. Cùng một ý: gán một con số cho trạng thái, chứng minh con số đó đi đúng một chiều, và kết luận về hành vi dài hạn mà không phải mô phỏng.

Trong toán olympiad, bất biến và bán bất biến là một chủ đề riêng: tô màu bàn cờ, xét tính chẵn lẻ, xét tổng theo modulo — tất cả để chứng minh một cấu hình *không thể* đạt tới. Đây cũng là cách dùng mà lập trình viên hay quên: bất biến không chỉ để chứng minh thuật toán đúng, mà còn để chứng minh một việc là bất khả thi, tức là để *ngừng tìm* một lời giải không tồn tại.

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Vật lý	Định luật bảo toàn năng lượng, động lượng	Cùng một chiến lược: thay vì theo dõi diễn biến chi tiết, tìm đại lượng không đổi rồi suy trực tiếp ra kết luận. Khác: bất biến vật lý do tự nhiên quy định, còn bất biến thuật toán do bạn <i>chọn</i> — chọn khéo hay vụng quyết định chứng minh dễ hay khó.
Lý thuyết điều khiển	Hàm Lyapunov: hàm không âm, giảm dọc quỹ đạo $\Rightarrow$ hệ ổn định	Chính là đại lượng giảm ở mục 2, phiên bản liên tục. Cả hai đều chứng minh “rồi sẽ tới đích” mà không cần giải ra nghiệm. Khác: hệ liên tục cần thêm điều kiện để loại trường hợp giảm mãi mà không tới nơi.
Toán olympiad	Bất biến và bán bất biến: chẵn lẻ, tô màu, tổng theo modulo	Dùng để chứng minh <i>bất khả thi</i> — công dụng hay bị bỏ quên. Trong lập trình, nó tiết kiệm hàng giờ: nhận ra cấu hình đích vi phạm một bất biến thì bạn ngừng tìm thuật toán ngay.
Kế toán	Ghi sổ kép: tổng nợ luôn bằng tổng có	Một bất biến toàn hệ thống được kiểm tra liên tục; lệch một đồng là dấu hiệu có lỗi ở đâu đó. Tương đương assert chạy ở mọi bước.
Cơ sở dữ liệu	Ràng buộc toàn vẹn và tính chất giao dịch	Giao dịch được định nghĩa đúng bằng ngôn ngữ bất biến: hệ đúng trước, có thể tạm sai ở giữa, phải đúng lại khi kết thúc — y hệt cấu trúc khởi tạo / duy trì / kết thúc.
Ước lượng trạng thái, SLAM	Ràng buộc khả quan sát: bộ lọc không được tự sinh thông tin về những hướng mà dữ liệu không quan sát được	Đây là một bất biến thật của bộ ước lượng, và vi phạm nó gây ra sai lệch có hệ thống chứ không phải nhiễu ngẫu nhiên. Cùng bài học với mục 6: bất biến vi phạm thâm lạng nguy hiểm hơn lỗi làm chương trình sập.
Trình biên dịch	Bất biến kiểu (type safety): chương trình đã qua kiểm kiểu thì không kẹt ở trạng thái vô nghĩa	Cùng khuôn ba bước, áp lên toàn ngôn ngữ thay vì một vòng lặp. Đây là ví dụ cho thấy bất biến mở rộng được từ một hàm lên cả một hệ thống.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì kiểm thử chỉ chứng minh cho những dữ liệu đã thử. Ba loại lỗi sống sót qua kiểm thử ngẫu nhiên: lỗi ở trường hợp suy biến mà bộ sinh ngẫu nhiên hầu như không tạo ra (mảng rỗng, mọi phần tử bằng nhau, $n = 1$); lỗi chỉ xuất hiện ở kích thước lớn (tràn số, quá thời gian); và lỗi ở những cấu hình xác suất thấp (một tiêu chí tham lam hỏng đúng khi có hoà). Đó là ý câu của Dijkstra: kiểm thử cho thấy sự hiện diện của lỗi, không cho thấy sự vắng mặt.
- A2. Vì viết bất biến chính là trả lời câu hỏi “sau i bước tôi giữ đúng thông tin gì”, mà đó là câu hỏi thiết kế chứ không phải câu hỏi kiểm chứng. Nếu thông tin đang giữ đủ để bước tiếp thì bạn đã có thuật toán; nếu chưa đủ thì phần còn thiếu chỉ đúng ra thứ phải bổ sung — rất thường xuyên nó trở thành một chiều mới trong trạng thái quy hoạch động. Ví dụ mở đầu ở chương A/01 là minh họa: bất biến “cur là tổng lớn nhất của đoạn kết thúc tại j ” vừa là lời giải vừa là chứng minh.
- A3. Vì tham lam đưa ra một khẳng định mạnh hơn nhiều. Quy hoạch động chỉ khẳng định “tôi đã xét mọi khả năng phân rã”, và chứng minh chỉ cần kiểm rằng công thức truy hồi bao phủ hết các trường hợp. Tham lam khẳng định “lựa chọn cục bộ này an toàn cho *mọi* dữ liệu” — một mệnh đề toàn thể, cần lập luận đối chỗ để chứng minh, và sai chỉ với một phản ví dụ. Code ngắn hơn chính vì nó bỏ qua việc xét các khả năng, và cái giá là gánh nặng chứng minh chuyển sang bạn.
- A4. Sai ở tính dừng. Bất biến chỉ nói “nếu vòng lặp kết thúc thì kết quả đúng”; một vòng lặp vô hạn giữ mọi bất biến hoàn hảo. Cần thêm một đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng — *hi-lo* trong tìm kiếm nhị phân, số nhỏ hơn trong thuật toán Euclid. Trong đệ quy, đại lượng đó là kích thước bài toán con, và chỗ hỏng phổ biến là một nhánh gọi lại chính nó với cùng kích thước.
- A5. Khi có một cơ chế kiểm chứng khác đủ mạnh và rẻ hơn — điển hình là stress test đối chiếu với lời giải vét cạn, vốn bắt được gần hết lỗi logic trên dữ liệu nhỏ. Cũng hợp lý khi thuật toán chỉ

là biến thể rất sát một kết quả đã biết, hoặc khi chi phí của một lần sai là nhỏ và phát hiện được ngay. Không hợp lý là bỏ cả hai — không chứng minh và cũng không stress test — rồi dựa vào cảm giác “nhìn có vẻ đúng”.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn leo cầu thang trong bóng tối và muốn chắc chắn rằng cuối cùng mình sẽ lên tới sân thượng. Bạn cần hai điều. Điều thứ nhất: mỗi bước chân đều đặt lên một bậc thật, không hụt — đó là bất biến, thứ luôn đúng sau mỗi bước. Điều thứ hai: số bậc còn lại giảm đi sau mỗi bước — đó là đại lượng giảm, thứ bảo đảm bạn không đi mãi. Rất nhiều người chỉ nghĩ tới điều thứ nhất và ngạc nhiên khi chương trình chạy mãi không dừng.

Chứng minh một thuật toán tham lam thì giống chuyện khác. Bạn nói “lúc nào cũng cứ chọn cái kết thúc sớm nhất là ổn”. Người ta hỏi “sao biết?”. Cách trả lời thuyết phục nhất không phải kể vài ví dụ, mà là: đưa tôi bất kỳ cách xếp tốt nhất nào của bạn, tôi sẽ sửa nó từng chút một cho giống cách của tôi, và mỗi lần sửa tôi chứng minh nó không xấu đi. Sửa xong thì hai cách giống hệt nhau, vậy cách của tôi cũng tốt nhất.

Cuối cùng, có một điều nên nhớ về kiểm thử: chạy thử giống như kiểm tra vài viên gạch trong một bức tường. Thấy chục viên tốt không có nghĩa cả tường tốt. Nhưng chỉ cần tìm ra một viên vỡ là đủ biết tường có vấn đề — nên khi nghi ngờ, hãy tìm viên vỡ ở đúng những chỗ dễ vỡ nhất: góc tường, mép, chỗ mới xây.

Một câu: Bất biến trả lời “tôi đang giữ gì”; đại lượng giảm trả lời “tôi có dừng không”; đối chỗ trả lời “vì sao chọn thế là an toàn”.

Chỗ không rút gọn được: Không có quy trình máy móc nào tìm ra bất biến đúng; nó đến từ việc nhìn ví dụ nhỏ và hỏi “sau bước này tôi thực sự biết điều gì”.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba việc phải kiểm của bất biến	Khởi tạo (đúng trước lần lặp đầu), duy trì (đúng trước $\Rightarrow$ đúng sau), kết thúc (bất biến + điều kiện thoát = kết luận cần).
••• Bất biến không hứa điều gì	Không hứa vòng lặp dừng. Cần thêm đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng.
••• Bất biến của tìm kiếm nhị phân (lower bound)	Mọi vị trí $< lo$ có giá trị $< x$; mọi vị trí $\geq hi$ có giá trị $\geq x$. Từ đó suy ra mọi chi tiết cài đặt.
••• Lập luận đối chỗ	Lấy lời giải tối ưu bất kỳ, xét chỗ khác đầu tiên so với tham lam, chứng minh sửa cho giống tham lam không làm tệ đi; lặp lại tới khi trùng nhau.
••• Lập luận cắt-dán	Nếu lời giải tối ưu chứa một lời giải bài con không tối ưu, thay bằng lời giải tối ưu của bài con sẽ được kết quả tốt hơn — mâu thuẫn. Đây là chứng minh cấu trúc con tối ưu.
••• Vì sao tham lam khó chứng minh hơn quy hoạch động	Tham lam khẳng định “lựa chọn cục bộ an toàn cho mọi dữ liệu” — mệnh đề toàn thể; quy hoạch động chỉ khẳng định “đã xét mọi phân rã”.
•• Bà tiên đệ quy (Erickson)	Giả định mọi lời gọi đệ quy đã trả kết quả đúng; chỉ lo việc ghép kết quả bài con thành kết quả bài gốc. Đây chính là bước quy nạp.
•• Ba nơi phản ví dụ hay ẩn	Trường hợp suy biến; ranh giới hoà nhau của tiêu chí; và mọi chỗ lập luận của bạn phải nói “hiển nhiên”.
•• Phản ví dụ kinh điển của tham lam đối tiền	Mệnh giá 1, 3, 4 và số tiền 6: tham lam cho $4+1+1$, tối ưu là $3+3$.
•• Vì sao đường đi dài nhất không có cấu trúc con tối ưu	Ghép hai đường dài nhất của hai bài con có thể tạo đỉnh lặp, nên phép cắt-dán không hợp lệ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Ba mức chứng minh	Một câu bằng lời cho mọi vòng lặp; lập luận đầy đủ không hình thức cho tham lam và truy hồi; hình thức hoá chỉ cho chỗ thật tinh vi.
●● Quy nạp mạnh dùng khi nào	Khi bài toán chia thành nhiều mảnh không đều, cần giả định đúng với mọi giá trị nhỏ hơn n chứ không chỉ $n - 1$.
● Tham lam luôn đi trước	Chứng minh bằng quy nạp rằng sau k bước, tham lam tốt ít nhất bằng mọi lời giải khác sau k bước.
● Khi nào bỏ chứng minh là hợp lý	Khi có stress test thay thế, khi thuật toán là biến thể sát một kết quả đã biết, hoặc khi sai thì rẻ và phát hiện ngay. Không hợp lý là bỏ cả hai.
●● Máy trạng thái và bất biến bảo toàn	Trạng thái đầu + quan hệ chuyển; vị từ đúng lúc đầu và được giữ qua mỗi bước thì đúng ở mọi trạng thái tới được. Vòng lặp, bài duyệt không gian trạng thái và bất biến olympiad là cùng một khung.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao tìm ra bất biến đúng? Nhìn vòng lặp rồi tôi vẫn không biết viết gì.

| Đi ngược: hỏi “khi vòng lặp kết thúc, tôi muốn có kết luận gì?” rồi hỏi “mệnh đề nào, cộng với điều kiện thoát, cho ra kết luận đó?”. Với tìm kiếm nhị phân, kết luận muốn có là “ lo là vị trí đầu tiên $\geq x$ ”, điều kiện thoát là $lo = hi$; mệnh đề khớp là “đáp án nằm trong $[lo, hi]$ ”. Một cách khác cũng hiệu quả: viết ra trạng thái các biến sau mỗi vòng cho một ví dụ $n = 5$ cụ thể, rồi tìm câu mô tả đúng mọi dòng của bảng đó.

2. Chứng minh thuật toán Dijkstra bằng bất biến như thế nào?

| Bất biến: với mọi đỉnh đã “chốt”, nhãn khoảng cách của nó bằng khoảng cách ngắn nhất thật. Duy trì: khi chốt đỉnh u có nhãn nhỏ nhất trong các đỉnh chưa chốt, giả sử tồn tại đường ngắn hơn tới u ; đường đó phải rời tập đã chốt tại một cạnh nào đó, tới một đỉnh v chưa chốt, và vì mọi trọng số không âm nên phần đầu của đường đó đã dài ít nhất bằng nhãn của v , mà nhãn của v không nhỏ hơn nhãn của u — mâu thuẫn. Chú ý chỗ dùng giả thiết trọng số không âm: đó chính xác là chỗ thuật toán hỏng với cạnh âm.

3. Tôi chứng minh tham lam đúng nhưng vẫn sai khi nộp. Chuyện gì xảy ra?

| Ba khả năng, theo thứ tự tần suất. Thứ nhất, chứng minh có một bước bạn nói “rõ ràng” mà thực ra không đúng — hãy đọc lại đúng những chỗ đó. Thứ hai, tiêu chí bạn cài không hoàn toàn là tiêu chí bạn chứng minh, thường ở cách phá hoà khi hai phần tử bằng nhau. Thứ ba, thuật toán đúng nhưng cài sai. Cách phân biệt nhanh: stress test với vết cạn trên $n \leq 8$. Nếu sai trên dữ liệu nhỏ thì là lỗi một hoặc ba; nếu chỉ sai ở dữ liệu lớn thì thường là tràn số hoặc quá thời gian, không phải lỗi ý tưởng.

4. Bất biến trong code (assert) có thay được chứng minh không?

| Không thay được, nhưng bổ sung rất tốt. assert kiểm bất biến trên những dữ liệu thực sự chạy qua, nên nó bắt lỗi ở đúng chỗ phát sinh thay vì để lỗi lan ra và chỉ lộ ở kết quả cuối — điều này rút ngắn thời gian gỡ lỗi rất nhiều. Cách dùng tốt: mỗi bất biến bạn viết thành chú thích, nếu kiểm được rồi thì viết luôn thành assert. Nhớ tắt trong bản nộp nếu nó tốn thời gian.

5. Chứng minh quy hoạch động gồm những gì?

| Ba phần. Một, định nghĩa trạng thái phải rõ ràng và đủ: từ trạng thái, mọi quyết định sau này không cần biết thêm gì về quá khứ. Hai, công thức truy hồi phải bao phủ: mọi cách tạo ra trạng thái này đều nằm trong một nhánh của công thức. Ba, thứ tự tính phải hợp lệ: khi tính một trạng thái, mọi trạng thái nó phụ thuộc đã được tính — tức đồ thị phụ thuộc không có chu trình. Phần lớn lỗi quy hoạch động nằm ở điểm một (trạng thái thiếu chiều) hoặc điểm hai (quên một nhánh).

6. Có cần chứng minh cả những thuật toán kinh điển tôi chỉ dùng lại?

| Không cần chứng minh lại, nhưng cần biết *giả thiết* của chúng, vì đó là chỗ chúng hỏng. Dijkstra cần trọng số không âm. Tham lam Huffman cần chi phí là tổng có trọng số. Sắp xếp hai giai đoạn cần quan hệ so sánh có tính bắc cầu — một hàm so sánh không nhất quán sẽ làm `std::sort` hành xử không xác định, kể cả sắp chương trình. Quy tắc: thuộc giả thiết, không cần thuộc chứng minh.

7. Chứng minh bằng phản chứng và bằng đối chỗ khác nhau chỗ nào?

| Đối chỗ là một dạng chứng minh có tính xây dựng: nó chỉ ra cách biến lời giải tồi ưu thành lời giải tham lam, nên đọc xong bạn hiểu *vì sao* tham lam an toàn. Phản chứng chỉ nói “giả sử sai thì mâu thuẫn”, đúng nhưng ít soi sáng hơn. Trong thực hành, đối chỗ có lợi thế lớn: nếu phép đối không thực hiện được, chỗ hỏng thường chính là phản ví dụ đang cần tìm.

8. Phương pháp hàm thế năng ở chương A/03 có liên quan gì tới bất biến không?

| Rất gần nhau. Bất biến là mệnh đề đúng ở mọi thời điểm; hàm thế năng là một *đại lượng số* mà ta phát biểu bất biến về nó — “ Φ luôn không âm” — và ta lập luận qua độ biến thiên của nó. Đại lượng giảm để chứng minh tính dừng cũng cùng họ: cũng là một số nguyên với bất biến “không âm” và tính chất “giảm mỗi bước”. Ba công cụ này là ba biến tấu của cùng một ý: gán một con số cho trạng thái rồi lập luận về cách con số đó thay đổi.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết bất biến và chứng minh tính dừng cho sắp xếp chèn	[27]	Bất biến: đoạn $a[0..i-1]$ đã sắp và gồm đúng các phần tử ban đầu của đoạn đó.
● Viết bất biến cho thuật toán Euclid, chỉ ra đại lượng giảm	Bài tập kinh điển	Bất biến $\gcd(a, b)$ không đổi; đại lượng giảm là $\min(a, b)$.
● Tìm phản ví dụ cho tham lam “luôn lấy đồng tiền lớn nhất” với mệnh giá bất kỳ	Bài tập kinh điển	Mệnh giá 1, 3, 4 với số tiền 6. Tự tìm thêm một bộ mệnh giá khác cũng hỏng.
●● Chứng minh tham lam xếp lịch theo thời điểm kết thúc sớm nhất	[28]	Viết đầy đủ cả hai cách: đối chỗ và “tham lam luôn đi trước”.
●● Chứng minh cấu trúc con tối ưu của bài dây con chung dài nhất bằng cắt-dán	[27]	Xét ký tự cuối của hai chuỗi, chia hai trường hợp; cắt-dán ở từng trường hợp.
●● Chỉ ra chính xác chỗ chứng minh Dijkstra dùng giả thiết trọng số không âm	Chương C/17	Sau đó dựng một đồ thị có cạnh âm làm Dijkstra sai, càng nhỏ càng tốt.
●● Viết bất biến cho thuật toán hai con trỏ tìm cặp có tổng bằng x	Chương B/06	Bất biến: mọi cặp có chỉ số ngoài đoạn $[l, r]$ hiện tại đều đã bị loại một cách hợp lệ.
●●● Chứng minh thuật toán bỏ phiếu Boyer–Moore tìm phần tử quá bán	Bài tập kinh điển	Bất biến khó tìm; gợi ý: nghĩ theo kiểu “triệt tiêu từng cặp phần tử khác nhau”.
●●● Chứng minh tham lam của Kruskal đúng bằng lập luận đối chỗ	[28]	Tính chất cắt (cut property); đây là bài tập tiêu biểu cho cả họ chứng minh matroid ở chương C/15.
●●● Tìm một bài mà tham lam “có vẻ đúng” và dùng stress test để bác bỏ	Chương E/25	Mục tiêu kép: luyện viết bộ sinh test và luyện thói quen không tin cảm giác.
●●● Chứng minh tính dừng của Ford–Fulkerson với trọng số nguyên, và chỉ ra vì sao có thể không dừng với trọng số vô tỉ	Chương C/18	Đại lượng giảm chỉ tồn tại khi các bước tăng luồng bị chặn dưới bởi một hằng số dương.

Kết chương và đọc tiếp

Với chương này, phần A đã cấp đủ hai cái thước: [A/03](#) cho “đủ nhanh không”, chương này cho “có đúng không”. Ba công cụ đáng mang theo là bất biến (tôi đang giữ gì), đại lượng giảm (tôi có dừng không) và lập luận đối chỗ (vì sao lựa chọn này an toàn). Điểm quan trọng nhất, và cũng là điểm dễ quên nhất: cả ba đều là công cụ *thiết kế*, không chỉ công cụ kiểm chứng.

Còn một mảnh nữa của nền tảng: bộ toán mà các chương sau sẽ dùng rải rác — tổng, truy hồi, tổ hợp, xác suất. Đó là chương [A/05](#), và nó là chương duy nhất trong quyển sách này không nên đọc tuần tự. Sau đó, phần B bắt đầu với câu hỏi mới: dữ liệu nên nằm ở dạng nào để thao tác tôi cần trở nên rẻ.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Lehman, Leighton, Meyer, <i>Mathematics for Computer Science</i> , chương “State Machines”	sách giáo trình MIT, bản 2018	Nguồn của khung máy trạng thái và nguyên lý bất biến. Đọc mục 6.1–6.2 cùng bài tập Fifteen Puzzle — khoảng hai mươi trang, và là cách vào đề tốt nhất cho người viết code.
nền tảng	Hoare, “An Axiomatic Basis for Computer Programming”	Communications of the ACM, 1969	Bài mở đầu của cả ngành đặc tả và chứng minh chương trình. Ngắn, và đọc được ngay cả khi bạn không định dùng phương pháp hình thức.
tra khi cần	Gries, <i>The Science of Programming</i>	Springer, 1981	Sách dạy viết chương trình <i>cùng lúc</i> với chứng minh; đọc khi muốn kỹ năng phát biểu bất biến trở thành phản xạ.
tra khi cần	Erickson, <i>Algorithms</i>	bản điện tử miễn phí, 2019	Chương về đệ quy và quy nạp; nguồn của hình ảnh “bà tiên đệ quy” ở mục 3.

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, “The Role of Deliberate Practice in the Acquisition of Expert Performance,” *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [7] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [10] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [11] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [12] C. A. R. Hoare, “An Axiomatic Basis for Computer Programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] D. Gries, *The Science of Programming*. Springer, 1981.
- [14] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [15] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [16] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [17] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [18] e-maxx / cp-algorithms contributors. “cp-algorithms: Algorithms for Competitive Programming.” (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [19] C. A. R. Hoare, “Quicksort,” *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [20] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [21] R. E. Tarjan, “Amortized Computational Complexity,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [22] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [23] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [24] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.

- [25] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.
- [26] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [34] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [35] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [36] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [37] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [38] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [39] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [40] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Toán cho thuật toán (The Mathematics You Actually Reuse)

PHỤ THUỘC A/03 cho ký hiệu tiệm cận. Chương này không nên đọc tuần tự: đọc lướt một lượt để biết trong đó có gì, rồi quay lại đúng mục khi một chương sau cần tới.

TÓM TẮT

Đây là kho toán dùng lại nhiều nhất trong 23 chương còn lại, chia theo *câu hỏi nó trả lời* chứ không theo tên môn. Mục 1 là tổng và chặn — dùng mỗi lần bạn phân tích một vòng lặp. Mục 2 là tổ hợp đếm — dùng cho mọi bài đếm và cho việc ước lượng kích thước không gian tìm kiếm. Mục 3 là xác suất và kỳ vọng, với một công cụ mạnh bất thường là kỳ vọng tuyến tính. Mục 4 là số học modulo, mục 5 là đại số rời rạc, mục 6 là lý thuyết trò chơi tối thiểu. Nếu chỉ nhớ một điều: ba thứ quay lại nhiều nhất là chuỗi hình học bị chặn bởi hai lần số hạng lớn nhất, tổng điều hoà xấp xỉ $\ln n$, và kỳ vọng của tổng bằng tổng các kỳ vọng dù các biến phụ thuộc nhau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chuỗi hình học** — tổng dạng $1 + r + r^2 + \dots$; khi $r > 1$, tổng bị chi phối bởi số hạng cuối, khi $r < 1$ thì bởi số hạng đầu.
- **tổng điều hoà** — $H_n = 1 + 1/2 + \dots + 1/n \approx \ln n + 0,577$. Xuất hiện mỗi khi có “lần thứ i tốn $1/i$ ” hoặc “đếm bội số”.
- **hệ số nhị thức** — $\binom{n}{k}$ — số cách chọn k phần tử từ n phần tử, không kể thứ tự.
- **biến chỉ báo (indicator)** — biến nhận giá trị 1 nếu một sự kiện xảy ra và 0 nếu không. Kỳ vọng của nó bằng đúng xác suất của sự kiện.
- **số học modulo** — làm việc với phần dư khi chia cho m ; phép cộng, trừ, nhân đều bảo toàn phần dư, phép chia thì cần nghịch đảo.
- **nguyên lý chuồng bồ câu** — nhốt $n + 1$ con vào n chuồng thì có chuồng chứa từ hai con trở lên. Công cụ chứng minh tồn tại đơn giản nhất.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chuỗi $1 + 2 + 4 + \dots + n$ lại “chỉ bằng” $O(n)$, trong khi chuỗi $1 + 1/2 + \dots + 1/n$ lại không phải $O(1)$?
- Q2. Kỳ vọng của tổng bằng tổng các kỳ vọng ngay cả khi các biến phụ thuộc chằng chịt. Vì sao điều đó lại đúng, và vì sao nó mạnh đến vậy?
- Q3. Vì sao gần như mọi bài đếm trong lập trình thi đấu đều yêu cầu kết quả theo modulo một số nguyên tố lớn?
- Q4. Nguyên lý chuồng bồ câu tầm thường tới mức trẻ con cũng hiểu. Vì sao nó lại chứng minh được những điều không hiển nhiên chút nào?
- Q5. Khi nào một bài toán “tính giá trị sau 10^{18} bước” lại giải được, và cấu trúc nào làm nó giải được?

Chương này khác mọi chương khác trong quyển sách: nó không có một mạch lập luận từ đầu tới cuối, vì nó là kho chứ không phải là bài. Cách dùng đúng là đọc lướt một lượt ngay bây giờ —

đủ để nhớ “trong đó có gì” — rồi quay lại từng mục khi một chương sau thực sự cần. Mỗi mục đều ghi rõ nó quay lại ở chương nào.

Nguyên tắc chọn nội dung cũng cần nói rõ: tôi chỉ giữ những mảnh toán mà tôi thấy quay lại *ít nhất ba lần* trong các chương sau. Rất nhiều thứ đẹp đẽ đã bị bỏ ra ngoài, không phải vì chúng không hay mà vì học một công cụ không có nhu cầu đi kèm thì hai tuần sau sẽ quên sạch.

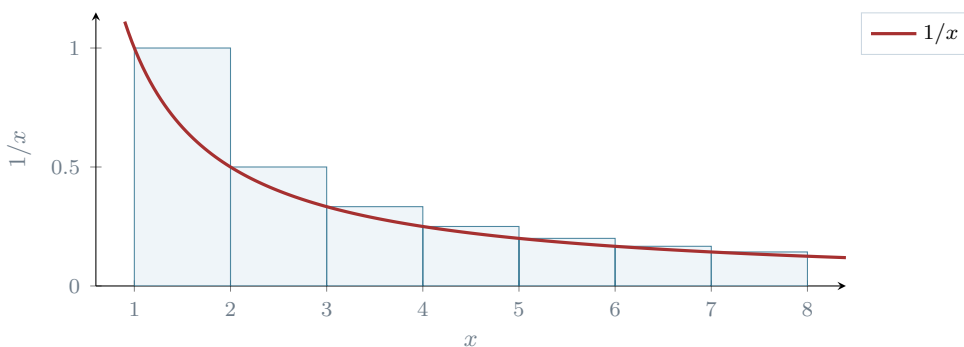
5.1 Tổng và chặn: đọc chi phí của một vòng lặp

Ba dạng tổng chiếm gần hết những gì bạn cần.

Cấp số cộng. $1 + 2 + \dots + n = n(n+1)/2 = \Theta(n^2)$. Xuất hiện khi vòng lặp trong chạy tới chỉ số của vòng lặp ngoài — ví dụ mọi thuật toán “xét mọi cặp”.

Cấp số nhân. $1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1 < 2 \cdot 2^k$. Điều đáng nhớ không phải công thức mà là câu này: **tổng của một cấp số nhân tăng bị chặn bởi hai lần số hạng lớn nhất**. Câu đó giải thích ngay chi phí khấu hao của mảng động (A/03), chi phí xây dựng cây phân đoạn, và chi phí tổng cộng của kỹ thuật nhân đôi. Ngược lại, với $r < 1$ thì tổng bị chặn bởi hằng số nhân số hạng đầu: $1 + 1/2 + 1/4 + \dots < 2$ — lý do vì sao chia đôi mãi vẫn chỉ tốn tuyến tính.

Tổng điều hoà. $H_n = \sum_{i=1}^n 1/i \approx \ln n$. Đây là tổng bị đánh giá sai nhiều nhất: nó *không* hội tụ, nhưng tăng cực chậm. Hình 5.1 cho thấy vì sao xấp xỉ bằng $\ln n$: tổng các cột chữ nhật kẹp giữa hai tích phân. Tổng này xuất hiện ở ba chỗ rất khác nhau và đáng nhớ cả ba: sàng Eratosthenes tốn $\sum_p n/p \approx n \ln \ln n$ (D/21); quicksort ngẫu nhiên tốn trung bình $2n \ln n$ so sánh (C/14); và bài toán “sưu tập đủ bộ” cần trung bình nH_n lần thử.



Hình 5.1. Vì sao $H_n \approx \ln n$. Mỗi cột có diện tích $1/i$, nên tổng diện tích các cột là H_n ; đường cong $1/x$ có tích phân từ 1 tới n bằng $\ln n$. Các cột kẹp đường cong từ trên, cho $\ln n < H_n < \ln n + 1$. Bài học tổng quát hơn: khi cần chặn một tổng, hãy so nó với tích phân của hàm tương ứng — đây là mẹo dùng được cho hầu hết các tổng đơn điệu.

Ngoài ba dạng trên, hai xấp xỉ đáng nhớ: $\log_2(n!) = \Theta(n \log n)$ — công thức Stirling, và là lý do cận dưới của sắp xếp so sánh là $\Omega(n \log n)$ (D/24); và $\binom{n}{k} \leq (en/k)^k$ — dùng để chặn kích thước không gian tìm kiếm.

5.2 Tổ hợp đếm: bao nhiêu khả năng, và đếm chúng thế nào

Bốn công thức nền, mỗi công thức ứng với một câu hỏi.

Chọn có thứ tự, không lặp: $n!/(n-k)!$. Chọn không thứ tự, không lặp: $\binom{n}{k}$. Chọn có lặp, có thứ tự: n^k . Chọn có lặp, không thứ tự (bài toán chia kẹo): $\binom{n+k-1}{k}$ — kỹ thuật “sao và vách ngăn”.

Hai đẳng thức đáng thuộc vì chúng biến thành thuật toán. *Đẳng thức Pascal* $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ chính là công thức truy hồi để tính bảng hệ số nhị thức bằng quy hoạch động, và cách chứng minh nó — “phần tử thứ n hoặc được chọn hoặc không” — là khuôn mẫu của mọi lập luận chia trường hợp trong chương C/16. *Nguyên lý bao hàm–loại trừ* cho phép đếm hợp của các tập bằng cách cộng rồi trừ các giao; nó là nền của kỹ thuật đếm bù — đếm cái xấu rồi lấy tổng trừ đi — vốn thường dễ hơn đếm trực tiếp cái tốt.

Số Catalan $C_n = \frac{1}{n+1} \binom{2n}{n}$ đáng biết vì nó đếm một họ bài toán trông rất khác nhau nhưng cùng cấu trúc: số dãy ngoặc đúng độ dài $2n$, số cây nhị phân có n nút, số cách chia đa giác thành tam giác, số đường đi không vượt đường chéo. Khi bạn tính tay các trường hợp nhỏ và ra dãy 1, 2, 5, 14, 42 thì gần như chắc chắn bài của bạn có cấu trúc Catalan — một ví dụ đẹp cho heuristic “làm ví dụ nhỏ rồi tra dãy số” ở chương A/01.

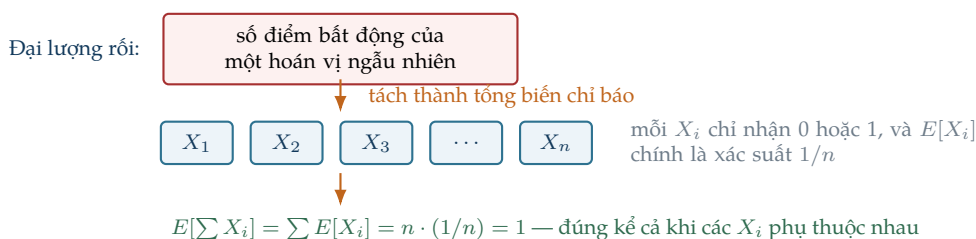
Nguyên lý chuồng bồ câu tầm thường về phát biểu nhưng mạnh về hệ quả, vì nó chứng minh sự tồn tại mà không cần chỉ ra vật thể. Ví dụ dùng trong thuật toán: trong $n+1$ số bất kỳ từ 1 tới $2n$, luôn có hai số mà số này chia hết số kia; hoặc trong một dãy n^2+1 số phân biệt luôn có dãy con đơn điệu độ dài $n+1$. Cách dùng thực dụng nhất: chứng minh rằng một cấu trúc lặp lại phải xuất hiện sau một số bước nhất định — đó chính là cơ sở của thuật toán tìm chu trình ở chương C/20.

5.3 Xác suất và kỳ vọng: công cụ mạnh nhất là công cụ đơn giản nhất

Nếu chỉ được giữ một công cụ xác suất cho thuật toán, đó là *kỳ vọng tuyến tính*: kỳ vọng của tổng bằng tổng các kỳ vọng, $E[X+Y] = E[X] + E[Y]$, **kể cả khi X và Y phụ thuộc nhau**. Câu cuối là chỗ làm nó mạnh, và cũng là chỗ người mới không tin.

Cách dùng chuẩn gồm ba bước: viết đại lượng cần tính thành *tổng của các biến chỉ báo*; tính kỳ vọng của từng biến chỉ báo, mà kỳ vọng đó chỉ là xác suất; rồi cộng lại. Ví dụ: số điểm bất động trung bình của một hoán vị ngẫu nhiên. Đặt $X_i = 1$ nếu phần tử i đứng đúng chỗ. Ta có $E[X_i] = 1/n$, và các X_i rõ ràng phụ thuộc nhau — nhưng không sao: $E[\sum X_i] = n \cdot (1/n) = 1$. Trung bình đúng một điểm bất động, bất kể n lớn thế nào. Tính trực tiếp bằng cách xét phân phối thì rắc rối hơn nhiều bậc.

Cùng kỹ thuật này giải thích chi phí trung bình của quicksort ngẫu nhiên: đặt biến chỉ báo cho sự kiện “phần tử thứ i và thứ j từng được so sánh với nhau”, tính xác suất của nó là $2/(|i-j|+1)$, rồi cộng — và tổng điều hoà xuất hiện, cho $\Theta(n \log n)$.



Hình 5.2. Quy trình ba bước của kỳ vọng tuyến tính. Điểm làm nó mạnh bất thường: phép cộng kỳ vọng không cần bất kỳ giả thiết độc lập nào, nên một đại lượng có phân phối rất rối vẫn tính được bằng cách chẻ thành nhiều câu hỏi có/không rất dễ.

Ba bất đẳng thức đuôi, xếp theo thứ tự cần thêm giả thiết và cho kết luận mạnh hơn: *Markov* chỉ cần biến không âm, cho chặn yếu; *Chebyshev* cần phương sai, cho chặn tốt hơn; *Chernoff* cần tổng các biến độc lập, và cho chặn giảm theo hàm mũ. Chernoff là lý do vì sao “lặp lại thuật toán ngẫu nhiên k lần rồi lấy đa số” làm xác suất sai giảm cực nhanh — nền của cả chương C/20.

CẦN NHỚ

Ba mẫu toán quay lại nhiều nhất trong 23 chương còn lại: (i) tổng cấp số nhân tăng $\leq$ hai lần số hạng lớn nhất; (ii) $H_n \approx \ln n$, tăng nhưng cực chậm; (iii) kỳ vọng tuyến tính đúng kể cả khi các biến phụ thuộc. Nếu chỉ mang theo ba thứ từ chương này, hãy mang ba thứ đó.

5.4 Số học modulo: vì sao mọi bài đếm đều lấy dư

Bài đếm hay cho kết quả khổng lồ — số cách xếp có thể là 2^{10^5} — nên đề bài yêu cầu kết quả theo modulo một số nguyên tố lớn, thường là $10^9 + 7$ hoặc 998 244 353. Đây không phải để làm khó mà để bài toán vẫn có ý nghĩa mà không cần số nguyên lớn.

Bốn điều cần nhớ. Cộng, trừ, nhân đều “đi xuyên qua” phép lấy dư: $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$. Phép chia thì không, và phải thay bằng nhân với *nghịch đảo modulo*, chỉ tồn tại khi số chia nguyên tố cùng nhau với m — đây là lý do người ta chọn m nguyên tố. Lũy thừa tính bằng bình phương liên tiếp, $O(\log n)$ phép nhân. Và cẩn thận tràn số: với $m \approx 10^9$, tích hai số dư đã cỡ 10^{18} , vừa sát giới hạn của số nguyên 64 bit — một chỗ sai kinh điển. Chi tiết ở chương D/21.

5.5 Đại số rời rạc và trò chơi: hai mảnh nhỏ nhưng hay dùng

Lũy thừa ma trận biến các bài “tính trạng thái sau 10^{18} bước” thành khả thi, với điều kiện quan hệ truy hồi là *tuyến tính*: nếu $v_{k+1} = Av_k$ thì $v_n = A^n v_0$, và A^n tính được bằng bình phương liên tiếp trong $O(d^3 \log n)$. Đây là câu trả lời cho câu hỏi mở đầu thứ năm: cấu trúc làm bài giải được chính là tính tuyến tính. Cùng ý tưởng áp cho đếm đường đi độ dài k trong đồ thị, vì lũy thừa ma trận kể đếm đúng thứ đó.

Đại số trên $\mathbb{F}_2$ — cộng là phép XOR — cho phép giải hệ phương trình tuyến tính bằng các thao tác bit, rất nhanh. Ứng dụng hay gặp: tìm tập con có XOR lớn nhất, kiểm tra một số có biểu diễn được thành XOR của tập con hay không (dùng “cơ sở tuyến tính” của tập số).

Trò chơi tổ hợp. Với các trò chơi hai người, thông tin đầy đủ, không hoà, người không đi được thì thua: mỗi thế cờ là *thua* nếu mọi nước đi dẫn tới thế *thắng* cho đối thủ, và ngược lại. Với trò chơi tách được thành nhiều thành phần độc lập, định lý Sprague–Grundy nói giá trị của cả trò chơi là XOR các giá trị Grundy của từng phần — đó là lý do trò Nim giải bằng XOR các đồng. Điều đáng nhớ về mặt phương pháp: cách tìm ra quy luật này luôn là tính tay các thế nhỏ rồi tìm mẫu, đúng heuristic số một ở chương A/01.

5.6 Gốc rễ: những mẫu toán này ra đời để giải bài gì

Điều thú vị là gần như mọi mảnh toán trong chương này đều ra đời từ một bài toán rất cụ thể, thường là rất trần tục, chứ không từ nhu cầu trừu tượng.

Xác suất bắt đầu từ cờ bạc: trao đổi thư giữa Pascal và Fermat năm 1654 về cách chia tiền cược khi một ván bị bỏ dở giữa chừng. Khái niệm kỳ vọng ra đời chính từ câu hỏi đó — “ván này đáng giá bao nhiêu nếu dừng bây giờ” — và đó vẫn là cách hiểu đúng nhất về kỳ vọng khi bạn dùng nó để phân tích thuật toán ngẫu nhiên.

Tổng điều hoà và hằng số γ đi kèm đến từ Euler, trong bối cảnh tính toán các chuỗi vô hạn thế kỷ 18. Hệ số nhị thức thì cổ hơn nhiều, xuất hiện độc lập ở nhiều nền toán học — Ấn Độ, Ba Tư, Trung Quốc — trước khi mang tên Pascal ở châu Âu; tam giác Pascal được biết tới ở Trung Quốc dưới tên tam giác Dương Huy từ thế kỷ 13.

Bất đẳng thức Chernoff, thứ hiện đại nhất trong danh sách, ra đời năm 1952 trong thống kê — bài toán là kiểm định giả thuyết, cần biết xác suất một tổng lớn các quan sát lệch xa kỳ vọng nhỏ tới đâu. Ngành thuật toán mượn nó vào thập niên 1980 khi thuật toán ngẫu nhiên trở nên phổ biến, và ngày nay nó có mặt ở khắp nơi từ băm (B/08) tới học máy.

Cuối cùng, một gốc rễ đáng nhớ vì nó nói lên cách toán và thuật toán tương tác: *Concrete Mathematics*[30] của Graham, Knuth và Patashnik ra đời từ chính chương “Toán học chuẩn bị” trong TAOCP. Knuth thấy rằng để phân tích thuật toán tới hằng số, ông cần một bộ công cụ toán mà các giáo trình toán thời đó không dạy theo cách dùng được — thiên về tổng, truy hồi và tiệm cận. Nói cách khác, nhu cầu phân tích thuật toán đã tạo ra một môn toán riêng, chứ không chỉ mượn.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1654 — Pascal, Fermat	Chia tiền cược thế nào khi ván bạc bị bỏ dở.	Khái niệm kỳ vọng; nền của toàn bộ phân tích thuật toán ngẫu nhiên (C/20).
Thế kỷ 13–17 — tam giác Dương Huy / Pascal	Đếm số cách chọn trong tổ hợp và trong khai triển nhị thức.	Đẳng thức Pascal — cũng chính là công thức truy hồi quy hoạch động đầu tiên trong lịch sử (C/16).
Thế kỷ 18 — Euler	Tính các chuỗi vô hạn và xấp xỉ tổng bằng tích phân.	$H_n \approx \ln n + \gamma$; kỹ thuật chặn tổng bằng tích phân, dùng suốt trong phân tích vòng lặp.
1730 — Stirling, de Moivre	Xấp xỉ $n!$ cho các bài toán xác suất với n lớn.	$\log(n!) = \Theta(n \log n)$ — cận dưới của sắp xếp so sánh dựa trực tiếp vào đây (D/24).
1952 — Chernoff	Kiểm định giả thuyết: xác suất một tổng lệch xa kỳ vọng.	Chặn đuôi giảm theo hàm mũ; lý do “lặp lại rồi lấy đa số” hiệu quả đến thế trong thuật toán ngẫu nhiên.
1989 — Graham, Knuth, Patashnik	Phân tích thuật toán tới hằng số cần bộ công cụ toán mà giáo trình thời đó không dạy theo cách dùng được.	<i>Concrete Mathematics</i> : tổng, truy hồi, tiệm cận trở thành một môn riêng sinh ra từ nhu cầu của ngành thuật toán.

5.7 Liên hệ ngang: cùng công cụ, khác miền

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thống kê và học máy	Bất đẳng thức tập trung, cận tổng quát hoá, biến chỉ báo	Cùng bộ công cụ đuôi (Markov, Chebyshev, Chernoff), chỉ khác câu hỏi: ở đây ta chặn xác suất thuật toán chạy chậm, ở đó chặn xác suất mô hình sai trên dữ liệu mới.
Vật lý thống kê	Hàm phân hoạch, đếm trạng thái vi mô	Cả hai đều đếm số cấu hình rồi lấy tổng có trọng số. Kỹ thuật “đếm bù” và bao hàm–loại trừ xuất hiện ở cả hai. Khác: vật lý quan tâm giới hạn nhiệt động, thuật toán quan tâm chi phí tính chính xác con số đó.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Lý thuyết thông tin	Entropy, cận dưới cho mã hoá	$\log_2(\text{số khả năng})$ là số bit tối thiểu; đây chính là dạng thông tin của cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh (D/24). Cùng một phép đếm, hai cách phát biểu.
Mật mã	Số học modulo, lũy thừa nhanh, nghịch đảo	Cùng công cụ ở D/21, nhưng đảo ngược mục tiêu: ở đây ta muốn tính nhanh, ở đó người ta dựa vào việc <i>bài toán ngược</i> không tính nhanh được.
Tài chính định lượng	Kỳ vọng và định giá theo giá trị kỳ vọng	Cùng khái niệm ra đời năm 1654 cho cùng loại câu hỏi “ván này đáng giá bao nhiêu”. Khác: tài chính phải quan tâm cả phương sai và rủi ro đuôi, còn phân tích thuật toán trung bình thường chỉ cần kỳ vọng — trừ khi có hạn kỳ, lúc đó đuôi mới là thứ quyết định.
Lý thuyết trò chơi kinh tế	Chiến lược tối ưu, cân bằng	Trò chơi tổ hợp ở mục 6 là trường hợp rất hẹp: hai người, thông tin đầy đủ, tổng bằng không. Kinh tế học tổng quát hơn nhiều nhưng mất đi lời giải sạch kiểu Sprague–Grundy.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chúng có bản chất khác hẳn. $1 + 2 + 4 + \dots + n$ là cấp số nhân tăng, và mỗi số hạng lớn hơn tổng của tất cả số hạng trước cộng lại, nên tổng bị chặn bởi hai lần số hạng cuối — vẫn là $O(n)$. Còn $1 + 1/2 + \dots + 1/n$ giảm quá chậm: các số hạng không giảm theo cấp số nhân mà theo tỉ lệ nghịch, và tổng của chúng phân kỳ, chỉ có điều phân kỳ chậm như $\ln n$. Hình 5.1 cho thấy điều đó bằng diện tích.
- A2. Đúng vì kỳ vọng là một tổng có trọng số theo xác suất, và phép lấy tổng thì hoán đổi được với phép lấy tổng — không cần bất kỳ giả thiết độc lập nào. Mạnh vì nó tách một bài toán rối thành nhiều bài rất dễ: bạn không cần biết phân phối của tổng, chỉ cần biết xác suất của từng sự kiện nhỏ. Ví dụ số điểm bất động của hoán vị ngẫu nhiên: các biến chỉ báo phụ thuộc chằng chịt, nhưng mỗi cái có kỳ vọng $1/n$, nên tổng bằng 1.
- A3. Vì kết quả thật thường lớn hơn mọi kiểu số nguyên có sẵn, mà bài toán lại không muốn ép bạn cài số lớn. Lấy dư giữ được cấu trúc cần thiết: cộng, trừ, nhân đều đi xuyên qua phép lấy dư. Chọn số nguyên tố vì khi đó mọi số khác 0 đều có nghịch đảo, nên phép chia — cần cho công thức có giai thừa — vẫn dùng được. Riêng 998 244 353 còn được chọn vì nó có căn nguyên thủy với bậc là lũy thừa lớn của 2, điều kiện để chạy biến đổi Fourier trên số nguyên (D/21).
- A4. Vì nó là một trong số ít công cụ chứng minh sự tồn tại mà không cần chỉ ra vật thể. Sức mạnh nằm ở việc *chọn chuồng* — bước sáng tạo thật sự. Với bài “trong $n + 1$ số từ 1 tới $2n$ luôn có hai số chia hết nhau”, chuồng là “phần lẻ lớn nhất của mỗi số”, và chỉ có n phần lẻ khả dĩ. Chọn được chuồng đúng thì kết luận thành hiển nhiên; toàn bộ độ khó nằm ở bước chọn đó.
- A5. Khi phép chuyển trạng thái là *tuyến tính*, tức trạng thái mới là tổ hợp tuyến tính của trạng thái cũ. Lúc đó n bước là phép nhân ma trận lũy thừa n , tính được bằng bình phương liên tiếp trong $O(\log n)$ phép nhân ma trận. Đó là lý do dãy Fibonacci sau 10^{18} bước tính được còn phần lớn quy hoạch động thì không — và cũng là gợi ý ngược: gấp n lên tới 10^{18} , hãy tìm cách viết truy hồi ở dạng tuyến tính.

NÓI LẠI THẬT ĐƠN GIẢN

Toán trong chương này không phải để chứng minh định lý, mà để trả lời bốn câu hỏi rất đời thường khi bạn viết chương trình.

Câu thứ nhất: “vòng lặp này chạy bao nhiêu lần?” Nếu mỗi lần bạn làm gấp đôi thì tổng công việc chỉ nhiều gấp đôi lần cuối cùng — giống như bạn gấp đôi số tiền tiết kiệm mỗi tháng, tổng tiền cả năm cũng chỉ hơn tháng cuối một chút. Còn nếu mỗi lần bạn chỉ làm bớt đi một phần rất nhỏ, tổng có thể lớn lên chậm mà không bao giờ dừng — đó là tổng điều hoà.

Câu thứ hai: “có bao nhiêu khả năng?” Bốn cách đếm ở mục 2 trả lời gần hết, và điều quan trọng nhất là biết mình đang đếm có thứ tự hay không, có lặp hay không.

Câu thứ ba: “trung bình thì mất bao lâu?” Mẹo duy nhất bạn thực sự cần là: đừng cố tính trung bình của cả cục. Hãy chẻ nó thành nhiều câu hỏi có/không nhỏ xíu, tính xác suất từng cái, rồi cộng. Cộng được kể cả khi chúng ảnh hưởng lẫn nhau — đó là điều bất ngờ và tiện nhất trong cả chương. Câu thứ tư: “số quá lớn thì làm sao?” Lấy dư. Cộng, trừ, nhân vẫn chạy bình thường sau khi lấy dư; chỉ có phép chia là phải làm kiểu khác.

Một câu: Toán ở đây không phải để chứng minh định lý, mà để trả lời “bao nhiêu lần”, “bao nhiêu cách”, “trung bình bao lâu” và “số lớn quá thì sao”.

Chỗ không rút gọn được: Chọn đúng “chuồng” cho nguyên lý chuồng bồ câu, hay chọn đúng biến chỉ báo, vẫn là bước sáng tạo không có công thức.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Chặn cấp số nhân tăng	Tổng $\leq$ hai lần số hạng lớn nhất. Giải thích khẩu hao của mảng động, chi phí xây cây phân đoạn, kỹ thuật nhân đôi.
••• Tổng điều hoà	$H_n \approx \ln n$: phân kỳ nhưng cực chậm. Xuất hiện ở sàng nguyên tố, quicksort ngẫu nhiên, bài sưu tập đủ bộ.
••• Kỳ vọng tuyến tính	$E[\sum X_i] = \sum E[X_i]$ kể cả khi phụ thuộc. Quy trình: viết thành tổng biến chỉ báo $\rightarrow$ mỗi kỳ vọng là một xác suất $\rightarrow$ cộng.
••• Bốn công thức đếm nền	Có thứ tự không lặp $n!/(n-k)!$; không thứ tự không lặp $\binom{n}{k}$; có lặp có thứ tự n^k ; có lặp không thứ tự $\binom{n+k-1}{k}$.
••• Bốn quy tắc modulo	Cộng/trừ/nhân đi xuyên qua phép lấy dư; chia phải dùng nghịch đảo; lũy thừa bằng bình phương liên tiếp; coi chừng tràn khi nhân hai số cỡ 10^9 .
•• Đẳng thức Pascal	$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ — “phần tử thứ n được chọn hoặc không”; khuôn mẫu của mọi lập luận chia trường hợp trong quy hoạch động.
•• Dãy 1, 2, 5, 14, 42	Số Catalan — dãy ngoặc đúng, cây nhị phân, chia đa giác thành tam giác. Gặp dãy này khi tính tay thì bài có cấu trúc Catalan.
•• Ba bất đẳng thức đuôi	Markov (chỉ cần không âm, yếu nhất); Chebyshev (cần phương sai); Chernoff (cần độc lập, chặn giảm theo hàm mũ).
•• Khi nào lũy thừa ma trận dùng được	Khi truy hồi tuyến tính; cho $O(d^3 \log n)$ với n tới 10^{18} . Cũng dùng để đếm đường đi độ dài k trong đồ thị.
•• Stirling và hệ quả	$\log_2(n!) = \Theta(n \log n)$ — cơ sở trực tiếp của cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh.
•• Nguyên lý chuồng bồ câu	Chứng minh tồn tại mà không cần chỉ ra vật thể; toàn bộ độ khó nằm ở bước chọn chuồng.
• Sprague–Grundy	Trò chơi tách thành phần độc lập: giá trị chung là XOR các giá trị Grundy; Nim là trường hợp riêng.
• Cơ sở tuyến tính trên $\mathbb{F}_2$	Cho phép hỏi “số này có là XOR của một tập con không” và “XOR lớn nhất là bao nhiêu” trong $O(n \log C)$.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao phân tích quicksort ngẫu nhiên lại ra tổng điều hoà?

| Vì ta đếm bằng biến chỉ báo cho sự kiện “hai phần tử ở vị trí i và j trong thứ tự đã sắp có được so sánh với nhau hay không”. Chúng chỉ được so sánh nếu một trong hai được chọn làm chốt trước mọi phần tử nằm giữa chúng, xác suất là $2/(j-i+1)$. Cộng theo mọi cặp thì mỗi khoảng cách d đóng

góp cỡ $n \cdot 2/(d+1)$, và tổng theo d chính là tổng điều hoà — cho $2n \ln n$. Đây là ví dụ mẫu mực cho quy trình ba bước ở mục 3.

2. Khi nào tôi nên đếm bù thay vì đếm trực tiếp?

| Khi điều kiện “tốt” có dạng “không có cái nào vi phạm” còn điều kiện “xấu” dễ mô tả. Ví dụ đếm hoán vị không có điểm bất động thì khó, nhưng đếm hoán vị có ít nhất một điểm bất động lại làm được bằng bao hàm–loại trừ. Dấu hiệu nhận biết trên đề: có chữ “không”, “ít nhất một”, “khác nhau đôi một”.

3. Tôi luôn nhầm giữa $\binom{n+k-1}{k}$ và $\binom{n}{k}$. Có cách nhớ nào không?

| Hãy nhớ mô hình vật lý thay vì công thức. Chia k viên kẹo giống nhau cho n đứa trẻ: xếp k viên kẹo thành hàng và cắm $n-1$ vách ngăn vào; mỗi cách sắp xếp k viên và $n-1$ vách là một cách chia, tổng cộng $\binom{n+k-1}{k}$ cách. Còn $\binom{n}{k}$ là chọn k đứa trẻ trong n đứa. Hai bài khác nhau ở chỗ vật được chia có phân biệt hay không.

4. Chernoff và Chebyshev khác nhau trên thực tế bao nhiêu?

| Rất nhiều, và khoảng cách lớn dần theo số phép thử. Chebyshev cho xác suất lệch giảm theo $1/t^2$; Chernoff cho giảm theo e^{-t} . Với 100 lần tung đồng xu, hỏi xác suất được từ 70 mặt ngửa trở lên: Chebyshev cho một chặn lỏng cỡ vài phần trăm, Chernoff cho chặn nhỏ hơn nhiều bậc. Cái giá là Chernoff đòi các biến độc lập, còn Chebyshev chỉ cần phương sai — nên khi có phụ thuộc thì Chebyshev vẫn dùng được còn Chernoff thì không.

5. Tại sao $10^9 + 7$ lại phổ biến đến thế?

| Ba lý do thực dụng. Nó là số nguyên tố, nên mọi số khác 0 có nghịch đảo. Nó nhỏ hơn 2^{30} , nên tích hai phần dư nhỏ hơn 2^{60} , vừa trong số nguyên 64 bit có dấu. Và nó đủ lớn để hai kết quả khác nhau hiếm khi trùng phần dư một cách tình cờ. Còn 998 244 353 được chọn khi bài cần biến đổi Fourier trên số nguyên, vì $998\,244\,353 = 119 \cdot 2^{23} + 1$ có căn nguyên thủy bậc 2^{23} .

6. Nếu tôi kém toán, tôi có bị chặn trần trong việc luyện thuật toán không?

| Không bị chặn ở phần lớn nội dung. Cấu trúc dữ liệu, đồ thị, quy hoạch động, tham lam — chiếm phần lớn bài tập và phỏng vấn — chỉ cần đúng những gì trong chương này, mà phần lõi chỉ có ba mẫu ở khối *Cần nhớ*. Toán trở thành rào cản thật sự ở ba nơi hẹp: số học nâng cao (D/21), một số bài đếm tổ hợp khó, và phân tích thuật toán ngẫu nhiên tinh vi. Chiến lược hợp lý là học ba mẫu lõi thật chắc rồi bổ sung theo nhu cầu, chứ không học toán trước rồi mới bắt đầu.

7. Cách nhanh nhất để nhận ra một dãy số quen thuộc khi tính tay ví dụ nhỏ?

| Tính năm tới sáu số hạng đầu rồi so với vài dãy thường gặp: 2^n ; $n!$; Fibonacci 1,1,2,3,5,8; Catalan 1,1,2,5,14,42; số Stirling; số Bell 1,1,2,5,15,52 (chú ý dễ nhầm với Catalan ở ba số đầu). Nếu không khớp, hãy thử tra trong từ điển dãy số nguyên trực tuyến. Nhưng nhớ: nhận ra dãy mới là gợi ý, vẫn phải chứng minh vì sao bài của bạn cho ra dãy đó.

8. Có nên học hàm sinh không?

| Chỉ khi bạn đi sâu vào các bài đếm. Hàm sinh là công cụ đẹp và mạnh cho việc giải truy hồi và đếm cấu trúc, nhưng trong phần lớn bài tập thực tế thì cây đệ quy và quy hoạch động đã đủ. Xếp nó vào loại “biết là có, học khi gặp bài thật cần” — và khi đó *Concrete Mathematics*[30] là nguồn đúng.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Tính tổng $\sum_{i=1}^n i$, $\sum 2^i$, $\sum 1/i$ và xác định bậc của từng cái	Tự luyện	Mục tiêu là phản xạ: nhìn vòng lặp ra ngay bậc, không phải nhớ công thức.
• Tính $\binom{n}{k} \bmod p$ cho n tới 10^6 bằng bảng giai thừa và nghịch đảo	Chương D/21	Tiền xử lý giai thừa và nghịch đảo giai thừa một lần; mỗi truy vấn $O(1)$.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tính số điểm bất động trung bình của hoán vị ngẫu nhiên bằng mô phỏng, so với lý thuyết	Thực nghiệm	Kết quả bằng 1 với mọi n — hãy tự kiểm chứng rồi giải thích bằng kỳ vọng tuyến tính.
•• Đếm số dãy ngoặc đúng độ dài $2n$ bằng hai cách: quy hoạch động và công thức Catalan	AtCoder DP, CSES[33]	Hai cách phải khớp; nếu lệch, gần như luôn do quên trường hợp biên $n = 0$.
•• Tính Fibonacci thứ 10^{18} theo modulo bằng lũy thừa ma trận	Bài tập kinh điển	Chú ý thứ tự nhân ma trận và tràn số khi nhân hai phần tử.
•• Chứng minh trong $n + 1$ số bất kỳ từ 1 tới $2n$ luôn có hai số mà số này chia hết số kia	Bài tập kinh điển	Chuồng là “phần lẻ lớn nhất”; toàn bộ bài nằm ở bước chọn chuồng.
•• Bài sưu tập đủ bộ: cần trung bình bao nhiêu lần mở gói để có đủ n loại thẻ	Bài tập kinh điển	Chia thành n giai đoạn, mỗi giai đoạn là biến hình học; tổng cho nH_n .
••• Chứng minh quicksort ngẫu nhiên có kỳ vọng $2n \ln n$ so sánh	[40]	Biến chỉ báo cho từng cặp, xác suất $2/(j - i + 1)$; đây là bài tập kinh điển nhất về kỳ vọng tuyến tính.
••• Dựng cơ sở tuyến tính trên $\mathbb{F}_2$ và tìm XOR lớn nhất của một tập con	Bài tập thi đấu kinh điển	Khử Gauss trên bit; sau khi có cơ sở, thuật toán tham lam từ bit cao xuống bit thấp.
••• Tính giá trị Grundy cho một trò chơi lấy sỏi có luật lạ, tìm quy luật	Bài tập kinh điển	Tính tay các thế nhỏ, lập bảng, tìm chu kỳ — đúng quy trình heuristic số một ở A/01.
••• Dùng bao hàm–loại trừ đếm số hoán vị không có điểm bất động (số mất trật tự)	Bài tập kinh điển	Kết quả xấp xỉ $n!/e$; một kết quả đẹp và bất ngờ đáng tự dẫn ra.

Kết chương và đọc tiếp

Chương này khép lại phần A. Nếu ba chương trước cho bạn quy trình, thước chi phí và thước tính đúng, thì chương này là cái hộp dụng cụ đặt cạnh bàn — không cần thuộc hết, chỉ cần biết trong đó có gì và mở đúng ngăn khi cần. Ba mẫu quay lại nhiều nhất, nhắc lại lần cuối: cấp số nhân bị chặn bởi hai lần số hạng lớn nhất, tổng điều hoà xấp xỉ $\ln n$, và kỳ vọng tuyến tính không cần độc lập.

Từ đây, phần B đổi câu hỏi. Bốn chương vừa rồi hỏi “tôi nghĩ thế nào, và làm sao biết mình đúng”. Bảy chương tiếp theo hỏi một câu rất khác: *dữ liệu nên nằm ở dạng nào để thao tác mà tôi lặp lại nhiều nhất trở nên rẻ*. Bắt đầu bằng thứ đơn giản nhất và cũng bị đánh giá thấp nhất — mảng, cùng những kỹ thuật hai con trở biến $O(n^2)$ thành $O(n)$ (B/06).

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, “The Role of Deliberate Practice in the Acquisition of Expert Performance,” *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [7] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [10] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [11] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [12] C. A. R. Hoare, “An Axiomatic Basis for Computer Programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] D. Gries, *The Science of Programming*. Springer, 1981.
- [14] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [15] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [16] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [17] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [18] e-maxx / cp-algorithms contributors. “cp-algorithms: Algorithms for Competitive Programming.” (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [19] C. A. R. Hoare, “Quicksort,” *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [20] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [21] R. E. Tarjan, “Amortized Computational Complexity,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [22] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [23] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [24] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.

- [25] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.
- [26] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [34] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [35] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [36] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [37] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [38] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [39] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [40] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, "The Role of Deliberate Practice in the Acquisition of Expert Performance," *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [7] KTH Royal Institute of Technology. "KACTL: KTH Algorithm Competition Template Library." (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [10] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [11] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [12] C. A. R. Hoare, "An Axiomatic Basis for Computer Programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] D. Gries, *The Science of Programming*. Springer, 1981.
- [14] J. L. Bentley and M. D. McIlroy, "Engineering a Sort Function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [15] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [16] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [17] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [18] e-maxx / cp-algorithms contributors. "cp-algorithms: Algorithms for Competitive Programming." (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [19] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [20] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [21] R. E. Tarjan, "Amortized Computational Complexity," *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [22] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [23] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [24] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.

- [25] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.
- [26] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [34] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [35] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [36] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [37] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [38] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [39] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [40] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.